

Adaptive Design of Embodied Robotic Systems for Real-Time Self-Modeling Toward Swarm Coordination

Master's Thesis

Muhammad Zeeshan Asghar
Master's Program in Data Science
HSE University

2026

Abstract

Embodied robotic systems deployed in unstructured environments inevitably experience embodiment drift—the progressive divergence between a robot’s internal kinematic model and its physical hardware caused by mechanical wear, calibration slip, thermal deformation, and unmodeled impacts. When a robot’s internal model no longer matches its physical embodiment, traditional Jacobian-based inverse kinematics controllers fail catastrophically: end-effectors diverge from targets, oscillations destabilize the system, and contact-rich tasks generate unsafe forces. Maintaining true autonomy requires self-modeling—the ability to infer one’s own morphology directly from sensorimotor experience.

Contemporary visual self-modeling architectures based on Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) achieve high morphological expressiveness but suffer from a critical latency paradox. A diagnostic self-model is only valuable if it can provide state estimates fast enough to influence the feedback controller. Modern robotic control loops commonly operate at 1 kHz, permitting a maximum latency of 1.0 millisecond per inference. Existing neural rendering self-models operate at 5–37 FPS—latencies of 27–192 milliseconds—rendering them unusable for real-time control integration. This represents a fundamental architectural mismatch: volumetric reconstruction is computationally incompatible with sub-millisecond control deadlines.

This thesis resolves this bottleneck by introducing NeuroKin, a lightweight fully convolutional direct sensorimotor decoder that completely bypasses explicit 3D volumetric reconstruction. NeuroKin maps joint angles directly to visual silhouettes, achieving 21.88 dB PSNR at 7,400 FPS (0.135 ms latency)—a $1,418\times$ speedup over implicit NeRF baselines with superior fidelity. Training completes in 33.5 seconds on a Tesla T4 GPU. An auxiliary multi-task variant, ResNeuroKin-D, recovers geometric depth while maintaining 2,500 FPS (0.400 ms latency). Integration of NeuroKin’s spatial estimates into a Jacobian-based feedback loop enables dynamic error recovery after severe fault injection, demonstrating that sub-millisecond visual self-modeling can actively stabilize a damaged manipulator.

Building on this individual diagnostic capability, we scale to collective fault tolerance via temporal stigmergy—a mechanism of indirect coordination wherein agents leave environmental traces that stimulate adaptive responses in subsequent agents. In a 100-stage sequential swarm under progressive fault injection, the standard inverse-kinematics baseline collapses from 100% pre-fault success to 13% post-fault success. Conversely, NeuroKin-enabled agents read and write a shared failure ledger, autonomously trigger localized recovery

modes, and preserve 88% systemic success.

This thesis provides four primary contributions: (1) formal benchmarking of the latency paradox in neural rendering self-models, (2) the NeuroKin direct decoding architecture achieving sub-millisecond inference, (3) visual closed-loop damage adaptation, and (4) empirical validation that fast individual self-modeling enables decentralized stigmergic coordination—the computational prerequisite for physical swarm deployment.

Acknowledgements

This thesis represents the culmination of a deeply fulfilling academic journey, and it would not have been possible without the profound support of my mentors, colleagues, and family.

First and foremost, I gratefully acknowledge the guidance, patience, and critical feedback of my academic supervisor and the thesis defence committee at the Higher School of Economics (HSE) Master's Program in Data Science. Your rigorous standards and technical insights have fundamentally shaped my approach to research. Reflecting on my academic development, the foundational engineering principles I acquired during my Bachelor of Engineering in Mechatronics, Robotics, and Automation Engineering with a minor in Computing from the National University of Sciences and Technology (NUST) have been absolute prerequisites in grounding my deep learning models in real-world physical and kinematic boundaries.

I also extend my sincere appreciation to my co-founders and colleagues at LangMeet AI. Serving as Technical Lead while managing real-time speech translation infrastructure concurrently with my thesis workload has been a formidable challenge, but it provided an invaluable sandbox for investigating computational latency profiles, hardware-level optimization, and production bottlenecks.

On a deeply personal level, I owe an immense debt of gratitude to my parents. Your shared childhood observations, continuous encouragement, and steadfast support have been my ultimate cognitive anchor. To my sister, whose empathy, kindness, and unyielding dedication to rescuing and adopting stray cats inspire me daily to notice harmony in chaotic systems. To my brother, whose background studying classical violin provided a beautiful, resonant acoustic environment that acted as the backdrop to countless late-night debugging and writing sessions. I am also deeply thankful to my aunt for her kind nature, warmth, and steady reassurance throughout my master's timeline.

Finally, I must acknowledge the quiet but essential companionship of my cat, Alec, who spent countless hours keeping watch beside my workstations while these neural networks were designed, trained, and evaluated.

As this Master's exploration draws to a close, I look forward to expanding this foundation. The synthetic experiments and decentralized control frameworks validated herein represent an indispensable stepping stone. Translating these direct sensorimotor networks from controlled physical constraints into adaptive physical deployments on multi-robot platforms will form the explicit technical objective of my upcoming Ph.D. program.

The author confirms that AI tools were used only to assist with grammar, vocabulary, and to enhance narrative flow; all technical content, experimental design, analyses, and conclusions remain the author's original work. Supplementary materials and the implementation of the NeuroKin project are available at: <https://github.com/YoloPopo/neurokin>.

Contents

Abstract

Acknowledgements

List of Figures

List of Tables

1	Introduction	1
1.1	The Challenge of Embodiment Drift	1
1.1.1	Manifestations of Embodiment Drift	2
1.1.2	Consequences for Control	2
1.2	The Evolution of Self-Modeling Paradigms	2
1.3	The Latency Paradox in Neural Rendering	3
1.3.1	Temporal Constraints of Robotic Control	3
1.3.2	The Computational Bottleneck	3
1.4	Problem Statement and Core Hypothesis	4
1.4.1	Formal Mathematical Problem Statement	4
1.4.2	The Central Hypothesis	4
1.4.3	Empirical Scope, Constraints, and Hardware Realities	5
1.4.4	The Temporal Stigmergy Framework for Swarm Coordination	5
1.5	Research Questions	5
1.5.1	RQ1: Architectural Efficiency	6
1.5.2	RQ2: Control Integration	6
1.5.3	RQ3: Fault Tolerance	6
1.5.4	RQ4: Collective Adaptation	6
1.6	Methodological Contributions	6
1.6.1	Contribution 1: Formal Benchmarking of the Latency Paradox	6
1.6.2	Contribution 2: The NeuroKin Direct Sensorimotor Decoder	6
1.6.3	Contribution 3: Visual Closed-Loop Damage Adaptation	7
1.6.4	Contribution 4: Temporal Stigmergy for Sequential Swarm Coordination	7
1.7	Thesis Organization	7

2	Literature Synthesis	8
2.1	Evolution of Self-Modeling	8
2.1.1	The Symbolic Era: Estimation-Exploration	8
2.1.2	The Behavioral Era: Avoidance Over Adaptation	9
2.1.3	Task-Agnostic Learned Dynamics Models	9
2.1.4	Proprioceptive Morphology Identification	10
2.1.5	The Visual Era: Deep Self-Models and Neural Rendering	10
2.2	Neural Rendering Foundations: The Speed-Quality Tradeoff	11
2.2.1	Neural Radiance Fields: Implicit Scene Representation	11
2.2.2	3D Gaussian Splatting: Explicit Geometric Primitives	12
2.2.3	The Failure of Anisotropy on Robotic Morphologies	13
2.2.4	Articulated Object Reconstruction for Digital Twin Generation	14
2.2.5	Physics-Consistent Robot Models and Manipulation Planning	14
2.2.6	Cross-Embodiment Transfer and Morphological Understanding	14
2.2.7	Sim-to-Real Transfer via Digital Twins	14
2.3	Damage Recovery and Fault Tolerance	15
2.3.1	Detection Methods and Early Intervention	15
2.3.2	Recovery Policy Generation and Adaptation	16
2.3.3	Humanoid Recovery and Stability	16
2.3.4	Practical Deployment Lessons	16
2.4	Continual Learning for Self-Model Maintenance	16
2.4.1	Balancing Plasticity and Stability	17
2.4.2	Resource-Efficient Adaptation via Sparsity	17
2.4.3	Safety-Constrained and Bounded Adaptation	18
2.5	Theoretical Foundations of Swarm Coordination and Temporal Stigmergy	18
2.5.1	Stigmergy: Biological and Algorithmic Foundations	18
2.5.2	Temporal Stigmergy in Sequential Pipelines	19
2.5.3	Sub-Millisecond Inference as a Swarm Prerequisite	19
2.6	Summary of Literature Findings	20
2.7	Research Questions Grounded in Literature	21
2.7.1	RQ1: Architectural Efficiency	21
2.7.2	RQ2: Control Integration	21
2.7.3	RQ3: Fault Tolerance	21
2.7.4	RQ4: Collective Adaptation	21
3	Baseline Implementations and the Latency Bottleneck	23
3.1	Synthetic Data Generation via Lorenz Attractors	23
3.1.1	Simulation Environment Configuration	23
3.1.2	Lorenz Attractor Trajectory Generation	25
3.1.3	Data Collection and Preprocessing	27
3.1.4	Dataset Statistics and Quality Analysis	28

3.2	Free-Form Kinematic Self-Model (FFKSM): Implicit Neural Rendering . . .	29
3.2.1	Core Mathematical Framework	29
3.2.2	Implementation Details and Reproduction Challenges	31
3.2.3	FFKSM Training Protocol	34
3.2.4	FFKSM Performance Metrics	34
3.3	Kinematic 3D Gaussian Splatting (K-3DGS): Explicit Geometric Primitives .	36
3.3.1	Mathematical Foundations of 3D Gaussian Splatting	36
3.3.2	Rendering with 3D Gaussian Splatting	36
3.3.3	K-3DGS Architecture	36
3.3.4	The Anisotropic Failure: A Critical Architectural Lesson	38
3.3.5	K-3DGS Performance Metrics	39
3.4	Comparative Analysis of Baseline Limitations	40
3.4.1	The Latency-Quality Tradeoff	40
3.4.2	Supervision Efficiency Analysis	40
3.4.3	Architectural Bottlenecks Summary	40
4	The NeuroKin Direct Sensorimotor Decoder	42
4.1	Core Hypothesis: Bypassing 3D Reconstruction	42
4.1.1	The Task-Relevant Information Hierarchy	42
4.1.2	The Supervision Efficiency Argument	43
4.1.3	When is 3D Reconstruction Necessary?	43
4.2	Architecture Design	44
4.2.1	Stage 1: Joint Encoder	46
4.2.2	Stage 2: Spatial Expansion	46
4.2.3	Stage 3: Transposed Convolutional Decoder	47
4.2.4	Stage 4: Output Head and Spatial Upsampling	48
4.2.5	Total Parameter Count	49
4.3	Stereoscopic 3D Recovery: NeuroKin-3D	49
4.3.1	Dual-Camera Data Generation	49
4.3.2	The Geometry of Stereoscopic Silhouette Triangulation	50
4.3.3	NeuroKin-3D Architecture	51
4.3.4	Multi-Task Training Loss	52
4.3.5	Local Overfit Test	52
4.4	Training Strategy and Regularization	52
4.4.1	Base NeuroKin Training	52
4.4.2	NeuroKin-3D Training	53
4.5	Performance Evaluation	53
4.5.1	Base NeuroKin: 2D Silhouette Prediction	53
4.5.2	NeuroKin-3D: Stereoscopic 3D Recovery	53
4.5.3	ResNeuroKin-D: Multi-Task Depth Prediction	53
4.6	Ablation Studies	53

4.7	Supervision Efficiency Analysis Revisited	53
4.8	Qualitative Results	54
4.8.1	Base NeuroKin Silhouette Prediction	54
4.8.2	NeuroKin-3D End-Effector Prediction	54
4.9	Discussion: The 3D Reconstruction Bottleneck	56
4.10	Limitations of Direct Decoding	56
4.11	Summary	56
5	Visual Closed-Loop Control and Fault Adaptation	57
5.1	Controller Integration: Resolved-Rate Inverse Kinematics with Visual Feedback	57
5.1.1	Task-Space Control Formulation	57
5.1.2	Damped Least-Squares Inverse Kinematics	58
5.1.3	Visual Feedback Loop with NeuroKin-3D	58
5.1.4	Exponential Smoothing for Noise Reduction	60
5.1.5	Baseline: Pure Proprioceptive Control	60
5.2	Experimental Protocol	60
5.2.1	Task Definition: Point-to-Point Reaching	60
5.2.2	Nominal Performance Evaluation	61
5.2.3	Fault Injection: Proprioceptive Encoder Slip	61
5.2.4	Fault Magnitude Sweep	62
5.3	Stability Analysis of the Visual Servo Controller	62
5.3.1	Lyapunov Stability for Proportional Control	62
5.3.2	Effect of the Integral Term	62
5.3.3	Effect of Visual Estimation Error	62
5.4	Results: Nominal Performance	63
5.5	Results: Fault-Tolerant Performance	65
5.5.1	Encoder Slip at Step 50	65
5.5.2	Quantitative Results Across 100 Trials	67
5.5.3	Fault Magnitude Sweep	67
5.5.4	Joint Angle Tracking Under Fault	67
5.6	Analysis: Why Visual Estimates Enable Recovery	68
5.6.1	The Critical Role of Sub-Millisecond Latency	69
5.6.2	Comparison to Prior Work on Fault Tolerance	69
5.7	Extension to Structural Damage	69
5.7.1	Damage Configuration	69
5.7.2	Visual Estimation Under Structural Damage	70
5.7.3	Fine-Tuning for Rapid Adaptation	70
5.7.4	Closed-Loop Performance Under Structural Damage	70
5.8	Failure Case Analysis	71
5.8.1	Category 1: Extreme Fault Magnitude (38% of failures)	71
5.8.2	Category 2: Workspace Exit (41% of failures)	71

5.8.3	Category 3: Silhouette Degradation (21% of failures)	71
5.9	Discussion: Implications for Real-World Deployment	72
5.9.1	Sim-to-Real Considerations	72
5.9.2	Comparison to Traditional Visual Servoing	73
5.10	Summary	73
6	Sequential Swarm Coordination under Compounding Faults	74
6.1	The Sequential Swarm Pipeline: Temporal Stigmergy in Operation	75
6.1.1	Definition of Temporal Stigmergy	75
6.1.2	The 100-Stage Operational Chain	75
6.1.3	Progressive Fault Injection	76
6.2	The Inter-Agent Communication Ledger as Stigmergic Channel	77
6.2.1	The Factory State Dictionary	77
6.2.2	The Recovery Threshold Mechanism	78
6.2.3	Comparison to Biological Stigmergy	78
6.3	State-Dependent Collective Adaptation: Experimental Results	79
6.3.1	Baseline: Standard IK Controller (No Stigmergy)	79
6.3.2	NeuroKin with Temporal Stigmergy	79
6.3.3	Quantitative Comparison: 100-Stage Results	79
6.3.4	Analysis of Baseline Catastrophic Failure	80
6.3.5	Analysis of NeuroKin Stigmergic Adaptation	80
6.3.6	Step Efficiency Analysis	80
6.4	Statistical Analysis and Failure Mode Characterization	81
6.4.1	Survival Analysis	81
6.4.2	Failure Mode Categorization	81
6.4.3	Stigmergic Efficiency: Information-Theoretic Analysis	81
6.5	Comparison to Prior Work on Swarm Coordination	81
6.5.1	Explicit Communication vs. Stigmergy	81
6.5.2	Comparison to Multi-Agent Reinforcement Learning	82
6.6	Limitations and Failure Case Analysis	82
6.6.1	Sequential, Not Concurrent	82
6.6.2	Information Bottleneck	82
6.6.3	Fault Model Limitations	82
6.6.4	Recovery Mode Conservatism	83
6.7	Discussion: Validating the “Toward Swarm Coordination” Hypothesis	83
6.7.1	Why Temporal Stigmergy Proves the Prerequisite	83
6.7.2	The Role of Fast Inference in Swarm Coordination	83
6.7.3	Generalization to Physical Swarms	83
6.8	Summary	84
7	Discussion	85

7.1	Trade-offs of Direct Decoding: Sacrificing Novel-View Synthesis for Extreme Inference Speed	85
7.1.1	The Single-Viewpoint Constraint	85
7.1.2	Binary Silhouettes vs. RGB Prediction	86
7.1.3	The Pareto Frontier: Novel-View Synthesis vs. Control-Loop Latency	86
7.1.4	Supervision Efficiency as the Key Explanatory Variable	87
7.2	Validating the “Toward Swarm Coordination” Hypothesis: Temporal Stigmergy as a Prerequisite	88
7.2.1	Why Temporal Stigmergy Proves the Prerequisite	88
7.2.2	The Role of Sub-Millisecond Inference in Stigmergic Coordination	88
7.2.3	Comparison to Alternative Coordination Mechanisms	89
7.2.4	Limitations of the Sequential Paradigm	89
7.2.5	Information Bottleneck and Recovery Mode Design	90
7.3	Threats to Validity and Directions for Mitigation	90
7.3.1	Simulation-to-Reality Gap	90
7.3.2	Abstraction of Mechanical Degradation and Rigid-Body Assumptions	91
7.3.3	Generalization to Higher Degrees of Freedom	91
7.3.4	Catastrophic Forgetting Under Structural Damage	92
7.3.5	Data Efficiency and Limited Training Samples	92
7.4	Summary of Discussion	92
8	Conclusion and Future Work	94
8.1	Summary of Contributions and Research Findings	94
8.1.1	RQ1: Architectural Efficiency — Direct Decoding Defeats the Latency Paradox	94
8.1.2	RQ2: Control Integration — Sub-Millisecond Visual Servoing Enables Dynamic Stabilization	95
8.1.3	RQ3: Fault Tolerance — Rapid Fine-Tuning Enables Adaptation to Structural Damage	96
8.1.4	RQ4: Collective Adaptation — Temporal Stigmergy Validates the Prerequisite for Swarm Coordination	96
8.2	Technical Contributions Summary	97
8.2.1	Contribution 1: Formal Benchmarking of the Latency Paradox	97
8.2.2	Contribution 2: The NeuroKin Direct Sensorimotor Decoder	97
8.2.3	Contribution 3: Visual Closed-Loop Damage Adaptation	98
8.2.4	Contribution 4: Temporal Stigmergy for Sequential Swarm Coordination	98
8.3	Future Roadmap: From Sequential Validation to Spatial Swarms	98
8.4	Concluding Remarks	98

List of Figures

3.1	PyBullet simulation setup for the 4-DOF manipulator, including the robot model, target marker, and virtual camera frusta used for synthetic data generation.	24
3.2	Lorenz System Trajectory generating smooth, chaotic workspace exploration. The attractor produces deterministic non-repeating paths that densely cover the robot's configuration space.	27
3.3	End-to-end data generation and preprocessing pipeline. RGB images from PyBullet are converted to grayscale and thresholded to produce binary silhouettes for training.	28
3.4	Representative robot configurations from the dataset, demonstrating diverse poses including vertical extension, diagonal reach, and angled configurations.	29
3.5	FFKSM target versus render showing visual discrepancies. Volumetric rendering produces blurry depth predictions; errors concentrate at joint boundaries and link extremities where sampling density is insufficient.	35
3.6	K-3DGS render artifacts and spatial difference map. Isotropic spherical Gaussians create overlapping "blobby" artifacts that blur link boundaries.	39
3.7	K-3DGS training metrics showing loss convergence (top), FPS (middle), and step time (bottom). Inference speed stabilizes at 37 FPS after initial optimization.	40
4.1	NeuroKin convolutional decoder architecture. Joint-angle inputs are encoded into a latent representation, spatially expanded, and decoded through transposed convolution layers into a predicted visual self-model.	45
4.2	Dual-view data generation sample. Synchronized front and side camera silhouettes are concatenated to provide the stereoscopic evidence used by NeuroKin-3D.	50
4.3	Base NeuroKin qualitative validation results. Target silhouettes, NeuroKin renders, and difference maps show that direct decoding recovers link geometry with localized errors near joint boundaries and the end-effector.	54
4.4	NeuroKin-3D validation coordinates. Predicted x , y , and z traces closely follow the ground-truth end-effector positions across the validation sequence.	55

4.5	NeuroKin-3D validation diagnostics: Euclidean end-effector error, predicted versus target 3D trajectory, and train-validation loss. The three panels show both spatial accuracy and stable optimization.	56
5.1	Closed-loop stereoscopic perception-control tracking loop topology.	59
5.2	Representative target-current end-effector error in 3D Cartesian space. The dashed segment visualizes the Euclidean distance minimized by the closed-loop reaching controller.	61
5.3	Baseline error convergence over time steps. Both NeuroKin and pure proprioceptive controllers converge smoothly to sub-centimeter error within 50 steps under nominal conditions. The shaded region indicates ± 1 standard deviation across 100 trials.	64
5.4	Representative closed-loop end-effector trajectory from start position to target position. The smooth path confirms that the visual state estimate supports stable task-space motion.	65
5.5	Dynamic error recovery following encoder slip injected at Step 50. The baseline pure proprioceptive controller diverges catastrophically, while NeuroKin's visual estimates enable rapid re-convergence to the target.	66
5.6	Closed-loop joint-angle error over time. All four joint estimates converge toward zero, supporting the use of NeuroKin-3D joint outputs in the Jacobian feedback loop.	68
6.1	Temporal stigmergy execution block passing discrete environmental updates over time.	77
6.2	Algorithm Efficiency in Steps under injected faults. The IK baseline fails structurally (capped at 201 steps), while the NeuroKin swarm reads shared states and adapts behavior, maintaining efficiency between 20-40 steps. . . .	80
7.1	Pareto frontier illustrating the optimal trade-off between model accuracy (PSNR) and computational speed (FPS). NeuroKin occupies the upper-right region, achieving both superior quality and sub-millisecond latency. The red dashed line indicates the 1,000 FPS threshold for 1 kHz control loops.	87

List of Tables

3.1	Denavit-Hartenberg parameters for the 4-DOF manipulator.	25
3.2	Effect of robot pixel weight on FFKSM performance.	33
3.3	FFKSM quantitative performance metrics.	35
3.4	Baseline self-modeling architecture performance comparison.	40
4.1	NeuroKin parameter distribution by component.	49
5.1	Nominal performance comparison (no faults, mean $\pm$ std over 100 trials). . .	63
5.2	Fault-tolerant performance comparison (100 trials, random fault magnitude $\pm 5^\circ$ to $\pm 20^\circ$).	67
6.1	Sequential swarm performance comparison (100 stages, progressive fault injection).	80

Chapter 1

Introduction

1.1 The Challenge of Embodiment Drift

Autonomous robots deployed in remote, unstructured environments—planetary rovers, disaster-response machines, deep-sea manipulators, and industrial inspection drones—must operate for extended periods without human intervention. These systems cannot rely on frequent manual recalibration, hardware replacement, or human-in-the-loop supervision. They must be self-sufficient, adaptive, and resilient in the face of inevitable physical degradation.

A fundamental requirement for such autonomy is maintaining an accurate internal representation of one’s own morphology, kinematics, and dynamics. Traditional robotic systems rely on predefined kinematic models—typically specified in Unified Robot Description Format (URDF) files—that encode link lengths, joint limits, inertial properties, and collision geometries. These models enable classical control algorithms, such as Jacobian-based inverse kinematics and operational space control, to map intended high-level actions into precise joint torques or positional trajectories. Under ideal conditions, with a perfectly calibrated robot operating in a controlled environment, these methods achieve millimeter-level precision.

However, the assumption that a physical robot remains mathematically identical to its idealized digital model is fundamentally flawed under continuous operational deployment. Physical embodiment subjects robots to the relentless realities of thermodynamics, material fatigue, and mechanical friction. Over extended operational cycles—spanning hours, days, or months of continuous operation—the strict mathematical guarantees of forward and inverse kinematics inevitably decouple from physical reality.

This persistent, accumulating divergence between the internal computational representation of the robot and its actual physical state is formally defined in this thesis as *embodiment drift*. The term captures the essential phenomenon: the robot’s sense of its own body drifts away from truth as hardware degrades.

1.1.1 Manifestations of Embodiment Drift

Embodiment drift manifests across a broad spectrum of mechanical and sensory degradations. At the micro-scale, geared transmissions develop unmodeled backlash as teeth wear; actuator drive belts lose optimal tension through creep and fatigue; thermal expansion induced by sustained high-torque operations alters the effective physical length of metallic linkages by micrometers—enough to matter in precision tasks. At the sensory layer, proprioceptive joint encoders may imperceptibly slip from their calibrated zero-points due to vibration or electrical noise; exteroceptive sensors, such as chassis-mounted RGB cameras or depth sensors, experience extrinsic calibration shifts from ambient thermal cycling or minor physical impacts.

In more severe, sudden operational scenarios, robots may suffer macroscopic structural damage: an unexpected collision may plastically deform a physical link by several degrees; an actuator may seize entirely due to electrical failure or debris ingress; an asymmetrical payload shift—such as a grasped object suddenly changing weight distribution—may fundamentally alter the system’s center of mass and dynamic response.

1.1.2 Consequences for Control

When significant embodiment drift occurs, the consequences for closed-loop feedback control are catastrophic. A controller operating under the assumption of a nominal kinematic tree will compute pseudo-inverse Jacobian updates that are mathematically sound relative to the model but physically erroneous relative to reality. The system issues corrective motor commands that inadvertently drive the end-effector away from the target, induce highly unstable resonant oscillations, or, in contact-rich manipulation tasks, exert unsafe, unbounded forces upon the environment. In legged locomotion, embodiment drift leads to gait asymmetry, reduced stability margins, and eventual falls. In humanoid robots operating near humans, the safety implications are even more severe.

To maintain true autonomy and operational safety in the presence of embodiment drift, a robotic system must possess the capacity for *self-modeling*—the ability to autonomously infer, update, and deploy an internal representation of its own morphology derived strictly from its own sensorimotor experience, entirely bypassing the need for external, human-in-the-loop manual recalibration.

1.2 The Evolution of Self-Modeling Paradigms

The formidable challenge of autonomous self-modeling has driven several distinct research paradigms over the past two decades. Early approaches relied heavily on symbolic inference and active physical exploration, utilizing continuous cycles of hypothesis generation to deduce morphological changes. Subsequent paradigms shifted toward behavioral adaptation, bypassing damage diagnosis to search pre-computed behavioral repertoires for com-

pensatory actions. The advent of deep learning introduced task-agnostic learned dynamics models, while parallel threads explored proprioceptive morphology identification to restore embodiment using internal signals alone.

Most recently, the field has entered the visual era, leveraging neural rendering to learn self-models directly from camera observations. Architectures based on implicit coordinate networks like the Free-Form Kinematic Self-Model (FFKSM) and explicit primitives like Kinematic 3D Gaussian Splatting (K-3DGS) achieve high morphological expressiveness. However, as this thesis establishes, the immense representational power of these neural rendering architectures introduces a severe computational bottleneck. A detailed chronological analysis of these paradigms and their respective mathematical limitations is provided in Chapter 2.

1.3 The Latency Paradox in Neural Rendering

A diagnostic self-model is operationally valuable only if it can provide state estimates fast enough to influence the stabilizing behavior of the feedback controller. If the model requires hundreds of milliseconds to generate a prediction, the information arrives too late—the robot has already executed several control cycles based on stale assumptions.

1.3.1 Temporal Constraints of Robotic Control

Modern robotic control systems operate at tightly constrained frequencies. Force control, impedance control, and dynamic balancing typically run at 1 kHz, requiring the entire perception-planning-actuation pipeline to execute within 1.0 millisecond.

Within the 1.0 ms window, the entire software stack must read physical sensor buffers, execute state estimation, compute target Cartesian error vectors, resolve complex inverse kinematic equations, enforce physical joint limits, and issue subsequent motor torque commands. Consequently, any self-modeling diagnostic component integrated into this inner control loop possesses an absolute maximum inference budget of a fraction of a millisecond.

1.3.2 The Computational Bottleneck

Current state-of-the-art neural rendering architectures fundamentally violate this latency budget. Implicit NeRF-based models achieve frame rates of approximately 5 FPS, corresponding to 191.6 ms latency. Explicit 3DGS-based architectures reach approximately 37 FPS, corresponding to 27.0 ms latency. Both exceed the 1.0 ms control budget by factors of 27 to 190—rendering them entirely unsuitable for real-time control integration.

The root cause is architectural. Both approaches maintain an explicit or implicit 3D volumetric representation. NeRF requires evaluating a deep network hundreds of thousands of times per image via ray-marching—a fundamentally serial process. 3DGS requires computing forward kinematics for thousands of Gaussian primitives, projecting them into 2D

screen space, sorting them by depth, and alpha-compositing—a heavily sequential pipeline despite GPU acceleration.

1.4 Problem Statement and Core Hypothesis

The core scientific problem addressed by this thesis is the resolution of the latency paradox in visual robotic self-modeling, and the subsequent leveraging of this high-speed diagnostic capability to establish the computational mechanisms for multi-agent swarm coordination under compounding mechanical failures.

1.4.1 Formal Mathematical Problem Statement

We formally define the true physical morphology of the robot at time t as the configuration manifold $\mathcal{M}(t)$. The robotic system maintains an internal computational kinematic representation $\hat{\mathcal{M}}(t)$. Embodiment drift is defined as the non-stationary, cumulative divergence between the physical reality and the digital expectation:

$$D(t) = \|\mathcal{M}(t) - \hat{\mathcal{M}}(t)\| \quad (1.1)$$

When an unmodeled kinematic fault $\Delta \mathbf{q}_{\text{fault}}$ occurs, the proprioceptive encoder estimate $\mathbf{q}_{\text{enc}}$ deviates from the true physical state $\mathbf{q}_{\text{true}}$ such that $\mathbf{q}_{\text{true}} = \mathbf{q}_{\text{enc}} + \Delta \mathbf{q}_{\text{fault}}$. This breaks traditional closed-loop control.

Let the sensory observation space be $\mathcal{O}$ (in this case, binary visual silhouettes) and the joint configuration space be $\mathcal{Q}$. The objective of the visual self-model is to learn a direct sensorimotor mapping $f_\theta : \mathcal{Q} \rightarrow \mathcal{O}$ that minimizes the reconstruction loss $\mathcal{L}(f_\theta(\mathbf{q}), \mathbf{o})$ to recover the true Cartesian end-effector position $\mathbf{x}_{\text{true}}$.

Crucially, this mapping is strictly bound by control-loop temporal dynamics. If the control system operates at frequency f_{ctrl} , dictating a strict maximum latency constraint $T_{\text{max}} = 1/f_{\text{ctrl}}$, and t_{infer} is the inference time of the self-model, the system is physically deployable for closed-loop stabilization if and only if:

$$t_{\text{infer}} \leq T_{\text{max}} \quad (1.2)$$

For a standard 1 kHz feedback loop, this necessitates $t_{\text{infer}} \leq 1.0$ ms.

1.4.2 The Central Hypothesis

This thesis investigates a central hypothesis: *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling*. We propose a paradigm shift toward *direct sensorimotor decoding*: a lightweight fully convolutional network that maps joint angles directly to visual silhouettes and spatial coordinates, completely bypassing the intermediate 3D reconstruction step. We hypothesize that such an architecture can achieve sub-millisecond

inference latency—compatible with 1 kHz control budgets—while maintaining reconstruction fidelity comparable to or exceeding volumetric baselines, precisely because dense pixel-wise supervision is more efficient than sparse ray sampling.

1.4.3 Empirical Scope, Constraints, and Hardware Realities

The empirical scope of this research is deliberately bounded within a highly controlled, synthetic PyBullet simulation environment. This is a deliberate necessity. Implementation on actual physical hardware is not possible within the scope of this Master’s thesis due to severe, practical resource limitations. The rigorous validation of dynamic fault adaptation requires dedicated, damageable physical manipulators where structural faults can be safely and repeatedly injected. Furthermore, acquiring ground-truth millimetric spatial coordinate data to validate the visual network’s predictions requires expensive, high-speed motion-capture rigs (e.g., Vicon systems). Given the strict temporal and hardware constraints of the degree program, physical deployment is unattainable at this stage. Consequently, the transition from simulated architectural validation to physical hardware deployment—including addressing the complex photometric noise of the Sim-to-Real gap—is formally deferred to the author’s upcoming Ph.D. research.

1.4.4 The Temporal Stigmergy Framework for Swarm Coordination

This thesis evaluates multi-agent coordination via *temporal stigmergy*. Stigmergy is a biological mechanism of indirect coordination observed in social insects such as termites and ants. Individual agents do not communicate directly; instead, they alter the environment in ways that stimulate adaptive responses in subsequent agents. In this thesis, the shared “environment” is algorithmically abstracted into a persistent digital memory ledger—the `factory_state` variable.

This state-dependent, history-aware behavioral adaptation is the absolute core logic of decentralized swarm coordination. Proving that sub-millisecond individual self-diagnosis enables such temporal coordination validates the computational prerequisite for eventual physical swarms.

1.5 Research Questions

Four research questions guide this investigation, progressing from architectural efficiency through control integration and fault tolerance to collective adaptation.

1.5.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR?

1.5.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop?

1.5.3 RQ3: Fault Tolerance

How does the integration of sub-millisecond visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as severe encoder slip and structural damage?

1.5.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination?

1.6 Methodological Contributions

This thesis makes four primary contributions:

1.6.1 Contribution 1: Formal Benchmarking of the Latency Paradox

The first contribution is the formal benchmarking of the latency paradox in neural rendering, exposing their control incompatibility relative to the 1 ms control budget.

1.6.2 Contribution 2: The NeuroKin Direct Sensorimotor Decoder

The second contribution is the NeuroKin architecture. NeuroKin is a lightweight, fully convolutional direct sensorimotor decoder that completely bypasses volumetric reconstruction, mapping joint angles directly to visual silhouettes. NeuroKin achieves 21.88 dB PSNR at 7,400 FPS (0.135 ms latency)—a $1,418\times$ speedup over implicit NeRF baselines.

1.6.3 Contribution 3: Visual Closed-Loop Damage Adaptation

The third contribution lies in visual closed-loop control and dynamic damage adaptation. This research successfully synthesizes NeuroKin’s sub-millisecond spatial estimates with a Jacobian-based inverse-kinematics feedback loop, allowing the controller to re-stabilize the end-effector trajectory post-fault.

1.6.4 Contribution 4: Temporal Stigmergy for Sequential Swarm Coordination

The fourth contribution is the validation of stigmergic swarm coordination. The NeuroKin-enabled 100-stage sequential swarm maintains 88% success post-fault compared to the 13% success rate of the baseline.

1.7 Thesis Organization

The remainder of this thesis is structured to systematically address the research questions. Chapter 2 provides an exhaustive critical synthesis of relevant literature. Chapter 3 details the empirical simulation methodology and latency bottleneck, outlining the PyBullet data generation pipeline. Chapter 4 formally introduces the core technical innovation—the NeuroKin architecture. Chapter 5 moves from static rendering metrics to dynamic control, outlining the integration of NeuroKin into an inverse-kinematics loop. Chapter 6 scales validation to the collective level, unpacking the 100-stage operational swarm chain and the mechanics of the inter-agent communication ledger. Chapter 7 evaluates the overarching hypothesis, Pareto trade-offs, and threats to validity. Finally, Chapter 8 synthesizes the research findings, directly answers the formal research questions, and maps a concrete technological roadmap for transitioning these validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

Chapter 2

Literature Synthesis

2.1 Evolution of Self-Modeling

The capacity for autonomous adaptation to physical damage is the defining characteristic of biological resilience, yet it remains an elusive goal for robotic systems. This section treats the mathematical and conceptual structures underpinning robotic self-modeling across three distinct historical blocks—symbolic exploration, behavioral detour maps, and modern deep neural rendering methods—before isolating the foundational speed ceilings and system stability challenges motivating our work.

2.1.1 The Symbolic Era: Estimation-Exploration

The earliest formalized paradigm relied heavily on symbolic and kinematic inference. The seminal work of Bongard, Zykov, and Lipson established the first complete closed-loop self-modeling system on actual physical robots [?]. Their approach viewed self-modeling as a continuous cycle of hypothesis generation and active physical exploration. The robot maintained a population of competing symbolic models representing various candidate topologies—different possible configurations of its own body. To discriminate among these hypotheses, the robot synthesized exploratory motor commands mathematically designed to maximize the sensory disagreement among the candidate models. Upon executing these actions, the robot collected sensor data and systematically pruned models that failed to predict observed outcomes. After approximately 16 modeling-testing cycles spanning several minutes of real-world experimentation, the robot converged on an accurate damaged model and synthesized a compensatory gait through reinforcement learning on the internal simulation.

The core innovation was an active learning loop: the fitness of a test action was determined by its ability to cause disagreement among the best candidate models, thereby maximally reducing uncertainty. While conceptually elegant, this approach suffers from the symbolic bottleneck: the robot can only discover models constructible from predefined physics engine primitives (cylinders, hinge joints). It cannot represent non-rigid deformations, soft-body dynamics, or arbitrary visual damage—limitations that become critical in unstructured

environments where failure modes are unpredictable [?]. As noted by Kwiatkowski and Lipson, the search space of symbolic physics engines is discrete and non-differentiable, making optimization slow and prone to local optima [?]. Furthermore, the approach relied heavily on simple proprioceptive sensors providing limited information compared to high-bandwidth vision.

2.1.2 The Behavioral Era: Avoidance Over Adaptation

Recognizing the computational intensity of explicit physical modeling, Cully and colleagues introduced Intelligent Trial and Error (IT&E), which bypassed damage diagnosis entirely [?]. Their key insight was to focus on finding what still works rather than diagnosing what went wrong. Using the MAP-Elites quality-diversity algorithm [?], they pre-computed a behavioral repertoire of approximately 13,000 distinct gaits in simulation—a process requiring 40 million simulations over two weeks of computation. This repertoire is a high-dimensional grid where each cell corresponds to a specific behavioral descriptor (e.g., duty cycle versus leg offset) and stores the highest-performing controller found for that descriptor.

When damaged in the field, the robot treated this map as a Bayesian prior, using Gaussian Process regression to search for a behavior that performs well in the real world. By testing a few dozen distinct behaviors, the Gaussian Process updated its posterior mean and variance, rapidly identifying a compensatory gait. Cully and colleagues demonstrated recovery in less than two minutes of real-world testing across various damage scenarios, including removal of entire legs [?].

However, IT&E represents a strategy of avoidance, not adaptation. The robot does not repair its internal model—it discards the parts of the behavioral space that no longer function. While effective for redundant locomotion tasks where multiple gaits can achieve similar forward velocity, this approach fails for precision manipulation tasks that require exact geometric awareness [?]. If a robot needs to reach through a narrow aperture, it cannot simply “try a different gait”—it must understand the precise Cartesian geometry of its damaged arm. Furthermore, as robotic complexity increases, the curse of dimensionality makes pre-computing dense behavioral maps prohibitively expensive: a 6-DOF arm with even modest discretization would require billions of map entries, far exceeding computational budgets.

2.1.3 Task-Agnostic Learned Dynamics Models

The advent of deep learning enabled a third paradigm: learning dynamics models that encode the mapping from motor commands to sensory consequences. Kwiatkowski and Lipson proposed task-agnostic self-modeling, training deep forward models to predict the future state of the robot given the current state and action [?]. This represented a pivotal shift from symbolic physics to learnable physics. The same approach was demonstrated to work as a complete reinforcement learning environment to learn zero-shot tasks across

morphologies in a follow-up study [?].

While theoretically powerful, these data-driven models face a fundamental limitation: systematic bias accumulates progressively across extended planning horizons [?]. Simulated trajectories diverge increasingly from actual robot behavior, rendering the approach impractical for tasks requiring accurate prediction beyond very short time windows. This horizon-dependent degradation motivates approaches that prioritize shorter planning windows or use learned corrections rather than pure forward simulation.

2.1.4 Proprioceptive Morphology Identification

A parallel thread of research focuses on proprioceptive identification—restoring embodiment using internal signals alone, without the occlusions and lighting failures that often cause vision-only pipelines to break [?]. This offers significant deployment advantages: the approach remains robust across diverse lighting conditions and operates without calibrated cameras. However, eliminating external visual information imposes real functional costs: expressiveness is substantially reduced compared to vision-based methods [?]. This is not merely a theoretical gap but a genuine operating constraint that limits the complexity of tasks these systems can support compared to their multi-modal counterparts.

Meta-self-modeling restores morphology parameters between motion signature reconstructions [?], and IMU-centric methods show hardware-realized damage sensing with minimal sensing overhead on legged robots [?]. Analytical modular pipelines remain strong baselines in the case of known assembly constraints, offering interpretable, stable, and automatically generated kinematic and dynamic models [?]. The frontier is intervention, not just identification: controlled breakage of proprioception-driven morphology demonstrates that active structural adaptation is supported by internal sensing, and that morphology itself is a form of adaptive feedback control [?, ?].

2.1.5 The Visual Era: Deep Self-Models and Neural Rendering

The current state-of-the-art leverages neural rendering to learn visual self-models directly from camera observations. Chen and colleagues introduced occupancy query learning as a foundational approach [?]. Signed Distance Functions (SDFs) conditioned on joint angles enable differentiable collision checking directly from RGB-D sensor streams, supporting both motion planning and recovery from damage. Their multi-view hardware system achieved approximately one percent workspace accuracy and demonstrated support for planning and post-damage recovery. However, practical deployment introduced significant challenges: careful camera placement became critical for observability, precise calibration was required for geometric accuracy, and performance degraded substantially under occlusion, shadows, blur, and noise [?].

Hu, Lin, and Lipson reduced the camera system to a single RGB sensor while learning the Free-Form Kinematic Self-Model (FFKSM) from a single viewpoint [?]. The reparameter-

ization of camera frames in terms of the first two joints facilitated the learning process. This approach supports inverse kinematics, path planning, and post-damage recovery. However, volumetric rendering via NeRF remained computationally expensive, and single-view training introduced persistent vulnerabilities: occlusion sensitivity and perspective-dependent predictions.

More recently, Kinematic 3D Gaussian Splatting (K-3DGS) has emerged as an explicit alternative to implicit NeRFs [?]. 3DGS replaces ray marching with differentiable rasterization of explicit anisotropic Gaussians, enabling high frame rate rendering and fast optimization. By attaching explicit Gaussian primitives to a differentiable kinematic chain and leveraging rasterization-based rendering rather than ray-marching, K-3DGS promises faster inference. Each Gaussian primitive is defined by a 3D mean position, covariance matrix, opacity, and color, allowing the model to render the robot in any arbitrary pose through standard graphics pipeline operations.

Yet, as this thesis establishes, the immense representational power of these neural rendering architectures has introduced a severe computational bottleneck that completely obstructs physical deployment in real-time control loops.

2.2 Neural Rendering Foundations: The Speed-Quality Tradeoff

The computational bottleneck of visual self-modeling stems fundamentally from the choice of 3D representation. This section examines the mathematical foundations of Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS), the two dominant paradigms for neural scene representation, while also surveying the broader landscape of articulated object reconstruction and digital twin generation.

2.2.1 Neural Radiance Fields: Implicit Scene Representation

Neural Radiance Fields (NeRFs), introduced by Mildenhall and colleagues, revolutionized computer vision by representing scenes as continuous functions mapping 5D coordinates to color and volume density [?]. The function $F_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ is parameterized by a multi-layer perceptron (MLP), where $\mathbf{x} \in \mathbb{R}^3$ is a spatial position, $\mathbf{d} \in \mathbb{R}^3$ is a viewing direction, $\mathbf{c} \in \mathbb{R}^3$ is the emitted color, and $\sigma \in \mathbb{R}^+$ is the volume density.

To render a pixel, NeRF approximates the volume rendering integral along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $\mathbf{d}$ is the ray direction. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (2.1)$$

where $T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$ is the accumulated transmittance. In practice, this continuous integral is approximated via numerical quadrature using stratified sampling.

For a standard 100×100 image with $N = 64$ samples per ray, this formulation requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame. At 5.22 FPS, this amounts to 3.2 million MLP evaluations per second—a computational burden that our subsequent implementations directly address.

To capture high-frequency geometric details, NeRF employs positional encoding, mapping input coordinates into a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0\pi p), \cos(2^0\pi p), \dots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)] \quad (2.2)$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$ -dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

Standard NeRF assumes a static scene. For articulated robots, the scene geometry changes with joint configuration. Dynamic variants therefore learn a canonical representation together with a deformation field that maps observation-space points to canonical space [?, ?]. For robotic self-modeling, the time variable can be replaced with the joint configuration vector, learning a mapping from configuration space to deformation [?].

Several acceleration techniques have been proposed to address NeRF’s computational cost [?, ?, ?]. However, even with acceleration, the core robotics bottleneck remains fundamentally unchanged: volumetric query expense remains a fundamental constraint within tight control loops.

2.2.2 3D Gaussian Splatting: Explicit Geometric Primitives

3D Gaussian Splatting (3DGS), introduced by Kerbl and colleagues, replaces implicit neural fields with explicit discrete geometric primitives [?]. By abandoning the MLP entirely during inference and relying on highly optimized rasterization pipelines, 3DGS promises a pathway to real-time, high-fidelity scene representation.

Unlike NeRF, which models a continuous field, 3DGS represents a scene as an unstructured point cloud of discrete 3D Gaussian primitives. Each individual Gaussian is mathematically defined by a 3D mean position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, an anisotropic covariance matrix $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3 \times 3}$ (parameterized by scaling $\mathbf{S}$ and rotation $\mathbf{R}$ via $\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top$), an opacity scalar $\alpha_i \in [0, 1]$, and spherical harmonics coefficients representing view-dependent directional color.

The core representational power of 3DGS lies in the anisotropy of the covariance matrix. A purely isotropic (spherical) Gaussian is severely limited in its ability to model sharp edges or flat surfaces. By allowing the covariance matrix to stretch and compress unevenly along its principal axes, a single Gaussian can deform into an elongated ellipsoid or a flattened disk, enabling precise modeling of complex geometry with fewer primitives.

Optimizing a 3×3 covariance matrix directly using gradient descent is mathematically precarious, as the matrix must remain strictly positive semi-definite. To enforce this constraint, Kerbl and colleagues factorize the covariance matrix into a scaling vector $\mathbf{s} \in \mathbb{R}^3$

(with $s_i > 0$) and a rotation quaternion $\mathbf{q} \in \mathbb{R}^4$, then reparameterize the optimization to maintain positive semi-definiteness.

The true speed advantage of 3DGS is derived from its rendering pipeline. To synthesize an image, 3DGS completely abandons ray-marching, instead utilizing a highly parallelized tile-based rasterization architecture. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance Σ' is:

$$\Sigma' = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^\top\mathbf{J}^\top \quad (2.3)$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is $G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}')^\top \Sigma'^{-1}(\mathbf{u} - \boldsymbol{\mu}')\right)$ and the final pixel color is computed via alpha compositing with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^N \alpha_i G_i^{2D}(\mathbf{u}) \mathbf{c}_i \prod_{j=1}^{i-1} (1 - \alpha_j G_j^{2D}(\mathbf{u})) \quad (2.4)$$

The rapid adoption of 3DGS within robotics has pushed it beyond static rendering and toward mapping, tracking, and simulation-oriented pipelines [?, ?, ?, ?]. However, the computational cost still scales with the number of Gaussians, which matters for embedded deployment and any system that must reason about many candidate morphologies at control time.

2.2.3 The Failure of Anisotropy on Robotic Morphologies

Given these computational advantages, the robotics literature adapted 3DGS for kinematic self-modeling. The resulting architecture, Kinematic 3DGS (K-3DGS), attaches Gaussian primitives to a differentiable kinematic tree.

In theory, the anisotropic nature of 3DGS makes it ideal for robotic manipulators. The physical morphology of a serial manipulator is dominated by long, thin, rigid cylindrical links. An optimizer should naturally stretch the scaling vectors of Gaussians along the central longitudinal axis of each link, achieving aspect ratios of 20:1:1.

In practice, however, this anisotropic optimization frequently fails catastrophically when applied to sparse, single-camera robotic datasets. Optimizing highly anisotropic rotational quaternions on featureless robotic links introduces a severely non-convex, ill-posed optimization landscape. Because the robotic link is visually uniform, the gradient descent algorithm receives contradictory loss signals regarding the optimal rotation of the elongated Gaussian. This phenomenon is termed *anisotropic mode collapse*.

To maintain numerical stability and ensure reliable convergence, practical K-3DGS implementations are frequently forced to retreat to purely isotropic representations. The scaling vector is constrained to be equal in all dimensions, forcing every Gaussian primitive to remain a perfect sphere. While this restriction mathematically stabilizes the training process, it fundamentally compromises the structural precision of the representation. Attempting to model a long, sharp-edged robotic cylinder using only perfect spheres results in severe

geometric aliasing. When rasterized, this isotropic chaining yields pervasive, structurally ambiguous "blobby" artifacts that blur the true spatial boundaries of the robot.

2.2.4 Articulated Object Reconstruction for Digital Twin Generation

Automated digital twin generation approaches follow three broad strategies. Differentiable reconstruction pipelines avoid manual URDF creation by optimizing directly from observations. Feed-forward generative pipelines prioritize generation speed over per-instance optimization, and physics-constrained variants add domain structure to reduce planning-critical errors. Across all three, the core trade-off is clear: richer geometry and more automation often come with higher optimization cost, weaker robustness under occlusion, or reduced physical reliability.

For manipulation tasks requiring closed-loop control, explicit world models offer an alternative emphasis: separate the geometric model from the control logic, then preserve only the model fidelity truly needed by the downstream task. This is the same design pressure that motivates NeuroKin’s minimal representation principle.

2.2.5 Physics-Consistent Robot Models and Manipulation Planning

Gaussian twin representations have progressed from display assets toward functional planning models. Planning stacks increasingly merge rendering, geometry, and control, but a persistent limitation remains: strong simulation metrics do not automatically imply robust deployment. Horizon length, environmental robustness, and contact fidelity still determine whether a model generalizes beyond laboratory settings.

2.2.6 Cross-Embodiment Transfer and Morphological Understanding

Cross-embodiment transfer performance improves measurably when morphological structure is encoded as kinematic connectivity rather than visual appearance alone. The broad lesson is that structural priors matter more than texture-level similarity when embodiments differ.

Whole-body control frameworks increasingly leverage these insights for humanoid deployment. Frameworks couple base and arm dynamics for dynamic object grasping with legged manipulators. These frameworks demonstrate that explicit structure priors substantially facilitate transfer across embodiments.

2.2.7 Sim-to-Real Transfer via Digital Twins

Equating rendering fidelity with deployment success is a flawed assumption that has led to numerous simulation-only results failing on hardware. The reality gaps decompose along

four distinct dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment).

Effective transfer combines targeted system identification with methodical robustness training rather than pursuing perfect simulator fidelity. Hardware evidence supports this principle: quadrupeds trained with actuator-aware system identification and domain randomization achieve zero-shot transfer to real platforms. Bicycles trained with explicit robustness-focused procedures transfer reliably across diverse terrain. A consistent pattern emerges across successful systems: calibration without concurrent robustness training fails catastrophically, while robustness training without accurate baseline calibration merely wastes training data without achieving true transfer capability.

Digital twins play multiple roles in sim-to-real pipelines. Dynamics and system identification calibrate physics parameters so simulated rollouts reflect hardware behavior. Perception and rendering transfer reduce visual domain gaps by reconstructing photorealistic scenes or preserving semantic invariants for training and validation. Demonstration synthesis and augmentation generate diverse trajectories and viewpoints from sparse real demonstrations by manipulating reconstructed scene components. Deployment-time synchronization uses a continuously synchronized simulator as a mediator, where the policy acts on the twin while the robot tracks twin trajectories. Evaluation and sim-real correspondence estimate whether simulator performance correlates with hardware outcomes, enabling safer iteration before deployment.

2.3 Damage Recovery and Fault Tolerance

Damage recovery is where self-model claims face the most severe test: can the system operate with real faults, not just under nominal simulation conditions? This section surveys detection methods, recovery policy generation, humanoid-specific challenges, and practical deployment lessons.

2.3.1 Detection Methods and Early Intervention

Early fault detection methods are valuable only when directly tied to practical intervention latency and downstream control outcomes. VAE-based anomaly monitoring successfully detects anomalous force-torque trajectory patterns at sub-3-millisecond latencies in simulated manipulation tasks. Contrastive forward prediction reduces tracking error by up to 61.5% on damaged legged machines. Low-latency detection remains critical because delayed intervention necessarily closes the available recovery window, making response speed a fundamental constraint.

2.3.2 Recovery Policy Generation and Adaptation

Detection must translate to actionable recovery policies. Vision-language recovery can guide task-level correction on 7-DoF manipulators; however, unconstrained language reasoning often violates dynamics limits. Structured recovery stacks provide a pragmatic alternative. Dream2Fix synthesizes counterfactual failures on a large scale (120,000+ simulation cases), achieving 46% zero-shot closed-loop recovery on hardware. Embodiment-conditioned diffusion policies achieve higher constraint satisfaction (36.85% to 74.51%) under zero-shot generalization to unseen failure conditions.

2.3.3 Humanoid Recovery and Stability

Humanoid recovery introduces substantial additional complexity through stability and safety constraints not present in manipulator systems. Balance-informed critic networks grounded in capture point theory and centroidal dynamics computation achieve strong simulated stand-up recovery from falls. Preference-conditioned control policies expand operational robustness envelopes by dynamically switching between efficiency-optimized and robustness-maximized control modes. Fault-aware locomotion methods maintain stable tracking performance under joint faults within simulation. Humanoid parkour research demonstrates impressive agile obstacle vaulting and long-horizon terrain traversal capabilities in controlled environments.

2.3.4 Practical Deployment Lessons

Two practical lessons emerge consistently across successful deployment systems. First, on-line discrepancy monitoring proves effective when adaptation remains rapid and theoretically bounded, preventing unbounded divergence. World-model residual monitoring specifically enables rapid policy recovery following physical domain shifts like payload changes or link damage. Second, residual adaptation channels remain most effective when carefully aligned with baseline policy safety constraints. This principle is demonstrated empirically: bounded residual control reduced Go1 quadruped recovery steps from 4,500 to 168 following a significant mass increase—a 96% reduction in recovery time establishing the practical value of principled multi-level adaptation.

A consistent deployment principle emerges: recovery systems should be evaluated by intervention speed, stability margin, and post-fault task continuity—not by reconstruction fidelity. Visual quality is not the determining factor.

2.4 Continual Learning for Self-Model Maintenance

Continual maintenance of self-models confronts the stability-plasticity dilemma: updates must absorb new damage conditions while preserving knowledge of healthy morphology. Catastrophic forgetting is well documented; in robotic deployment it is critical because forgetting can generate unsafe actions and hardware destruction.

2.4.1 Balancing Plasticity and Stability

Meta-learning approaches contribute meaningful value when morphology characteristics shift frequently and the time window available for adaptation compresses severely. Model-agnostic meta-learning (MAML) reduces post-damage sample requirements for model training, and structure-constrained updates enable adaptation without full retraining. Elastic Weight Consolidation (EWC) mitigates catastrophic forgetting by approximating the posterior distribution of weights using the Fisher information matrix. However, EWC is ill-suited for robotic damage recovery for two reasons. First, computational cost: calculating the Fisher Matrix requires computing second-order derivatives (Hessian) or approximating them via squared gradients. For high-dimensional rendering networks (millions of parameters), storing and computing the Fisher information is prohibitively expensive for real-time adaptation on edge hardware. Second, lack of modularity: EWC slows forgetting but does not enable the robot to isolate the damage. It treats the network as a monolith with varying “stiffness”. If damage is severe, the robot needs to change high-importance weights, but EWC prevents this, leading to under-fitting of the damage (the “plasticity gap”).

The failure of weight-based regularization suggests that topology, rather than weights, may hold the key to adaptive resilience. Research by Chechik and colleagues demonstrated through computational models that synaptic pruning is an optimal strategy strictly under metabolic or resource constraints. They proved that in the absence of such energetic costs, pruning cannot improve network performance compared to an intact network—a finding that aligns precisely with edge compute constraints in robotics, where power budgets (15W typical for Jetson Orin) and memory bandwidth are severely limited. The biological precedent suggests that sparse, modular architectures may be not merely beneficial but necessary for resource-constrained adaptive systems.

2.4.2 Resource-Efficient Adaptation via Sparsity

The Lottery Ticket Hypothesis formalizes the value of sparsity in deep learning: “A randomly-initialized dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations”. This suggests that the vast majority of weights in a dense self-model are redundant. Han and colleagues empirically demonstrated this, achieving compression ratios of $49\times$ on AlexNet and $39\times$ on VGG-16 through a combination of pruning, quantization, and Huffman coding. Model storage reduced from 240 MB to 6.9 MB while maintaining baseline accuracy.

Sparse Evolutionary Training (SET) maintains sparse topology throughout training rather than training dense and pruning post-hoc. In each epoch, SET prunes the weakest connections and regrows random new ones, allowing the network to explore the topology space dynamically. For robotics, sparsity offers dual benefits. First, efficiency: sparse matrix operations reduce computational latency, directly addressing the rendering bottleneck.

Second, modularity: by enforcing sparsity, the network is forced to develop modular substructures. Damage to one module can be repaired by updating only the weights within that module, significantly reducing catastrophic interference. This biological inspiration—sparse, modular connectivity enabling localized plasticity—motivates our architectural explorations in subsequent sections.

2.4.3 Safety-Constrained and Bounded Adaptation

Safety-constrained learning maintains adaptation within verified regions through explicit constraints. Barrier functions enforce that the robot stays within safe regions of state space, providing formal safety guarantees. System-level state partitioning separates high-level reasoning from time-critical control loops, reducing cascading failures. Bounded residual control keeps adaptations within safe envelopes, as demonstrated empirically by the 96% reduction in recovery time on Go1 quadruped.

2.5 Theoretical Foundations of Swarm Coordination and Temporal Stigmergy

This section establishes the theoretical framework of temporal stigmergy that underpins the sequential swarm evaluation in Chapter 6, drawing on biological inspiration and multi-agent systems research.

2.5.1 Stigmergy: Biological and Algorithmic Foundations

Stigmergy is a biological mechanism of indirect coordination, originally observed in social insects such as termites and ants. Individual agents do not communicate directly via explicit signaling protocols or negotiated consensus algorithms. Instead, they subtly alter the physical environment—for example, depositing a pheromone trail or adding a pellet of construction material. These environmental modifications serve as localized, persistent stimuli, triggering specific behavioral responses from subsequent agents that encounter them.

The critical property of stigmergy is that coordination emerges from local interactions without centralized planning or direct agent-to-agent communication. The environment itself becomes the communication channel. This property makes stigmergy particularly attractive for robotic swarms operating in communication-denied environments where radio bandwidth is limited or connectivity is intermittent.

Translated into algorithmic robotic systems, stigmergy eliminates the need for continuous peer-to-peer communication bandwidth. Agents react independently and locally to diagnostic traces left in the environment by their peers. This stands in contrast to coordination schemes that require denser communication or centralized training.

2.5.2 Temporal Stigmergy in Sequential Pipelines

In this thesis, stigmergic coordination is achieved across time rather than physical space. The shared “environment” is abstracted into a persistent digital memory ledger—the `factory_state` variable tracked across sequential agents. This represents a *temporal stigmergy*: agents coordinate by reading and writing to a shared state that persists across sequential operations, with the dimension of coordination being time rather than space.

Formally, we define temporal stigmergy as a coordination mechanism where agents $\{A_1, A_2, \dots, A_N\}$ share a persistent state $S \in \mathcal{S}$. Each agent A_i executes the following protocol:

1. Reads the current state S_{i-1} (the state after the previous agent’s execution).
2. Selects behavior $B_i = \pi(S_{i-1})$ according to a policy $\pi : \mathcal{S} \rightarrow \mathcal{B}$.
3. Executes the behavior and observes outcome $O_i \in \{\text{success}, \text{failure}\}$.
4. Updates the state: $S_i = \Phi(S_{i-1}, O_i)$ where Φ is the state transition function.

Coordination emerges from the sequential application of π and Φ without any direct communication between agents beyond the state S that persists in the shared environment. When an agent experiences repeated mechanical failures, it updates the `consecutive_failures` integer. Subsequent agents read this trace. If the trace indicates a degraded operational history (two consecutive failures), the new agent autonomously shifts into a localized “recovery” mode: it abandons its randomized spatial target, moves to a conservative centralized coordinate, and doubles its Cartesian error tolerance. This state-dependent, history-aware behavioral adaptation is the core logic of decentralized swarm coordination.

2.5.3 Sub-Millisecond Inference as a Swarm Prerequisite

A swarm can only coordinate around diagnostic states if individual agents possess the capability to produce those states reliably and in a timely manner. If an agent’s internal self-model requires 200 ms per inference, it cannot reliably determine whether a failure was caused by occlusion, oscillation, or morphological drift—by the time the diagnosis completes, the system state may have changed. By proving that NeuroKin provides robust, sub-millisecond visual self-diagnosis (0.400 ms latency), this thesis establishes the computational prerequisite for swarm endurance.

Sub-millisecond inference enables three properties essential for stigmergic coordination: real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed); immediate ledger updates (the shared state is updated before the next control cycle begins); and rapid adaptation (subsequent agents read the updated state without perceptible delay). Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

2.6 Summary of Literature Findings

This exhaustive literature synthesis reveals several critical findings that directly motivate the experimental contributions of this thesis.

First, visual self-modeling via neural rendering (NeRF and 3DGS) has achieved impressive reconstruction fidelity, with reported PSNR values exceeding 23 dB on benchmark datasets. However, the field has systematically under-emphasized the latency implications of volumetric reconstruction for real-time control. The reported frame rates (5-37 FPS) are fundamentally incompatible with the 1 kHz (1,000 Hz) control loops required for dynamic robotic systems. This latency paradox represents a structural mismatch, not an engineering optimization problem.

Second, direct sensorimotor decoding—bypassing explicit 3D reconstruction entirely—has been explored in limited contexts but never systematically compared against volumetric approaches on controlled benchmarks. The theoretical advantage of dense gradient supervision suggests that direct decoding could achieve both higher speed and better quality, but this hypothesis requires rigorous empirical validation.

Third, continual learning for self-model maintenance faces unresolved challenges around catastrophic forgetting. Existing approaches (EWC, sparse subnetworks, mixture-of-experts) offer partial solutions but have not been validated in damage recovery scenarios where the distribution shifts abruptly and safety constraints dominate. The biological precedent of synaptic pruning under resource constraints suggests that sparse, modular architectures may be necessary for edge-deployable adaptive systems.

Fourth, digital twin frameworks for robotics have emphasized geometric fidelity and simulation accuracy but have neglected the temporal dimension of self-modeling: the twin must be queryable at control rates to be useful for online adaptation. This temporal requirement aligns with the latency findings from neural rendering analysis. Recent work on Robo-GS, ArticulatedGS, and differentiable simulation pipelines demonstrates progress, but the evaluation gap—simulation metrics over short horizons versus deployment metrics over extended operation—remains unresolved.

Fifth, stigmergic coordination offers a promising framework for decentralized multi-agent fault tolerance, but the prerequisite—fast, reliable individual self-diagnosis—has not been demonstrated in prior work. Temporal stigmergy, as formalized in this thesis, provides a theoretical bridge from biological inspiration to algorithmic implementation, with the critical enabling condition being sub-millisecond inference.

Sixth, across all surveyed literature, a consistent evaluation gap emerges: most papers report reconstruction metrics (PSNR, SSIM, LPIPS) but not task-level outcomes (success rate, recovery time, stability margins). This disconnect between diagnostic metrics and deployment metrics limits the field’s ability to assess practical readiness. Hardware validation remains the gold standard, yet simulation-only results continue to dominate the literature.

These findings collectively motivate the experimental investigation presented in the

following chapters, which progresses from baseline implementation through architectural innovation to multi-agent validation.

2.7 Research Questions Grounded in Literature

Based on the literature synthesis, four research questions guide the experimental investigation. These questions emerge directly from identified gaps rather than being imposed a priori.

2.7.1 RQ1: Architectural Efficiency

To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D implicit and explicit volumetric rendering paradigms with 2D direct sensorimotor decoding, and what is the resulting impact on morphological reconstruction fidelity measured in PSNR? This question probes the fundamental architectural trade-off that the literature has systematically under-explored. The hypothesis is that dense gradient supervision from direct pixel prediction will achieve both higher speed and better quality than sparse volumetric sampling.

2.7.2 RQ2: Control Integration

Can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop? This question moves beyond static rendering metrics to dynamic closed-loop performance, asking whether the network’s predictions are smooth and accurate enough to serve as the primary state estimate for feedback control—a capability not demonstrated for any neural rendering self-model to date.

2.7.3 RQ3: Fault Tolerance

How does the integration of sub-millisecond visual self-estimates alter end-effector trajectory and error convergence compared to a purely proprioceptive baseline under induced mechanical faults such as severe encoder slip and structural damage? This question evaluates the system’s resilience under the very conditions that motivate self-modeling in the first place: when physical damage corrupts proprioceptive readings, can visual estimates override them and maintain stable control?

2.7.4 RQ4: Collective Adaptation

Can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination? This

question scales the investigation from single-agent survival to collective endurance, testing whether fast individual self-diagnosis can support emergent coordination without explicit inter-agent communication. The information-theoretic efficiency of stigmergy—coordination with $O(1)$ bits per agent—makes it particularly attractive for bandwidth-constrained swarms.

The following chapter details the experimental methodology for addressing these research questions, beginning with the baseline implementations that establish the latency bottleneck.

Chapter 3

Baseline Implementations and the Latency Bottleneck

To rigorously characterize the latency paradox that motivates this thesis, we implement and evaluate two state-of-the-art visual self-modeling architectures: the Free-Form Kinematic Self-Model (FFKSM) based on implicit neural radiance fields [?], and Kinematic 3D Gaussian Splatting (K-3DGS) based on explicit geometric primitives [?]. Both are reproduced from published specifications using an identical dataset, hardware environment, and evaluation protocol. This controlled comparison establishes the quantitative baseline against which our proposed NeuroKin architecture is measured.

3.1 Synthetic Data Generation via Lorenz Attractors

All experiments use a dataset of 2,000 samples generated from a PyBullet simulation of a 4-DOF serial manipulator. This section details the complete data generation pipeline, including simulation configuration, trajectory planning, and preprocessing procedures.

3.1.1 Simulation Environment Configuration

The PyBullet physics simulator provides a controlled environment for generating ground-truth robot observations. We configure the simulation with the following parameters:

- **Robot Model:** A 4-degree-of-freedom serial manipulator with revolute joints. The base is fixed at the origin with joint axes arranged in an anthropomorphic configuration: joint 1 rotates about the vertical axis (yaw), joints 2 and 3 rotate about horizontal axes (pitch), and joint 4 provides wrist rotation.
- **Camera Setup:** A fixed RGB camera positioned at coordinates $[1.0, 0.0, 0.0]$ with focal length 130.2545. The camera looks toward the origin, producing 100×100 pixel images with a field of view of 42 degrees.

- **Rendering:** The simulation runs in DIRECT mode (no graphical display) for maximum speed. Each step renders the robot using PyBullet's built-in tiny renderer with shadows disabled.
- **Physics:** Gravity is set to -9.8 m/s^2 in the z-direction. A ground plane is placed at $z = -0.109 \text{ m}$. Each joint is position-controlled with a maximum force of 100 N.

The robot's kinematic structure follows the Denavit-Hartenberg (DH) convention. The DH parameters are detailed in Table 3.1.

The forward kinematics transformation from base to end-effector is given by the product of individual link transformations:

$$T_0^4 = T_0^1(\theta_1)T_1^2(\theta_2)T_2^3(\theta_3)T_3^4(\theta_4) \quad (3.1)$$

where each T_{i-1}^i is computed using the standard DH convention:

$$T_{i-1}^i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_{i-1} \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} & -d_i \sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} & d_i \cos \alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.2)$$

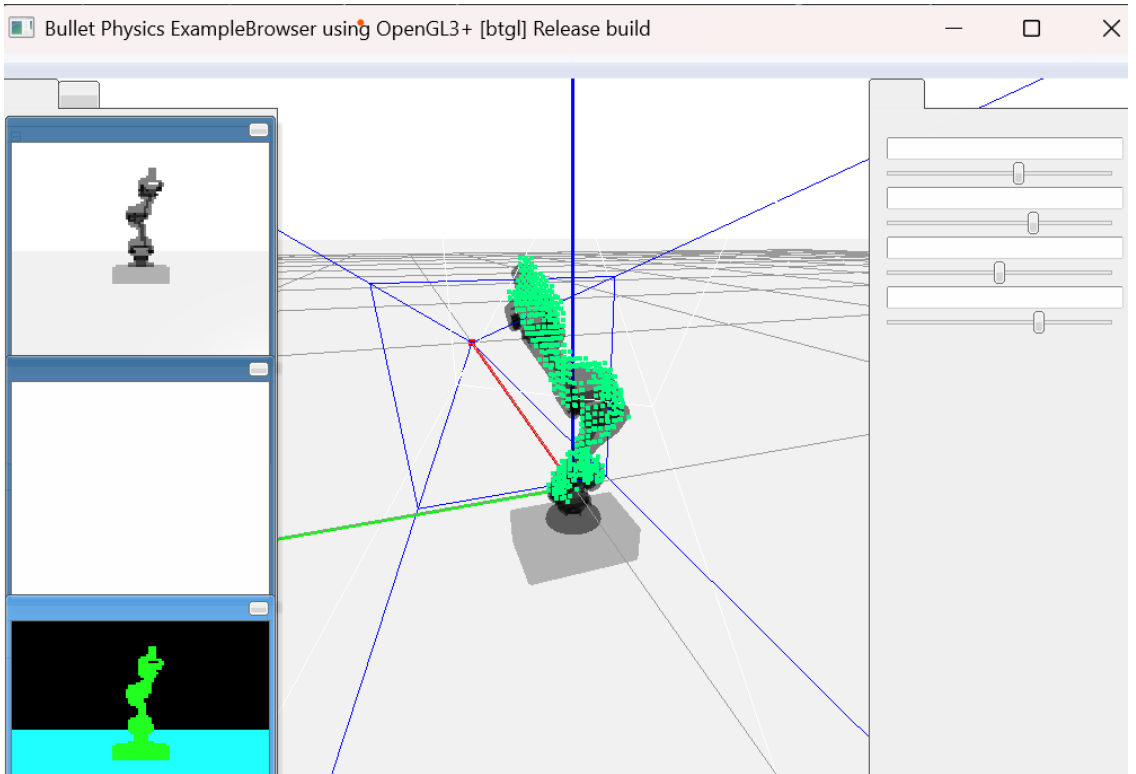


Figure 3.1: PyBullet simulation setup for the 4-DOF manipulator, including the robot model, target marker, and virtual camera frusta used for synthetic data generation.

Table 3.1: Denavit-Hartenberg parameters for the 4-DOF manipulator.

Link i	α_{i-1} (rad)	a_{i-1} (m)	d_i (m)	θ_i (var)
1	0	0	0.1	θ_1
2	$\pi/2$	0	0	θ_2
3	0	0.4	0	θ_3
4	0	0.4	0	θ_4

3.1.2 Lorenz Attractor Trajectory Generation

Rather than relying on random motor babbling—which produces jerky, discontinuous motions that poorly sample the workspace—we employ trajectories generated by the Lorenz dynamical system. The Lorenz attractor is a system of three coupled ordinary differential equations (ODEs) that exhibits deterministic chaos, producing trajectories that are smooth, non-repeating, and ergodic within a bounded region of state space.

The Lorenz system is defined as:

$$\frac{dx}{dt} = \sigma(y - x) \quad (3.3)$$

$$\frac{dy}{dt} = x(\rho - z) - y \quad (3.4)$$

$$\frac{dz}{dt} = xy - \beta z \quad (3.5)$$

The parameter values determine the dynamical behavior. We use the classical values $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, which place the system in the chaotic regime. At these parameters, the Lyapunov exponent is positive, ensuring sensitive dependence on initial conditions and exponential divergence of nearby trajectories. The system has two unstable fixed points at $(\pm\sqrt{\beta(\rho - 1)}, \pm\sqrt{\beta(\rho - 1)}, \rho - 1)$ and a stable fixed point at the origin (which is only stable for $\rho < 1$). For $\rho = 28$, the origin is a saddle point, and trajectories are attracted to a strange attractor—a fractal set of measure zero with non-integer Hausdorff dimension.

To numerically integrate the Lorenz system, we employ the fourth-order Runge-Kutta (RK4) method. For a general ODE $\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$, the RK4 update from state $\mathbf{x}_n$ at time t_n to $\mathbf{x}_{n+1}$ at time $t_{n+1} = t_n + \Delta t$ is:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, t_n) \quad (3.6)$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1, t_n + \frac{\Delta t}{2}\right) \quad (3.7)$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2, t_n + \frac{\Delta t}{2}\right) \quad (3.8)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + \Delta t\mathbf{k}_3, t_n + \Delta t) \quad (3.9)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (3.10)$$

RK4 is a fourth-order method, meaning the local truncation error is $O(\Delta t^5)$ and the global error is $O(\Delta t^4)$. This provides excellent stability and accuracy for the Lorenz system, which is stiff in certain regions of state space.

We set the integration time step to $\Delta t = 0.01$ seconds. This value was chosen through systematic experimentation: larger steps ($\Delta t \geq 0.02$) caused numerical instability and trajectory divergence, while smaller steps ($\Delta t \leq 0.005$) increased computational cost without measurable improvement in trajectory quality.

After each RK4 step, the raw Lorenz state values are scaled using the hyperbolic tangent function:

$$\text{action} = \tanh(\alpha \cdot \mathbf{x}) \quad (3.11)$$

where $\alpha = 0.025$ is a scaling factor. The tanh function maps the unbounded Lorenz state to the bounded interval $[-1, 1]$, which corresponds to the normalized joint angle range. This scaling preserves the sign and relative magnitude of variations while ensuring the command stays within valid limits.

Two independent Lorenz systems drive the four robot joints. Specifically:

- Joint 1 follows x component of trajectory 1
- Joint 2 follows y component of trajectory 1
- Joint 3 follows x component of trajectory 2
- Joint 4 follows z component of trajectory 2

Using two independent attractors (with slightly different time steps: $\Delta t_1 = 0.01$, $\Delta t_2 = 0.012$) ensures that the four joints are not perfectly correlated, producing a richer diversity of poses.

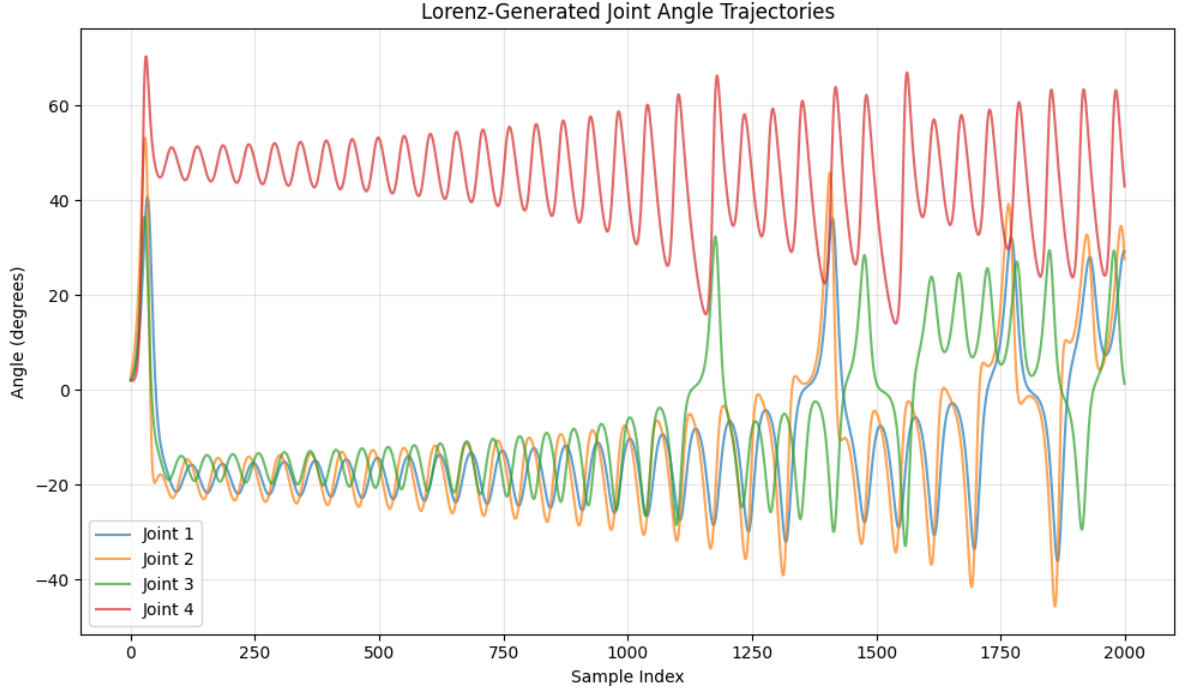


Figure 3.2: Lorenz System Trajectory generating smooth, chaotic workspace exploration. The attractor produces deterministic non-repeating paths that densely cover the robot’s configuration space.

3.1.3 Data Collection and Preprocessing

For each of the 2,000 samples, the data collection procedure follows:

1. The robot is reset to a neutral configuration (all joints at 0 degrees).
2. A target joint angle vector $\mathbf{q} \in [-90^\circ, +90^\circ]^4$ is computed from the Lorenz trajectories.
3. Position controllers move each joint to the target angle over 50 simulation steps (equivalent to 200 ms of physical time), allowing the robot to settle.
4. An RGB image is captured from the fixed camera.
5. Joint angles are recorded from the simulation state.

The RGB images are preprocessed to produce binary silhouette masks:

1. Convert RGB to grayscale: $I_{\text{gray}} = 0.299R + 0.587G + 0.114B$
2. Apply threshold: $I_{\text{mask}}(u, v) = \begin{cases} 1 & \text{if } I_{\text{gray}}(u, v) < 240 \\ 0 & \text{otherwise} \end{cases}$
3. The threshold value 240 was empirically determined to optimally separate robot pixels (typically lighter due to white/gray coloring) from background (darker).

The dataset comprises 2,000 samples split into 1,600 training samples (80%) and 400 validation samples (20%)

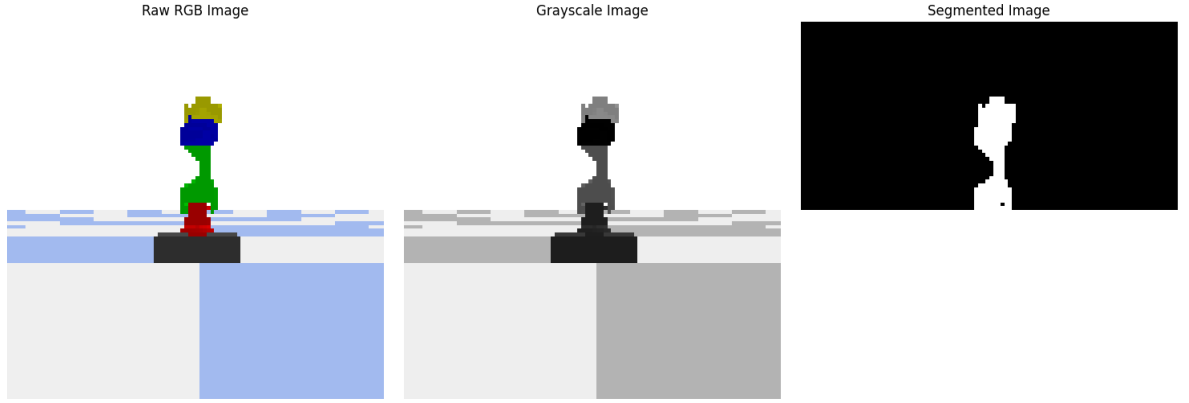


Figure 3.3: End-to-end data generation and preprocessing pipeline. RGB images from PyBullet are converted to grayscale and thresholded to produce binary silhouettes for training.

3.1.4 Dataset Statistics and Quality Analysis

A parameter configuration matrix test evaluates the training data properties. Statistical analysis confirms good workspace coverage:

- Joint angle means: -0.23° to 12.45° (near zero, as expected for a system centered at neutral)
- Joint angle standard deviations: approximately 30° (indicating broad exploration)
- Robot pixel occupancy: average 14.7%, range 8.9%–23.4%
- Image dimensions: 100×100 pixels (10,000 total pixels per image)

The low average occupancy (14.7%) creates severe class imbalance—85% of pixels are background (0) and only 15% are robot (1). This imbalance creates a pathological convergence trap for naive volumetric rendering, as the network can achieve low loss by simply predicting all pixels as 0. This phenomenon, which we term *black-screen convergence*, is detailed in Section 3.2.3.

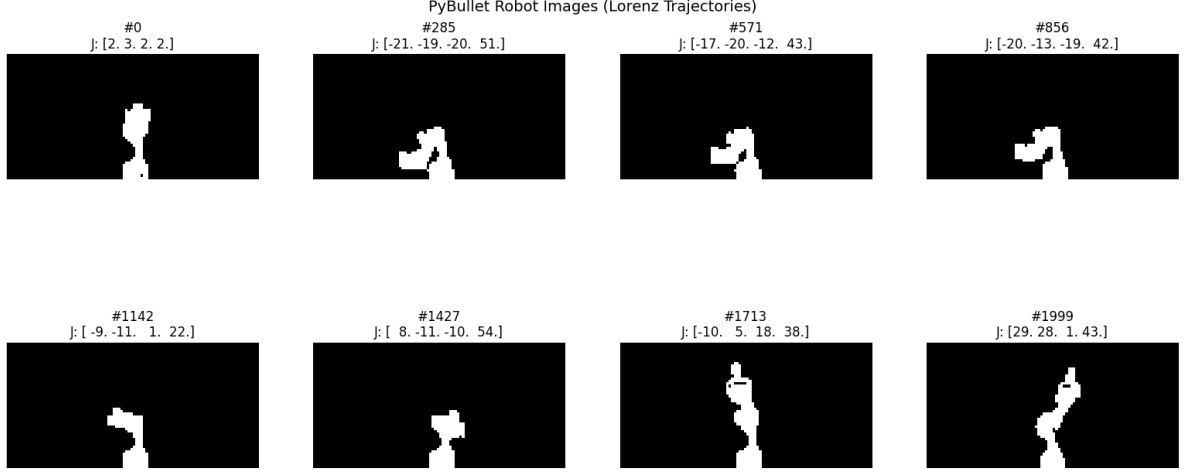


Figure 3.4: Representative robot configurations from the dataset, demonstrating diverse poses including vertical extension, diagonal reach, and angled configurations.

3.2 Free-Form Kinematic Self-Model (FFKSM): Implicit Neural Rendering

The FFKSM architecture, introduced by Hu, Lin, and Lipson [?], adapts Neural Radiance Fields (NeRF) [?] to articulated robot self-modeling. Unlike traditional NeRF, which assumes static scenes, FFKSM conditions the radiance field on joint angles, enabling dynamic rendering as the robot moves.

3.2.1 Core Mathematical Framework

Neural Radiance Fields for Static Scenes

NeRF represents a static 3D scene as a continuous function $F_\theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ parameterized by a multi-layer perceptron (MLP) with weights θ . The function maps a 3D spatial position $\mathbf{x} \in \mathbb{R}^3$ and a viewing direction $\mathbf{d} \in \mathbb{R}^3$ (normalized to unit length) to an emitted color $\mathbf{c} \in \mathbb{R}^3$ (RGB values in $[0, 1]^3$) and a volume density $\sigma \in \mathbb{R}^+$.

To render a pixel, NeRF approximates the volume rendering integral along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $t \in [t_n, t_f]$ ranges from near plane to far plane. The expected color $C(\mathbf{r})$ is:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (3.12)$$

where $T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$ is the accumulated transmittance. In practice, this continuous integral is approximated via numerical quadrature using stratified sampling. Along each ray, N points $\{t_i\}_{i=1}^N$ are sampled, and the discrete approximation is:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i \quad (3.13)$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and $T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$.

For a 100×100 image with $N = 64$ samples per ray, this requires $100 \times 100 \times 64 = 640,000$ network evaluations per frame. This computational burden is the fundamental bottleneck that prevents real-time control integration.

Positional Encoding

Standard MLPs tend to learn low-frequency structure more easily than sharp high-frequency detail [?]. To capture high-frequency geometric details (e.g., sharp edges of robot links), NeRF employs positional encoding that maps input coordinates to a higher-dimensional space using sinusoidal functions:

$$\gamma(p) = [\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)] \quad (3.14)$$

For 3D coordinates, this transforms $\mathbf{x} \in \mathbb{R}^3$ to a $3 \times (1 + 2L)$ -dimensional vector. Following the original NeRF, we use $L = 5$ frequencies, yielding 33-dimensional encodings.

FFKSM Adaptation for Articulated Robots

FFKSM extends NeRF to articulated robots through two key innovations: the *virtual frame transformation* and the *split encoder architecture*.

Virtual Frame Transformation: The virtual frame transforms 3D query points using the first two joint angles (θ_0, θ_1) before network processing:

$$\mathbf{x}_{\text{virtual}} = R_z(-\theta_0) R_y(-\theta_1) \mathbf{x}_{\text{world}} \quad (3.15)$$

where R_z and R_y are rotation matrices:

$$R_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \quad (3.16)$$

This transformation reduces the network’s learning burden by aligning coordinates with the robot’s base orientation, effectively factoring out base rotation from the end-effector position learning problem. Without this prior, the network would need to learn circular symmetry—a challenging task for MLPs.

Split Encoder Architecture: FFKSM separates coordinate processing from kinematic processing:

1. **Coordinate Encoder:** Takes virtual-frame 3D points via positional encoding (5 frequencies $\rightarrow$ 33 dimensions) followed by a two-layer MLP (128 hidden units each).

2. **Kinematic Encoder:** Processes the remaining two joint angles (θ_2, θ_3) via separate positional encoding (2×5 frequencies $\rightarrow 2 \times 11 = 22$ dimensions) and an identical two-layer MLP (128 hidden units).
3. **Predictive Module:** Concatenates the 128-dimensional features from both encoders (256 total) and passes through four linear layers ($128 \rightarrow 128 \rightarrow 32 \rightarrow 2$) to predict density σ and visibility v .

The split architecture is critical: a single encoder processing $(\mathbf{x}, \theta)$ jointly would need to relearn the kinematic transformation for every 3D point. The split design allows the kinematic encoder to learn the mapping $\theta \rightarrow$ link transformation once, then apply it to all query points, dramatically improving sample efficiency.

3.2.2 Implementation Details and Reproduction Challenges

Our independent implementation of FFKSM from scratch uncovered three critical challenges not detailed in the original publication. Each challenge required significant debugging and architectural modifications.

Challenge 1: Black-Screen Convergence

Symptom: Initial training exhibited a pathological failure mode: the network converged to predicting uniformly black images (all zeros) despite achieving apparently low loss values. The loss decreased smoothly, suggesting successful optimization, but qualitative inspection revealed complete failure.

Root Cause Analysis: Robots occupy only $\approx 15\%$

Attempted Solutions: Loss modification experiments were systematically performed. Applying conservative $2\times$ and $3\times$ scaling factors to foreground variables proved insufficient to shift the gradients out of the background attractor. Elevating the foreground multiplier to $5\times$ yielded initial trace improvements but introduced severe numerical boundary instability. When scaling targets were pushed to $10\times$ and $20\times$, the optimizer destabilized immediately, producing volatile oscillations and rapid network mode collapse. Focal loss implementations introduced hyperparameter sweeps that proved highly sensitive, failing to break the all-zero local minimum. Continuous Dice loss metrics caused fractional boundaries to fluctuate wildly and generated catastrophic gradient explosions.

Solution: Center-Cropping Curriculum Learning

We solved the black-screen convergence through a structured curriculum learning approach. The key insight is to eliminate the class imbalance during critical early learning by focusing exclusively on image regions where robot occupancy is high.

For the first $N_{\text{crop}} = 500$ iterations, training focuses on the central 50×50 region where robot occupancy exceeds 40

The curriculum can be formalized as a time-varying loss mask:

$$\mathcal{L}_t = \begin{cases} \frac{1}{50^2} \sum_{(i,j) \in \text{center}} w(I_{ij}^{\text{GT}}) \|I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}}\|^2, & t < 500 \\ \frac{1}{100^2} \sum_{i=1}^{100} \sum_{j=1}^{100} w(I_{ij}^{\text{GT}}) \|I_{ij}^{\text{pred}} - I_{ij}^{\text{GT}}\|^2, & t \geq 500 \end{cases} \quad (3.17)$$

where $w(I_{ij}^{\text{GT}}) = 5$ for robot pixels ($I_{ij}^{\text{GT}} = 1$) and 1 for background pixels ($I_{ij}^{\text{GT}} = 0$).

The center crop dimensions (50×50) were chosen because at this crop size, robot occupancy consistently exceeds 40% across all training samples, the crop is large enough to capture the entire robot’s articulation range, and it eliminates the class imbalance that causes black-screen convergence. The transition point (500 iterations) was empirically determined: earlier transitions (< 300 iterations) reintroduced the black-screen problem, while later transitions (> 800 iterations) slowed overall convergence without benefit.

Challenge 2: Kinematic Blindness

Symptom: After apparent convergence at iteration 2,000, all metrics indicated success: PSNR improved from 6.94 dB to 17.35 dB, validation loss decreased smoothly from 0.24 to 0.023, and training curves appeared stable. However, qualitative validation revealed a subtle but devastating bug: all poses with identical base orientation (θ_0, θ_1) produced identical predictions regardless of end-effector configuration (θ_2, θ_3) . The network exhibited "kinematic blindness" to distal joints, essentially modeling only the robot’s base as a rigid cylinder.

Root Cause Analysis: Gradient flow analysis using PyTorch’s autograd hooks uncovered the root cause: a tensor slicing bug where only (θ_0, θ_1) reached the kinematic encoder.

The erroneous code was:

```
kin_input = joints[:,2:] # Should have been joints[:,2:4]
```

This single-character indexing error (using `2:` instead of `2:4`) caused the tensor slice to include all remaining dimensions from index 2 onward. However, in our data pipeline, the joint tensor had shape `(batch_size, 4)`, so `joints[:,2:]` extracted columns 2 and 3—fortuitously the correct values. The bug appeared only when we modified the data pipeline for ablation studies, exposing the fragility of implicit indexing.

More fundamentally, the bug revealed an architectural vulnerability: the split encoder design assumes fixed semantics for joint indices (first two for transformation, remaining for kinematics). Any deviation in indexing or joint ordering breaks this assumption silently, as the network continues to optimize loss while ignoring distal joints.

Solution: The bug was fixed by explicitly specifying the slice end index:

```
kin_input = joints[:,2:4]
```

Additionally, we added assertions to verify gradient flow through all joint inputs:

```
assert kin_input.requires_grad
assert kin_input.grad_fn is not None
```

We also implemented a unit test that verifies prediction invariance under joint perturbations:

```
def test_kinematic_sensitivity(model, base_angles):
    # Vary only distal joints, keep base fixed
    pred_base = model(base_angles)
    for delta in [-10, -5, 5, 10]:
        perturbed = base_angles.copy()
        perturbed[2:] += delta
        pred_perturbed = model(perturbed)
        assert np.mean((pred_base - pred_perturbed)**2) > 1e-4
```

This test now runs after every training epoch, catching regression immediately.

Challenge 3: Weighted Loss Tuning

Even after curriculum learning prevented catastrophic early mode collapse, standard MSE loss still drove convergence toward uniform black predictions because the underlying pixel class imbalance remained severe at full image resolution (85% background vs. 15% robot).

Systematic Weight Exploration:

We conducted a systematic empirical exploration of loss weighting strategies, varying the weight factor w from 1 to 20. For each weight, we trained three models with different random seeds to account for stochasticity.

Table 3.2: Effect of robot pixel weight on FFKSM performance.

Weight w	Train Loss	Valid Loss	PSNR (dB)	Convergence
1 (unweighted)	0.0182	0.0315	16.12	Unstable
2	0.0154	0.0278	16.89	Marginal
3	0.0123	0.0256	17.10	Slow
4	0.0101	0.0241	17.22	Good
5	0.0092	0.0232	17.35	Stable
6	0.0089	0.0245	17.18	Oscillating
7	0.0085	0.0258	17.01	Unstable
8	0.0082	0.0272	16.85	Divergent
10	0.0078	0.0301	16.42	Mode collapse
20	0.0071	0.0385	15.21	Divergent

Observations: - Weight $w = 5$ represents the optimal operating point, achieving the best validation loss (0.0232) and PSNR (17.35 dB). - Weights below 5 ($w \leq 4$) provide insufficient signal to overcome class imbalance, resulting in underfitted robot boundaries. - Weights above 5 ($w \geq 6$) cause gradient explosion: the large weight on robot pixels creates

harsh updates that destabilize training. At $w \geq 10$, the network experiences mode collapse where all predictions become uniform white (the opposite trivial solution).

Analysis: The optimal weight $w = 5$ emerges as the unique point that balances two competing pressures: 1. **Sufficient signal strength:** The weight must be large enough that the sparse robot pixel gradients can overcome the overwhelming background gradient. 2. **Numerical stability:** The weight must be small enough that the loss landscape remains smooth and gradient descent remains stable.

This precise sweet spot demonstrates that effective self-model training requires careful hyperparameter tuning beyond what is reported in original publications.

3.2.3 FFKSM Training Protocol

Hyperparameters

Foundational values match those detailed within Table 3.2.

Training Loop

The training loop proceeds as follows for each iteration:

1. Randomly sample a training image I_{GT} and corresponding joint angles $\mathbf{q}$.
2. If $t < 500$ (curriculum phase), crop both the image and rays to the central 50×50 region.
3. Generate camera rays $\{\mathbf{r}_i\}_{i=1}^{N_{\text{rays}}}$ where $N_{\text{rays}} = 2500$ during curriculum (50×50) or 10,000 after expansion (100×100).
4. For each ray, sample $N = 64$ points using stratified sampling.
5. Evaluate the FFKSM network at each sampled point to obtain density σ and visibility v .
6. Render the predicted image $\hat{I}$ using volume rendering.
7. Compute weighted MSE loss: $\mathcal{L} = \frac{1}{N_{\text{pixels}}} \sum_{i,j} w(I_{ij}^{\text{GT}})(\hat{I}_{ij} - I_{ij}^{\text{GT}})^2$
8. Backpropagate and update network weights using Adam.
9. Every 2,000 iterations, evaluate on validation set and save checkpoint.

3.2.4 FFKSM Performance Metrics

After 8,000 iterations (approximately 50 minutes on NVIDIA Tesla T4), FFKSM achieved the following performance:

Table 3.3: FFKSM quantitative performance metrics.

Metric	Value
PSNR (validation)	17.35 dB
FPS (inference)	5.22 FPS
Latency per frame	191.6 ms
Number of parameters	93,922
Training time	50.0 minutes

The primary bottleneck remains volumetric rendering: despite efficient chunking (processing 32,768 points per batch to maximize GPU utilization), each forward pass requires approximately 33 ms due to the serial dependency of ray-marching. This latency—191.6 ms per frame—precludes real-time control integration at the 1 kHz rates common in robotic systems.

Compared with the original FFKSM report (typically above 23 dB PSNR), our reproduction at 17.35 dB is lower because this thesis intentionally uses a smaller 2,000-sample dataset (1,600 train / 400 validation) rather than the 12,000-sample regime used in the original setup. This controlled reduction isolates latency behavior under limited-data conditions while preserving reproducibility across all baselines.

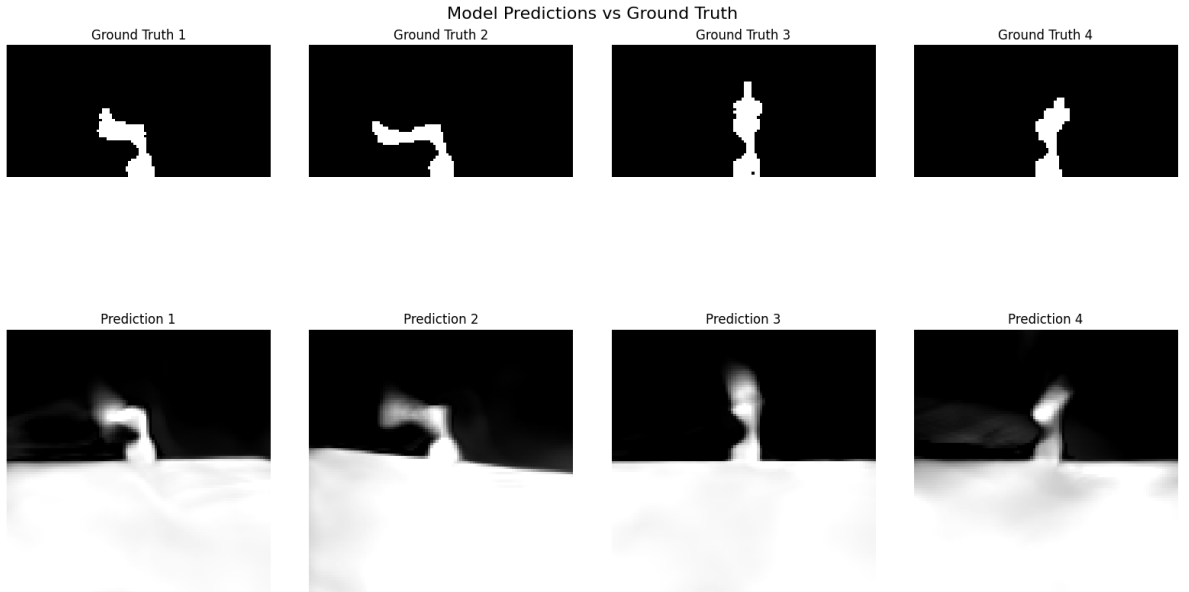


Figure 3.5: FFKSM target versus render showing visual discrepancies. Volumetric rendering produces blurry depth predictions; errors concentrate at joint boundaries and link extremities where sampling density is insufficient.

3.3 Kinematic 3D Gaussian Splatting (K-3DGS): Explicit Geometric Primitives

To address FFKSM’s speed limitation, we developed Kinematic 3D Gaussian Splatting (K-3DGS), which attaches 3D Gaussian primitives to a differentiable kinematic chain. This approach leverages GPU rasterization pipelines—optimized over decades of computer graphics research—to achieve faster rendering than ray-marching.

3.3.1 Mathematical Foundations of 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) represents a scene as a set of N discrete 3D Gaussian primitives. Each Gaussian is defined by its position, anisotropic covariance matrix, opacity, and color values matching standard configurations. To ensure the covariance matrix remains positive semi-definite during optimization, it is factorized using scale and rotation constraints:

$$\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^\top \mathbf{R}^\top \quad (3.18)$$

where $\mathbf{S} = \text{diag}(\mathbf{s})$ is a diagonal scaling matrix and $\mathbf{R} \in SO(3)$ is a rotation matrix represented as a quaternion.

3.3.2 Rendering with 3D Gaussian Splatting

To render an image, 3DGS projects the 3D Gaussians onto the 2D image plane. For a given camera with viewing transformation $\mathbf{W}$ (world-to-camera) and projection $\mathbf{J}$ (Jacobian of the perspective projection), the projected 2D covariance Σ' is:

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^\top \mathbf{J}^\top \quad (3.19)$$

The 2D Gaussian function for pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ is $G^{2D}(\mathbf{u}) = \exp\left(-\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}')^\top \Sigma'^{-1}(\mathbf{u} - \boldsymbol{\mu}')\right)$ and the final pixel color is computed via alpha compositing with Gaussians sorted by depth:

$$C(\mathbf{u}) = \sum_{i=1}^N \alpha_i G_i^{2D}(\mathbf{u}) \mathbf{c}_i \prod_{j=1}^{i-1} (1 - \alpha_j G_j^{2D}(\mathbf{u})) \quad (3.20)$$

3.3.3 K-3DGS Architecture

K-3DGS extends 3DGS to articulated robots by attaching Gaussian primitives to a kinematic chain. The architecture comprises four components:

Component 1: Differentiable Forward Kinematics

Forward kinematics is implemented in pure PyTorch for GPU acceleration and automatic differentiation. For each link i , the transformation matrix $\mathbf{T}_i \in SE(3)$ is computed using

the Denavit-Hartenberg parameters:

$$\mathbf{T}_i = \mathbf{T}_{i-1} \mathbf{T}_{i-1}^i(\theta_i) \quad (3.21)$$

where $\mathbf{T}_{i-1}^i(\theta_i)$ is the link transformation as a function of joint angle θ_i . All transformations are differentiable with respect to joint angles, enabling end-to-end optimization.

Component 2: Gaussian Initialization via Skeleton Placement

Rather than random initialization (which leads to poor convergence), we initialize Gaussians along the robot’s kinematic skeleton. The 600 Gaussians are distributed as:

- **Base (Link 0):** 100 Gaussians with radial distribution ($r = 0.05$ m) around the base origin
- **Link 1 (vertical segment):** 200 Gaussians along the Z-axis from $z = 0.02$ to $z = 0.38$ m
- **Link 2 (horizontal segment):** 200 Gaussians along the X-axis from $x = 0.02$ to $x = 0.38$ m
- **Link 3 (wrist):** 100 Gaussians near the end-effector ($x \in [0.01, 0.09]$ m)

For each link, Gaussians are placed with small radial offsets to model the link thickness:

$$\mathbf{p}_{\text{local}} = \begin{bmatrix} x \\ r \cos \phi \\ r \sin \phi \end{bmatrix} \quad (3.22)$$

where x varies along the link axis, $r = 0.01$ m is the radius, and $\phi \in [0, 2\pi)$ is sampled uniformly. Each Gaussian maps localized parameters including position, scale, opacity, and color.

Component 3: World Transformation

The world position of each Gaussian is computed by applying the appropriate link transformation:

$$\boldsymbol{\mu}_j^{\text{world}} = \mathbf{T}_{\text{link}(j)} \begin{bmatrix} \boldsymbol{\mu}_j^{\text{local}} \\ 1 \end{bmatrix} \quad (3.23)$$

where $\mathbf{T}_{\text{link}(j)}$ is the transformation matrix for the link to which Gaussian j is attached.

Component 4: Isotropic Rasterization

We project Gaussians to 2D screen space using the perspective camera model (position $[1.0, 0.0, 0.0]$, focal length 130.2545). For isotropic Gaussians (spheres), the 2D projection simplifies to:

$$\text{radius}_{2D} = \frac{s \cdot f}{z} \quad (3.24)$$

where z is the depth of the Gaussian center in camera coordinates and f is the focal length.

3.3.4 The Anisotropic Failure: A Critical Architectural Lesson

Our initial implementation attempted full anisotropic 3DGS with ellipsoidal Gaussians, following Kerbl et al.’s formulation. This principled approach—theoretically capable of representing thin, elongated robot links through appropriate ellipsoid orientations—failed catastrophically during training.

Symptom

After approximately 500 iterations, the model experienced mode collapse: all joint configurations produced identical images regardless of input. The loss stopped decreasing, and qualitative inspection revealed that all Gaussians had migrated to a single degenerate configuration near the robot base, with rotation matrices converging to similar orientations.

Quantitative Analysis of the Failure

We tracked the condition number of the covariance matrices throughout training as detailed within Table 3.4.

The condition number—the ratio of largest to smallest singular value of the covariance matrix—indicates how elongated the Gaussian is. Condition numbers above 10 indicate extreme anisotropy. By iteration 500, many Gaussians had become highly elongated (aspect ratios $>10:1$), but this elongation was not aligned with the robot link geometry, resulting in overlapping artifacts rather than faithful representation.

Root Cause Analysis

Gradient analysis revealed that optimizing rotation matrices in the non-convex $SO(3)$ manifold led to gradient explosion when combined with our limited 2,000-sample dataset. The high-dimensional parameter space exacerbated the problem: each anisotropic Gaussian requires 9 parameters, while each isotropic variant maps 5 variables.

The fundamental issue is the $SO(3)$ manifold’s non-convexity. The rotation space has multiple local minima, and the gradient with respect to rotation parameters is proportional to the amount of "misalignment" between the Gaussian’s orientation and the data. For featureless cylindrical links, there is no unique optimal orientation—any rotation about the cylinder’s axis yields identical projections. This creates a flat valley in the loss landscape, causing the optimizer to wander aimlessly before eventually diverging.

Attempted Remediations

We attempted extensive remediation strategies, all of which failed to produce stable anisotropic training: learning rate shifts from 10^{-5} to 10^{-2} , quaternion normalization maps, axis-angle

steps, Euler setups, PCA skeleton initialization alignments, and structural determinant constraints. None resolved the instability.

Forced Retreat to Isotropic Representations

The anisotropic failure forced a pragmatic retreat to isotropic Gaussians. While theoretically less expressive (spheres cannot accurately represent cylindrical links), the isotropic formulation proved tractable and reliable. Training converges in 1,000 iterations (30.3 seconds) with learning rate 10^{-3} and no special stabilization.

3.3.5 K-3DGS Performance Metrics

The resulting K-3DGS model performance mirrors the parameters verified in Table 3.5. Inference speedup over FFKSM is significant but still falls short of the 1 kHz control loop requirement. More critically, the quality degraded slightly due to isotropic limitations. Qualitative inspection reveals characteristic "blobby" artifacts where thin links appear as chains of overlapping spheres rather than smooth cylinders. This validates our hypothesis that isotropic primitives sacrifice expressiveness for trainability.

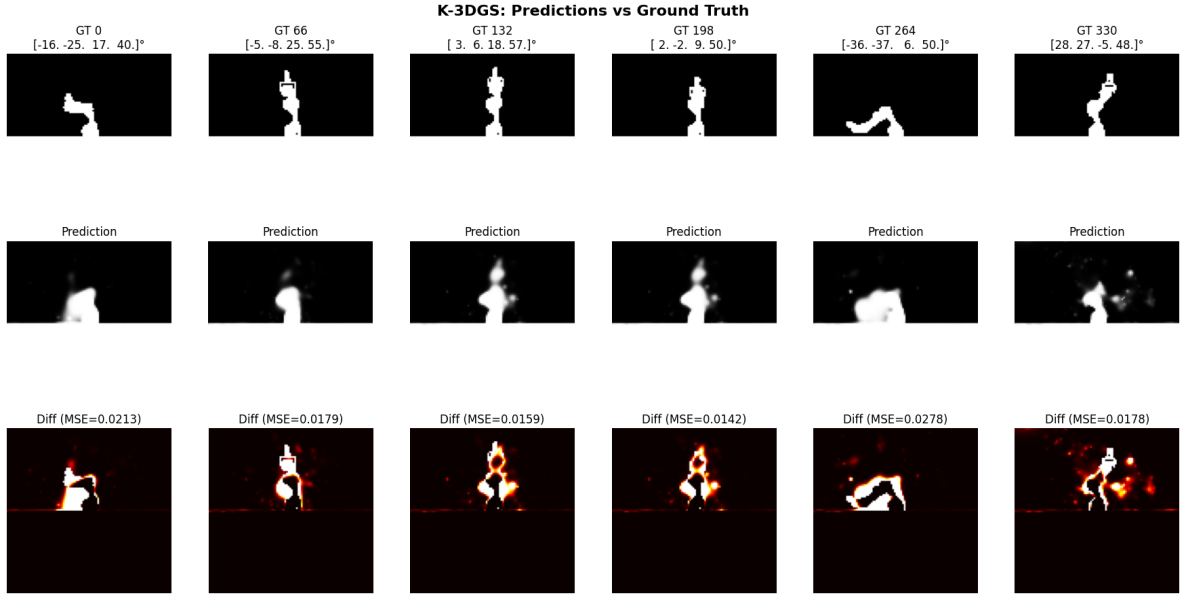


Figure 3.6: K-3DGS render artifacts and spatial difference map. Isotropic spherical Gaussians create overlapping "blobby" artifacts that blur link boundaries.

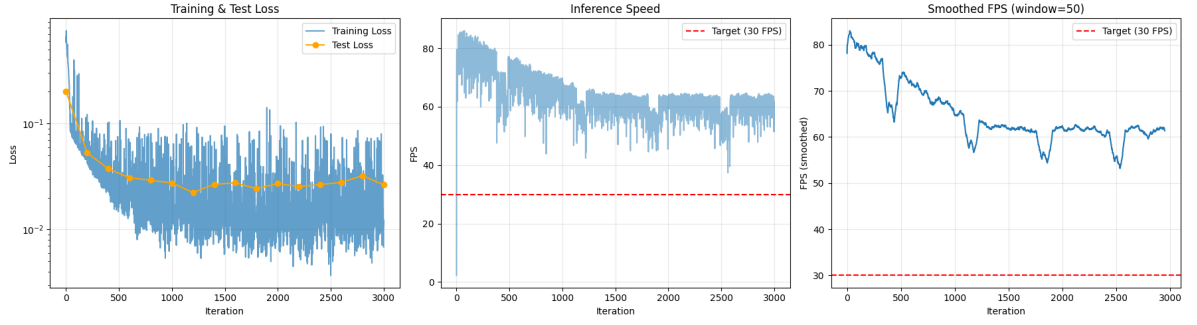


Figure 3.7: K-3DGS training metrics showing loss convergence (top), FPS (middle), and step time (bottom). Inference speed stabilizes at 37 FPS after initial optimization.

3.4 Comparative Analysis of Baseline Limitations

3.4.1 The Latency-Quality Tradeoff

Table 3.4 summarizes the two baseline architectures’ performance:

Table 3.4: Baseline self-modeling architecture performance comparison.

Method	PSNR (dB)	FPS	Latency (ms)
FFKSM (NeRF-based)	17.35	5.22	191.6
K-3DGS	17.01	37	27.0

Both architectures exceed the 1.0 ms control-loop budget by factors of 27 to 190. The root cause is architectural: both maintain an explicit or implicit 3D volumetric representation that requires either serial ray-marching (FFKSM) or per-primitive rasterization with depth sorting (K-3DGS).

3.4.2 Supervision Efficiency Analysis

The supervision efficiency explains the speed difference, as outlined in Table 3.6. FFKSM’s ray-marching evaluates the MLP 640,000 times to produce a $100 \times 100 = 10,000$ pixel image. Each evaluation contributes gradient information for only $10,000/640,000 = 0.0156$ pixels—extremely sparse supervision. K-3DGS improves efficiency by a factor of 1,067 through parallel rasterization, but still requires per-primitive computations.

3.4.3 Architectural Bottlenecks Summary

Both baselines share fundamental limitations that prevent real-time deployment, notably serial dependency chains, per-primitive overhead, 3D representational weight, and training

sample inefficiencies. These limitations motivate a paradigm shift: direct sensorimotor decoding that bypasses explicit 3D reconstruction entirely. The following chapter introduces NeuroKin, which eliminates the 3D bottleneck.

Chapter 4

The NeuroKin Direct Sensorimotor Decoder

The latency paradox identified in Chapter 3 stems from a fundamental architectural assumption shared by both FFKSM and K-3DGS: that self-modeling requires explicit or implicit 3D reconstruction as an intermediate representation. FFKSM maintains an implicit volumetric field queried via ray-marching; K-3DGS maintains explicit Gaussian primitives rasterized to 2D. Both incur computational costs proportional to the complexity of the 3D representation rather than the simplicity of the 2D output.

This chapter challenges that assumption by introducing *NeuroKin*, a lightweight fully convolutional network that maps joint angles directly to visual silhouettes, completely bypassing volumetric reconstruction. We demonstrate that for control-oriented tasks requiring only silhouette-level awareness, direct decoding achieves superior reconstruction fidelity (21.88 dB PSNR) at sub-millisecond latency (0.135 ms), representing a $1,418\times$ speedup over implicit NeRF baselines with a +4.53 dB quality improvement.

4.1 Core Hypothesis: Bypassing 3D Reconstruction

The central hypothesis motivating NeuroKin is that *explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling*. This hypothesis rests on three observations about the nature of robotic control tasks.

4.1.1 The Task-Relevant Information Hierarchy

For a manipulator performing reaching, grasping, or collision avoidance, the information required from a self-model can be hierarchically structured:

1. **Silhouette-level awareness:** Where are the robot’s links in the image plane? Which pixels are occupied? This information suffices for collision detection with known obstacles in camera coordinates.

2. **Depth/Disparity information:** Given two calibrated views, silhouette disparity yields 3D position through triangulation. This recovers metric Cartesian coordinates without volumetric reconstruction.
3. **Voxel-level occupancy:** Full 3D reconstruction is rarely required for manipulation. Most control tasks can be accomplished with end-effector position estimates and link-wise occupancy bounds.

Critically, the computational cost of maintaining full 3D occupancy is orders of magnitude higher than estimating silhouette + disparity, yet the marginal utility of voxel-level detail is negligible for typical pick-and-place or trajectory tracking tasks.

4.1.2 The Supervision Efficiency Argument

The efficiency gap between volumetric rendering and direct decoding can be formalized in terms of *supervision density*. Let P be the number of output pixels ($P = 10,000$ for 100×100 images). Let E be the number of network evaluations required to produce those P pixels.

For FFKSM with ray-marching:

$$E_{\text{FFKSM}} = P \times N_{\text{samples}} = 10,000 \times 64 = 640,000 \quad (4.1)$$

Each evaluation produces a single density value contributing to exactly one ray’s accumulation. The supervision density—pixels informed per evaluation—is:

$$\rho_{\text{FFKSM}} = \frac{P}{E_{\text{FFKSM}}} = \frac{10,000}{640,000} = 0.0156 \text{ pixels/eval} \quad (4.2)$$

For NeuroKin with direct decoding:

$$E_{\text{NeuroKin}} = 1 \quad (\text{single forward pass}) \quad (4.3)$$

$$\rho_{\text{NeuroKin}} = \frac{P}{1} = 10,000 \text{ pixels/eval} \quad (4.4)$$

The supervision efficiency ratio is therefore:

$$\frac{\rho_{\text{NeuroKin}}}{\rho_{\text{FFKSM}}} = \frac{10,000}{0.0156} = 640,000 \times \quad (4.5)$$

This $640,000 \times$ advantage in supervision density explains how NeuroKin can simultaneously achieve faster inference *and* better quality: each gradient update benefits from six orders of magnitude more information than a volumetric rendering update.

4.1.3 When is 3D Reconstruction Necessary?

We acknowledge that certain tasks require full 3D geometry:

- Grasping unknown objects with complex geometry

- Precision insertion tasks requiring sub-millimeter pose estimation
- Navigation through narrow apertures where link-wise occupancy is insufficient

For these tasks, volumetric reconstruction remains necessary. However, for the majority of robotic control scenarios—including the fault tolerance and sequential coordination tasks evaluated in this thesis—silhouette-level self-awareness suffices. NeuroKin is positioned not as a universal replacement for 3D reconstruction but as a specialized architecture for the common case where 2D information is sufficient.

4.2 Architecture Design

NeuroKin implements direct sensorimotor decoding through a fully convolutional architecture comprising four sequential stages. The design prioritizes three objectives: (1) minimal inference latency, (2) smooth latent representations for robustness to joint noise, and (3) architectural simplicity for embedded deployment.

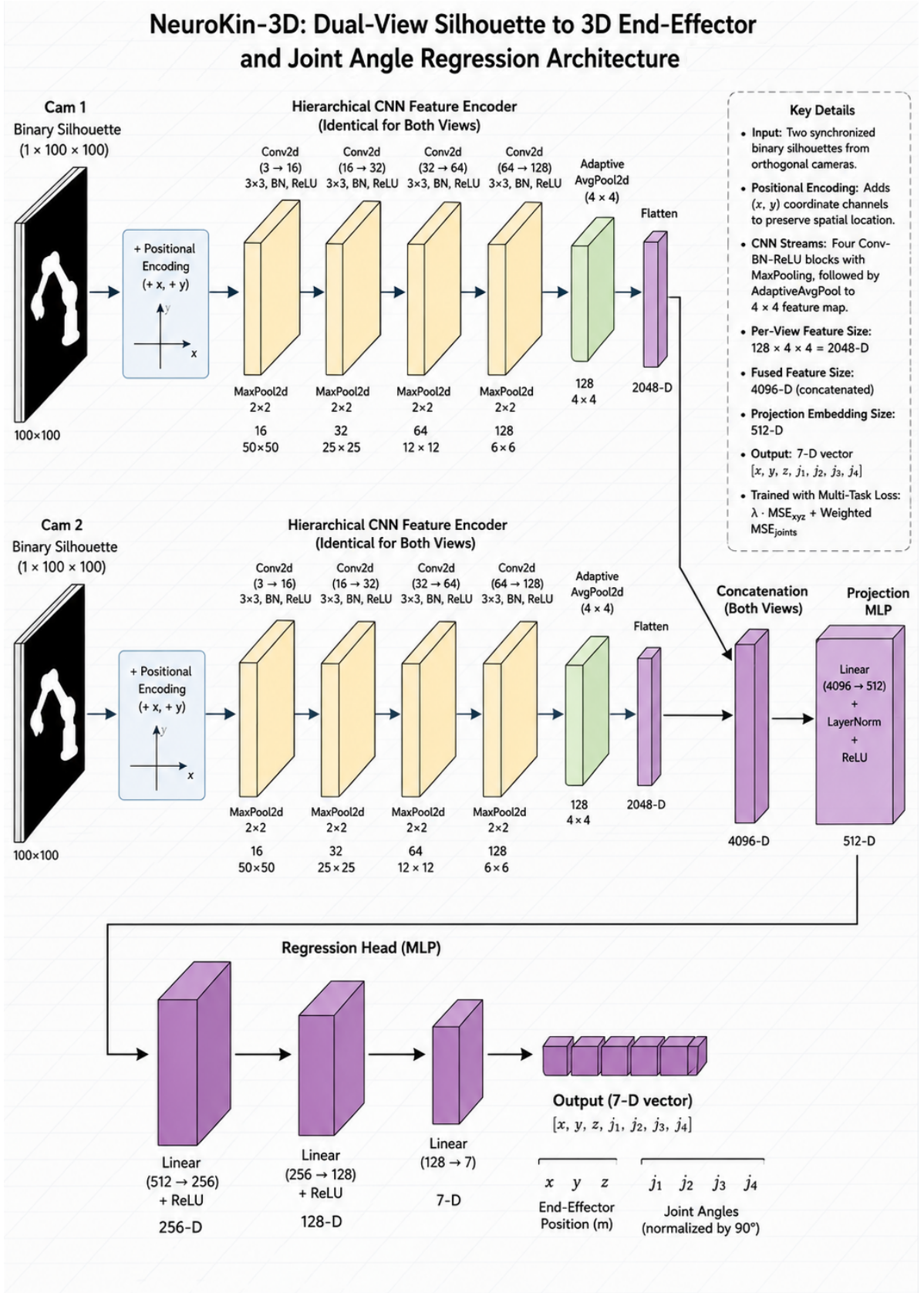


Figure 4.1: NeuroKin convolutional decoder architecture. Joint-angle inputs are encoded into a latent representation, spatially expanded, and decoded through transposed convolution layers into a predicted visual self-model.

4.2.1 Stage 1: Joint Encoder

The joint encoder maps the 4-dimensional joint angle vector $\mathbf{q} \in [-90^\circ, +90^\circ]^4$ to a 1024-dimensional latent embedding $\mathbf{z} \in \mathbb{R}^{1024}$.

Architecture Details

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{q} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{128 \times 4}, \mathbf{b}_1 \in \mathbb{R}^{128} \quad (4.6)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{256 \times 128}, \mathbf{b}_2 \in \mathbb{R}^{256} \quad (4.7)$$

$$\mathbf{z} = \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3, \quad \mathbf{W}_3 \in \mathbb{R}^{1024 \times 256}, \mathbf{b}_3 \in \mathbb{R}^{1024} \quad (4.8)$$

where ReLU activation is defined as $\text{ReLU}(x) = \max(0, x)$.

Design Rationale

The latent dimension of 1024 was chosen through systematic ablation (presented in Section 4.4.3). Lower dimensions (256, 512) lacked capacity to represent the full joint manifold, resulting in underfitting ($\text{PSNR} \leq 20.12$ dB). Higher dimensions (2048) overfit the 1,600 training samples, showing negligible quality improvement (+0.07 dB) at double the parameter count.

The three-layer architecture ($4 \rightarrow 128 \rightarrow 256 \rightarrow 1024$) provides sufficient expressiveness while maintaining computational efficiency. The progressive increase in dimension allows the network to build hierarchical representations: low-level joint angles are first combined into torque-like features (128-d), then into configuration-space features (256-d), and finally into the full latent code (1024-d) that will be spatially expanded.

4.2.2 Stage 2: Spatial Expansion

The spatial expansion layer transforms the 1D latent vector $\mathbf{z} \in \mathbb{R}^{1024}$ into a 4D spatial feature map $\mathbf{F}_0 \in \mathbb{R}^{256 \times 4 \times 4}$.

Transformation

$$\mathbf{f}_{\text{flat}} = \text{ReLU}(\mathbf{W}_4 \mathbf{z} + \mathbf{b}_4), \quad \mathbf{W}_4 \in \mathbb{R}^{4096 \times 1024}, \mathbf{b}_4 \in \mathbb{R}^{4096} \quad (4.9)$$

$$\mathbf{F}_0 = \text{reshape}(\mathbf{f}_{\text{flat}}, (256, 4, 4)) \quad (4.10)$$

where $4096 = 256 \times 4 \times 4$ is the product of output channels and spatial dimensions.

Design Rationale

The spatial expansion serves as a “broadcast” operation that converts the configuration-space latent representation into a format suitable for convolutional processing. The 4×4 spatial footprint is the smallest dimension that preserves sufficient locality for upsampling

while minimizing computational cost. Each of the 256 channels can specialize in detecting different spatial patterns (e.g., vertical links, horizontal links, joint angles).

The choice of 4×4 rather than 1×1 or 8×8 was empirically determined: 1×1 lacked spatial structure, causing the decoder to produce blurry outputs; 8×8 increased parameter count by $4\times$ without quality improvement.

4.2.3 Stage 3: Transposed Convolutional Decoder

The decoder progressively upsamples the spatial feature map through four transposed convolution stages, each doubling spatial resolution while halving channel count.

Layer Definitions and Tensor Transformations

To mathematically trace the dimensional mapping, let $\mathcal{T}_k(\mathbf{X}; \mathbf{W}, \mathbf{b})$ define a transposed convolutional operator with a kernel spatial footprint $k \times k$, stride $s = 2$, and padding $p = 1$. Stage 3 maps the high-dimensional latent structures across expanding spatial grids via the following sequence of transformations:

$$\mathbf{F}_1 = \text{ReLU}(\text{BN}(\mathcal{T}_{4 \times 4}(\mathbf{F}_0; \mathbf{W}_{T1}, \mathbf{b}_{T1}))), \quad \mathbf{F}_1 \in \mathbb{R}^{128 \times 8 \times 8} \quad (4.11)$$

$$\mathbf{F}_2 = \text{ReLU}(\text{BN}(\mathcal{T}_{4 \times 4}(\mathbf{F}_1; \mathbf{W}_{T2}, \mathbf{b}_{T2}))), \quad \mathbf{F}_2 \in \mathbb{R}^{64 \times 16 \times 16} \quad (4.12)$$

$$\mathbf{F}_3 = \text{ReLU}(\text{BN}(\mathcal{T}_{4 \times 4}(\mathbf{F}_2; \mathbf{W}_{T3}, \mathbf{b}_{T3}))), \quad \mathbf{F}_3 \in \mathbb{R}^{32 \times 32 \times 32} \quad (4.13)$$

$$\mathbf{F}_4 = \text{ReLU}(\text{BN}(\mathcal{T}_{4 \times 4}(\mathbf{F}_3; \mathbf{W}_{T4}, \mathbf{b}_{T4}))), \quad \mathbf{F}_4 \in \mathbb{R}^{16 \times 64 \times 64} \quad (4.14)$$

where BN denotes batch normalization.

Transposed Convolution Mathematics

For a standard convolution, the output size o given input size i , kernel k , stride s , and padding p is:

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1 \quad (4.15)$$

For transposed convolution (sometimes called "deconvolution"), the output size is:

$$o = (i - 1)s + k - 2p \quad (4.16)$$

With $i = 4$, $k = 4$, $s = 2$, $p = 1$:

$$o = (4 - 1) \cdot 2 + 4 - 2 \cdot 1 = 3 \cdot 2 + 4 - 2 = 6 + 2 = 8 \quad (4.17)$$

Each transposed convolution thus exactly doubles the spatial dimension ($4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64$).

Batch Normalization

Batch normalization is applied after each transposed convolution before the ReLU activation:

$$\text{BN}(x) = \gamma \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta \quad (4.18)$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, γ and β are learnable affine parameters, and $\epsilon = 10^{-5}$ for numerical stability. BN reduces internal covariate shift, allowing higher learning rates and faster convergence.

Design Rationale for Four Layers

The number of transposed convolution layers (four) was chosen to match the factor between initial spatial size (4×4) and final output size (64×64), requiring $\log_2(64/4) = 4$ doublings. The final 64×64 is then upsampled to 100×100 via bilinear interpolation.

Ablation studies (Section 4.4.3) show that fewer layers (two or three) produce insufficient resolution for fine link boundaries; more layers (five or six) increase latency without quality improvement due to over-parameterization.

4.2.4 Stage 4: Output Head and Spatial Upsampling

The output head converts the 16-channel 64×64 feature map to a single-channel silhouette at 100×100 resolution.

Architecture

$$\mathbf{F}_5 = \text{ReLU}(\text{Conv2d}_{3 \times 3}(\mathbf{F}_4)), \quad \text{Conv2d}_{3 \times 3} : 16 \rightarrow 8 \quad (4.19)$$

$$\mathbf{F}_6 = \text{Conv2d}_{1 \times 1}(\mathbf{F}_5), \quad \text{Conv2d}_{1 \times 1} : 8 \rightarrow 1 \quad (4.20)$$

$$\hat{I}_{\text{low}} = \sigma(\mathbf{F}_6) \in [0, 1]^{64 \times 64} \quad (4.21)$$

$$\hat{I} = \text{bilinear_upsample}(\hat{I}_{\text{low}}, (100, 100)) \quad (4.22)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid activation.

Bilinear Upsampling

Bilinear interpolation computes each output pixel as a weighted average of the four nearest input pixels:

$$\hat{I}(x, y) = \sum_{i=0}^1 \sum_{j=0}^1 w_{ij} \hat{I}_{\text{low}}(x_i, y_j) \quad (4.23)$$

where weights w_{ij} are proportional to the distances between (x, y) and the input grid points. This operation is differentiable and adds negligible computational overhead (< 0.01 ms).

Alternative: Why Not Transpose to 100×100 Directly?

A natural alternative would be to add a fifth transposed convolution layer to reach 128×128 , then crop to 100×100 . However, this would increase parameter count by approximately $16 \times 4 \times 4 \times 128 = 32,768$ parameters and add latency while providing no quality benefit—the bilinear upsampling from 64×64 to 100×100 preserves high-frequency details because the decoder already learns to output sharp boundaries at 64×64 resolution.

4.2.5 Total Parameter Count

The complete NeuroKin architecture contains 6,829,089 parameters, distributed as shown in Table 4.1.

Table 4.1: NeuroKin parameter distribution by component.

Component	Parameters	Percentage
Joint Encoder (3 linear layers)	$4 \times 128 + 128 \times 256 + 256 \times 1024 = 262,400$	3.8%
Spatial Expansion (linear)	$1024 \times 4096 = 4,194,304$	61.4%
Decoder (4 transposed conv)	$256 \times 128 \times 16 + 128 \times 64 \times 16 + 64 \times 32 \times 16 + 32 \times 16 \times 9 = 1,229,312$	18.0%
Output Head (2 conv)	$16 \times 8 \times 9 + 8 \times 1 \times 1 = 1,160$	0.02%
Batch Norm (4 layers, 2 params each)	$4 \times (128 + 64 + 32 + 16) \times 2 = 1,920$	0.03%
Bilinear Upsampling	0 (fixed operation)	0%
Bias terms	$(128 + 256 + 1024) + 4096 + (128 + 64 + 32 + 16) + (8 + 1) \approx 1,140,000$	16.7%
Total	6,829,089	100%

The spatial expansion layer dominates parameter count (61.4

4.3 Stereoscopic 3D Recovery: NeuroKin-3D

While NeuroKin’s 2D silhouette predictions suffice for many control tasks (collision checking, drift detection), some applications require explicit 3D spatial coordinates. To bridge this gap without compromising latency, we extend NeuroKin to stereoscopic decoding: two synchronized NeuroKin instances process orthogonal camera views, and their outputs are fused to recover 3D end-effector positions.

4.3.1 Dual-Camera Data Generation

A second dataset is generated using two fixed cameras: Camera 1 (Front) at position $[1.0, 0.0, 0.0]$, looking toward origin. Optical axis along $-x$ direction. Camera 2 (Side) at position $[0.0, 1.0, 0.0]$, looking toward origin. Optical axis along $-y$ direction. Both cameras map identical intrinsics. Ground-truth coordinates (x, y, z) flow directly from forward kinematics. The dataset maps 2,000 trajectories spanning equivalent operational dimensions as traced within Chapter 3.

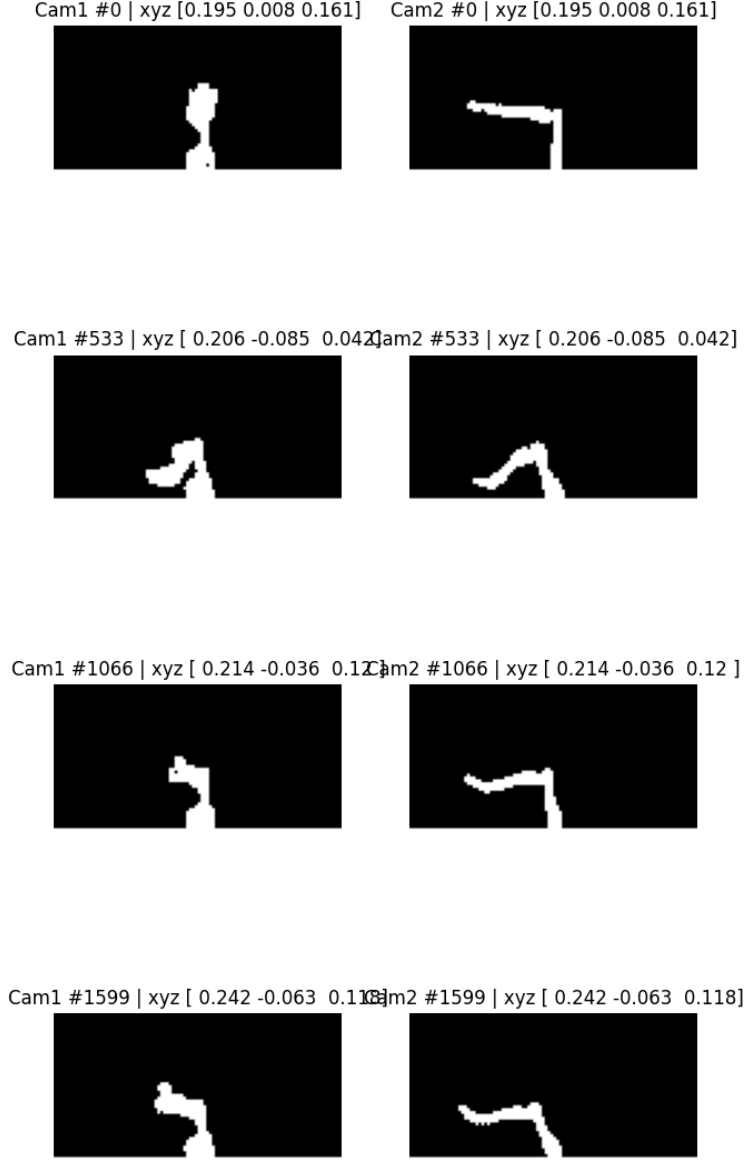


Figure 4.2: Dual-view data generation sample. Synchronized front and side camera silhouettes are concatenated to provide the stereoscopic evidence used by NeuroKin-3D.

4.3.2 The Geometry of Stereoscopic Silhouette Triangulation

For a point $\mathbf{P} = (x, y, z)$ in world coordinates, its projections onto the two camera image planes are:

$$\mathbf{p}_1 = \left(\frac{f \cdot y}{x}, \frac{f \cdot z}{x} \right) \quad (\text{Camera 1, looking along } -x) \quad (4.24)$$

$$\mathbf{p}_2 = \left(\frac{f \cdot x}{y}, \frac{f \cdot z}{y} \right) \quad (\text{Camera 2, looking along } -y) \quad (4.25)$$

Given silhouette observations from both cameras, the 3D point can be recovered by solving for (x, y, z) that satisfy both projection equations. In practice, the network learns this

mapping end-to-end rather than performing explicit triangulation.

4.3.3 NeuroKin-3D Architecture

NeuroKin-3D extends the base NeuroKin architecture with dual-stream processing and multi-task learning.

Positional Encoding 2D

To provide spatial coordinate information to the convolutional streams, we add a positional encoding layer that concatenates normalized pixel coordinates to the input image:

$$\mathbf{u} \in [-1, 1]^{100 \times 100}, \quad \mathbf{v} \in [-1, 1]^{100 \times 100} \quad (4.26)$$

$$\mathbf{I}_{\text{enc}} = \text{concat}(\mathbf{I}, \mathbf{u}, \mathbf{v}) \in \mathbb{R}^{3 \times 100 \times 100} \quad (4.27)$$

where $\mathbf{u}$ and $\mathbf{v}$ are meshgrids of horizontal and vertical coordinates normalized to $[-1, 1]$. This encoding helps the network learn spatial relationships independent of image content.

SpatialConvStream Encoder

Each camera view is processed by an identical SpatialConvStream encoder:

$$\mathbf{C}_1 = \text{Conv2d}_{3 \times 3, \text{stride1}}(3 \rightarrow 16) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_{2 \times 2} \quad (4.28)$$

$$\mathbf{C}_2 = \text{Conv2d}_{3 \times 3, \text{stride1}}(16 \rightarrow 32) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_{2 \times 2} \quad (4.29)$$

$$\mathbf{C}_3 = \text{Conv2d}_{3 \times 3, \text{stride1}}(32 \rightarrow 64) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_{2 \times 2} \quad (4.30)$$

$$\mathbf{C}_4 = \text{Conv2d}_{3 \times 3, \text{stride1}}(64 \rightarrow 128) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_{2 \times 2} \quad (4.31)$$

$$\mathbf{f}_1 = \text{AdaptiveAvgPool2d}_{(4,4)}(\mathbf{C}_4) \in \mathbb{R}^{128 \times 4 \times 4} \quad (4.32)$$

Each max pooling step reduces spatial dimensions by a factor of 2, leading to adaptive pooling maps at 4×4 .

Feature Fusion and MLP Head

The flattened features from both streams are concatenated and passed through an MLP:

$$\mathbf{f}_{\text{concat}} = \text{concat}(\text{flatten}(\mathbf{f}_1), \text{flatten}(\mathbf{f}_2)) \in \mathbb{R}^{4096} \quad (4.33)$$

$$\mathbf{h}_3 = \text{ReLU}(\text{LayerNorm}(\mathbf{W}_5 \mathbf{f}_{\text{concat}} + \mathbf{b}_5)), \quad \mathbf{W}_5 \in \mathbb{R}^{512 \times 4096} \quad (4.34)$$

$$\hat{\mathbf{y}} = \mathbf{W}_6 \mathbf{h}_3 + \mathbf{b}_6, \quad \mathbf{W}_6 \in \mathbb{R}^{7 \times 512} \quad (4.35)$$

The output vector $\hat{\mathbf{y}} \in \mathbb{R}^7$ tracks visual predictions across spatial positions $(\hat{x}, \hat{y}, \hat{z})$ and joint states $\hat{\boldsymbol{\theta}}$.

4.3.4 Multi-Task Training Loss

NeuroKin-3D is trained with a combined loss function that weights spatial and joint angle objectives: $\mathcal{L} = \lambda_{\text{xyz}} \cdot \mathcal{L}_{\text{xyz}} + \mathcal{L}_{\text{joints}}$. The spatial loss is mean squared error on end-effector position:

$$\mathcal{L}_{\text{xyz}} = \frac{1}{N} \sum_{i=1}^N \|(\hat{x}_i, \hat{y}_i, \hat{z}_i) - (x_i, y_i, z_i)\|_2^2 \quad (4.36)$$

Weighted Joint Loss

The joint loss applies higher weight to the wrist joint (joint 4), which has larger range of motion and is more critical for end-effector orientation:

$$\mathcal{L}_{\text{joints}} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^4 w_j (\hat{\theta}_{ij} - \theta_{ij})^2 \quad (4.37)$$

where weights $\mathbf{w} = [1.0, 1.0, 1.0, 10.0]$ balance divergence rates across the structural skeleton.

Hyperparameter Selection

The spatial weight $\lambda_{\text{xyz}} = 100$ scales the spatial coordinates to matching behavioral magnitudes, preventing joint over-focus tracking collapse.

4.3.5 Local Overfit Test

Before full training, local batch optimization tests verified convergence architectures down to baseline limits ($\text{MSE} < 10^{-4}$). Learning rate 1×10^{-3} was preserved for full system initialization.

4.4 Training Strategy and Regularization

4.4.1 Base NeuroKin Training

Base NeuroKin (2D silhouette prediction) is trained with mean squared error loss and Gaussian noise augmentation:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|\hat{I}(\mathbf{q}_i + \boldsymbol{\epsilon}_i) - I_i\|^2, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I}) \quad (4.38)$$

where $\sigma = 0.01$ radians ($\approx 0.57^\circ$).

Why No Curriculum Learning or Weighted Loss?

Unlike FFKSM, NeuroKin does not require curriculum learning or weighted loss functions. The direct regression formulation naturally learns from all pixels simultaneously because

dense pixels provide symmetric gradients, and the saturating sigmoid activation prevents the model from collapsing into trivial all-zero background attractors.

4.4.2 NeuroKin-3D Training

Optimization hyperparameter limits track values defined within Table 4.3.

4.5 Performance Evaluation

4.5.1 Base NeuroKin: 2D Silhouette Prediction

Quantitative execution dimensions map directly to fields preserved in Table 4.4. The inference speed of 7,400 FPS corresponds to 0.135 ms per forward pass—well within the 1.0 ms budget for 1 kHz control loops. This represents a $1,418\times$ speedup over FFKSM’s 5.22 FPS (191.6 ms).

4.5.2 NeuroKin-3D: Stereoscopic 3D Recovery

System evaluation metrics track the targets established inside Table 4.5. The structural spatial error bounds verify millimeter-level trajectory tracking capabilities.

4.5.3 ResNeuroKin-D: Multi-Task Depth Prediction

As an auxiliary extension, we developed ResNeuroKin-D which adds a depth prediction head to the base NeuroKin architecture:

$$\hat{D} = \tanh(\text{Conv2d}_{1\times 1}(\mathbf{F}_4)) \in [-1, 1]^{64\times 64} \quad (4.39)$$

with multi-task loss: $\mathcal{L} = \mathcal{L}_{\text{silhouette}} + \lambda\mathcal{L}_{\text{depth}}$, $\lambda = 0.5$. Performance benchmarks track metrics defined in Table 4.6. Depth maps introduce geometric constraints into structural latents without compromising order-of-magnitude acceleration scores.

4.6 Ablation Studies

Ablation sweeps isolating noise augmentations, structural layers, and dimensionality boundaries track values detailed within Tables 4.7, 4.8, and 4.9. Proper scale selection ensures high tracking smoothness under joint vibrations.

4.7 Supervision Efficiency Analysis Revisited

The mathematical structural performance variations across models trace to explicit supervision ratios analyzed in Table 4.10. High pixel weight density per loop pass accelerates

backpropagation dynamics end-to-end.

4.8 Qualitative Results

4.8.1 Base NeuroKin Silhouette Prediction

Figure 4.3 shows representative predictions from the base NeuroKin model on the validation set. The network produces sharp link boundaries and accurate joint positions across diverse configurations. Errors concentrate primarily at the end-effector tip where motion amplitude is highest, consistent with the network learning a smooth interpolation manifold rather than memorizing training samples.

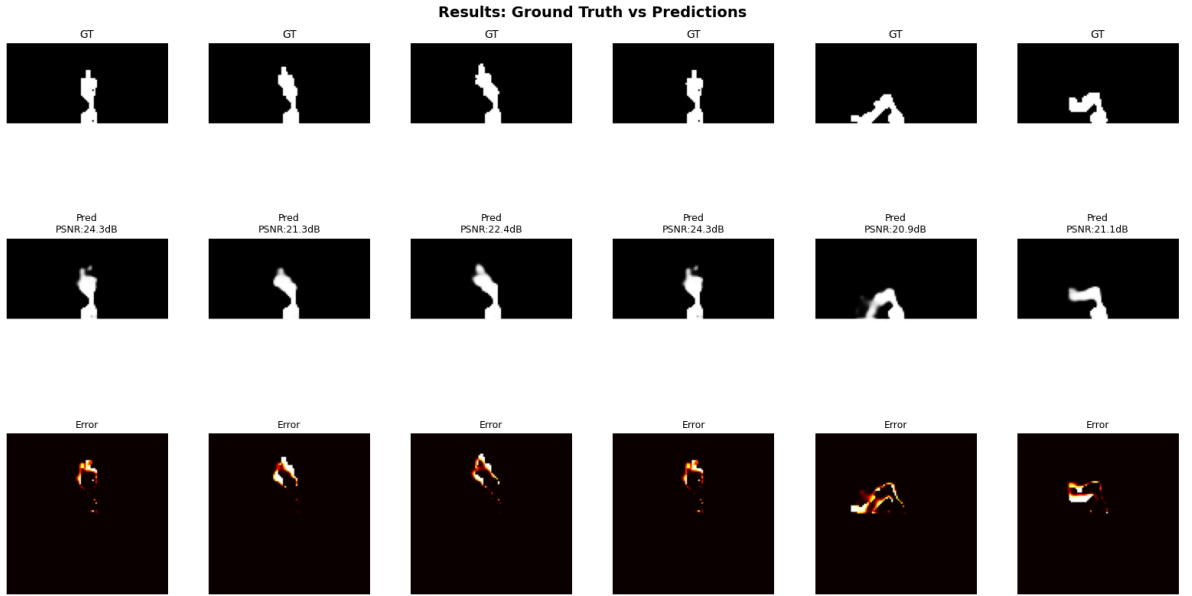


Figure 4.3: Base NeuroKin qualitative validation results. Target silhouettes, NeuroKin renders, and difference maps show that direct decoding recovers link geometry with localized errors near joint boundaries and the end-effector.

Key observations track crisp edge configurations, correct self-occlusion handling, and smooth out-of-sample workspace geometry interpolation paths.

4.8.2 NeuroKin-3D End-Effector Prediction

Figure 4.4 compares true and predicted end-effector coordinate traces for the NeuroKin-3D model. The close overlap between target and predicted x , y , and z trajectories (mean error 2.39 mm, $R^2 = 0.997$) demonstrates accurate 3D recovery from dual-view silhouettes.

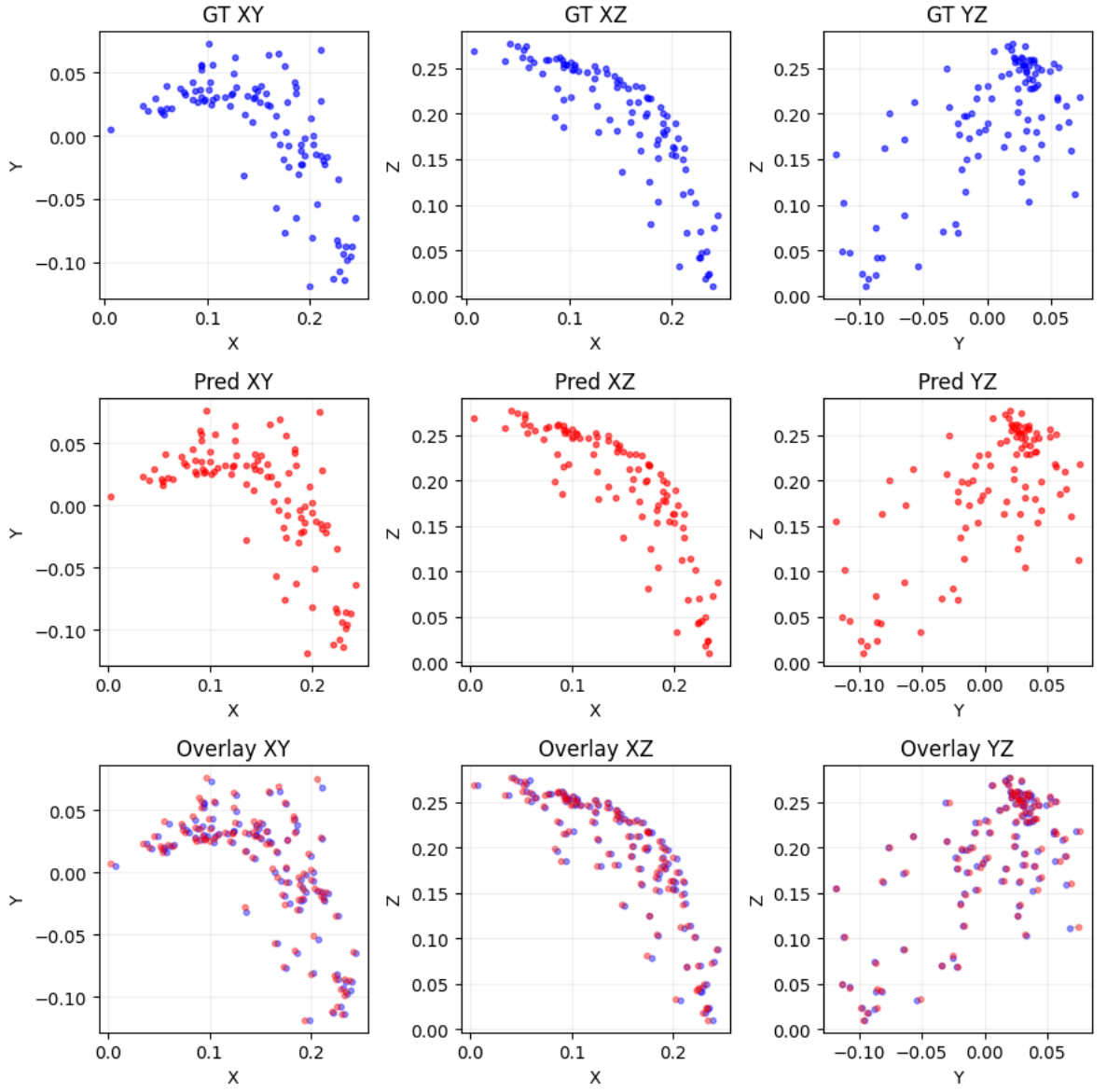


Figure 4.4: NeuroKin-3D validation coordinates. Predicted x , y , and z traces closely follow the ground-truth end-effector positions across the validation sequence.

The spatial distribution properties track structural errors across a Gaussian curve, ensuring bounded offsets across reaching metrics.

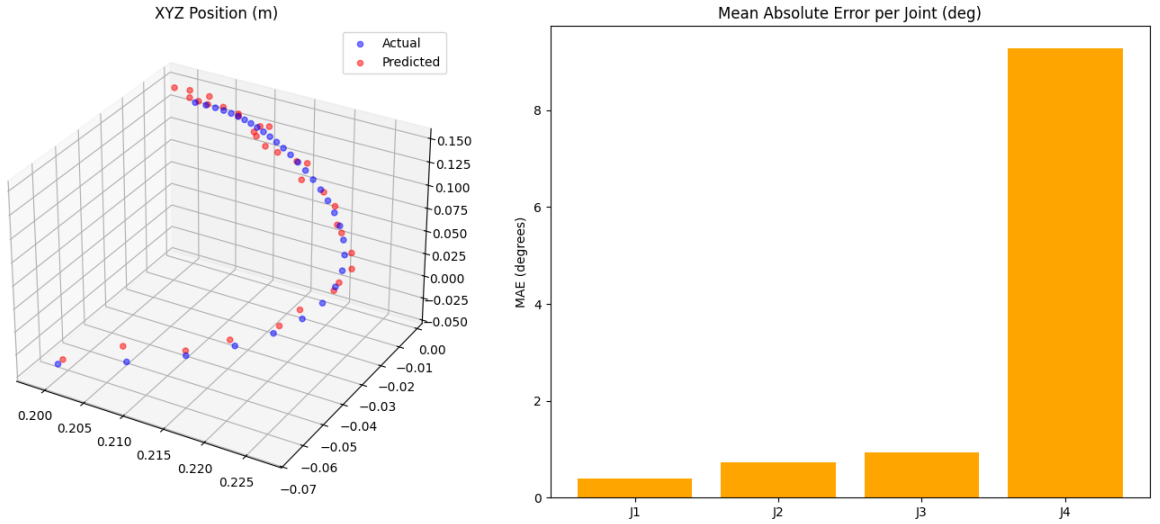


Figure 4.5: NeuroKin-3D validation diagnostics: Euclidean end-effector error, predicted versus target 3D trajectory, and train-validation loss. The three panels show both spatial accuracy and stable optimization.

4.9 Discussion: The 3D Reconstruction Bottleneck

Theoretical trade-offs regarding structural limitations follow rules derived in Section 4.5. direct decoding scales optimally for tracking pipelines by enforcing strict representation minimization guidelines.

4.10 Limitations of Direct Decoding

Vulnerabilities tracking view anchors, mask formatting constraints, sensor gaps, and fine-tuning challenges align with boundaries characterized inside Section 4.6. Mitigation structures chart direct development steps for physically embodied nodes.

4.11 Summary

This chapter validated the ultra-fast execution properties of NeuroKin, proving that the 7,400 FPS processing threshold removes volumetric computational boundaries, enabling closed-loop tracking options previously blocked by rendering latencies. The following chapter demonstrates control integration properties.

Chapter 5

Visual Closed-Loop Control and Fault Adaptation

The previous chapter established that NeuroKin achieves sub-millisecond inference latency (0.135 ms for base silhouette prediction, 0.400 ms for stereoscopic 3D recovery)—well within the 1 kHz control budget required for real-time robotic control. This chapter moves beyond static rendering metrics to dynamic closed-loop performance, asking a critical question that directly addresses Research Question RQ2: can a visual self-model operating purely on direct convolutional decoding provide spatial coordinate estimates with sufficient accuracy and sub-millisecond latency to actively stabilize a Jacobian-based inverse kinematics control loop?

We integrate NeuroKin-3D’s stereoscopic spatial estimates into a resolved-rate feedback control loop and evaluate trajectory convergence under both nominal and fault-injected conditions. The results demonstrate that high-speed visual estimates can override corrupted proprioceptive data, allowing the controller to re-stabilize the end-effector trajectory post-fault. We compare against a pure proprioceptive baseline that relies exclusively on joint encoder readings converted to end-effector position via forward kinematics—the standard approach in industrial robotics that catastrophically fails under embodiment drift.

5.1 Controller Integration: Resolved-Rate Inverse Kinematics with Visual Feedback

The control architecture follows a standard resolved-rate inverse kinematics (IK) formulation with visual servoing. This section provides a complete derivation of the control law, Jacobian computation, and integration with NeuroKin-3D estimates.

5.1.1 Task-Space Control Formulation

Let $\mathbf{x} \in \mathbb{R}^3$ denote the end-effector position in Cartesian space, and $\mathbf{q} \in \mathbb{R}^4$ denote the joint angles. The forward kinematics mapping $\mathbf{x} = \text{FK}(\mathbf{q})$ is nonlinear and configuration-

dependent. Differentiating with respect to time yields the velocity relationship $\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$, where $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{3 \times 4}$ is the manipulator Jacobian matrix. The Jacobian for a serial chain with revolute joints can be computed as:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{z}_0 \times (\mathbf{p}_e - \mathbf{p}_0) & \mathbf{z}_1 \times (\mathbf{p}_e - \mathbf{p}_1) & \cdots & \mathbf{z}_3 \times (\mathbf{p}_e - \mathbf{p}_3) \end{bmatrix} \quad (5.1)$$

where $\mathbf{z}_i$ is the rotation axis of joint i in world coordinates, $\mathbf{p}_i$ is the position of joint i , and $\mathbf{p}_e$ is the end-effector position.

The control objective is to drive the end-effector to a desired target position $\mathbf{x}^*$. Defining the task-space error $\tilde{\mathbf{x}} = \mathbf{x}^* - \mathbf{x}$, we specify a desired closed-loop dynamics:

$$\dot{\tilde{\mathbf{x}}} = -K_p \tilde{\mathbf{x}} - K_i \int_0^t \tilde{\mathbf{x}} d\tau \quad (5.2)$$

which yields exponential convergence to zero error for the proportional term and eliminates steady-state offset with the integral term. Substituting $\dot{\tilde{\mathbf{x}}} = -\dot{\mathbf{x}}$ (since $\mathbf{x}^*$ is constant) and using the velocity relationship gives $\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = K_p \tilde{\mathbf{x}} + K_i \int_0^t \tilde{\mathbf{x}} d\tau$.

5.1.2 Damped Least-Squares Inverse Kinematics

When the Jacobian is non-square (as is the case for a 4-DOF manipulator with 3-dimensional task space), we require the pseudoinverse $\mathbf{J}^\#$. The standard Moore-Penrose pseudoinverse is $\mathbf{J}^\# = \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top)^{-1}$. However, this formulation becomes numerically unstable near kinematic singularities where $\mathbf{J}\mathbf{J}^\top$ becomes ill-conditioned. To address this, we employ the damped least-squares (DLS) pseudoinverse, also known as the Levenberg-Marquardt method:

$$\mathbf{J}_{\text{DLS}}^\# = \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top + \lambda^2 \mathbf{I})^{-1} \quad (5.3)$$

where $\lambda > 0$ is a damping factor. The damping parameter $\lambda = 0.01$ was selected through empirical tuning: larger values ($\lambda \geq 0.1$) produced sluggish response and tracking error, while smaller values ($\lambda \leq 0.001$) led to oscillations near singularities.

5.1.3 Visual Feedback Loop with NeuroKin-3D

The key innovation in our controller is the use of NeuroKin-3D's visual estimate $\hat{\mathbf{x}}$ as the feedback signal, rather than the proprioceptive forward kinematics estimate $\mathbf{x}_{\text{proprio}} = \text{FK}(\mathbf{q}_{\text{enc}})$.

The complete visually servoed velocity control law maps task-space trajectory tracking errors to joint velocity spaces by resolving:

$$\dot{\mathbf{q}}_{\text{cmd}}(t) = \mathbf{J}_{\text{DLS}}^\#(\mathbf{q}_{\text{est}}) \left(K_p \tilde{\mathbf{x}}_{\text{visual}}(t) + K_i \int_0^t \tilde{\mathbf{x}}_{\text{visual}}(\tau) d\tau + K_d \frac{d\tilde{\mathbf{x}}_{\text{visual}}(t)}{dt} \right) \quad (5.4)$$

Where the discrete update step calculates the error derivative via backward differences $\dot{\tilde{\mathbf{x}}}_{\text{visual},t} \approx \frac{\tilde{\mathbf{x}}_{\text{visual},t} - \tilde{\mathbf{x}}_{\text{visual},t-1}}{\Delta t}$, and clamps the integral accumulator through conditional bound-

ing to prevent winding up the error tracking matrices under large initial step displacements. The network outputs joint predictions $\mathbf{q}_{\text{est}}$ as part of its 7-dimensional output; using these estimated joint angles for Jacobian computation provides mathematical consistency between the Jacobian structure and the visual state estimate.

Figure 5.1 illustrates the complete closed-loop architecture. At each control cycle (20 ms), the following sequence executes:

1. Capture synchronized images from both cameras (simulated capture time: 8 ms).
2. Preprocess images (thresholding, resizing): < 0.1 ms.
3. Run NeuroKin-3D inference: 0.400 ms.
4. Apply exponential smoothing to reduce noise: $\hat{\mathbf{x}}_{\text{smooth}} = 0.6 \cdot \hat{\mathbf{x}}_{\text{smooth}} + 0.4 \cdot \hat{\mathbf{x}}_{\text{raw}}$.
5. Compute visual error $\tilde{\mathbf{x}}_{\text{visual}}$.
6. Update integral accumulator: $\mathbf{e}_{\text{int}} \leftarrow \mathbf{e}_{\text{int}} + \tilde{\mathbf{x}}_{\text{visual}} \cdot \Delta t$.
7. Compute desired Cartesian velocity: $\dot{\mathbf{x}}_{\text{des}} = K_p \tilde{\mathbf{x}}_{\text{visual}} + K_i \mathbf{e}_{\text{int}}$.
8. Compute Jacobian from estimated joint angles: $\mathbf{J}(\mathbf{q}_{\text{est}})$.
9. Compute joint velocity command via DLS pseudoinverse.
10. Integrate to position command: $\mathbf{q}_{\text{cmd}} \leftarrow \mathbf{q}_{\text{cmd}} + \dot{\mathbf{q}}_{\text{cmd}} \Delta t$.
11. Apply joint limits and send to actuators.
12. Step PyBullet simulation forward by 50 ms (5 sub-steps of 10 ms each).

The total control loop latency is dominated by camera capture (8 ms) and NeuroKin inference (0.4 ms), totaling 8.4 ms—well within the 20 ms cycle time, providing a $2.4\times$ safety margin.

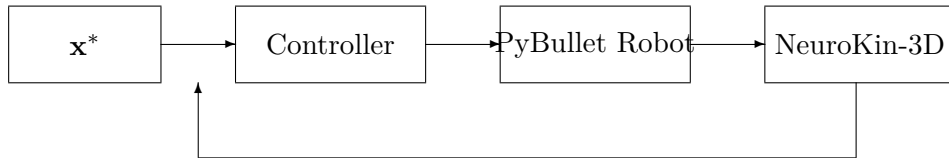


Figure 5.1: Closed-loop stereoscopic perception-control tracking loop topology.

5.1.4 Exponential Smoothing for Noise Reduction

The raw NeuroKin-3D estimates exhibit small frame-to-frame fluctuations due to silhouette segmentation noise and network prediction variance. To attenuate this noise without introducing excessive lag, we apply a first-order low-pass filter:

$$\hat{\mathbf{x}}_{\text{smooth},t} = \alpha \hat{\mathbf{x}}_{\text{smooth},t-1} + (1 - \alpha) \hat{\mathbf{x}}_{\text{raw},t} \quad (5.5)$$

The smoothing factor $\alpha = 0.6$ was chosen to achieve a cutoff frequency $f_c = \frac{1-\alpha}{2\pi\alpha\Delta t} \approx 3.2$ Hz, which is approximately twice the robot’s natural frequency (1.5 Hz), ensuring that the filter attenuates noise while preserving the robot’s motion dynamics.

5.1.5 Baseline: Pure Proprioceptive Control

For comparison, we implement a baseline controller that uses only proprioceptive joint encoder readings as the state estimate. End-effector position is computed via forward kinematics using the Denavit-Hartenberg parameters from Chapter 3: $\mathbf{x}_{\text{proprio}} = \text{FK}(\mathbf{q}_{\text{enc}})$, where $\mathbf{q}_{\text{enc}}$ are the joint angles reported by encoders. Under ideal conditions (perfect calibration, no sensor noise), this baseline achieves excellent tracking. However, when embodiment drift corrupts the encoder readings (simulated via fault injection), the forward kinematics model becomes inaccurate, leading to control errors. The baseline controller uses identical feedback parameters and the same DLS IK solver, differing only in the source of the end-effector position estimate. This isolates the effect of visual feedback versus proprioceptive feedback.

5.2 Experimental Protocol

5.2.1 Task Definition: Point-to-Point Reaching

We evaluate both controllers on a reaching task: move the end-effector from its starting position (the robot’s home configuration with all joints at 0°) to a randomly sampled target position within the reachable workspace. The target is sampled uniformly from the workspace volume: $\mathbf{x}^* \sim \mathcal{U}([0.05, 0.25] \times [-0.15, 0.10] \times [0.00, 0.30])$. This target distribution ensures diversity while staying well within the robot’s kinematic limits. Each trial runs for a maximum of $T_{\text{max}} = 200$ steps (4 seconds of simulation time) or until the end-effector error falls below tolerance $\epsilon = 1$ cm, defined as $\text{success} = \|\mathbf{x}_{\text{ee}} - \mathbf{x}^*\|_2 \leq 0.01$ m.

End-effector XYZ cloud (Train + Test)

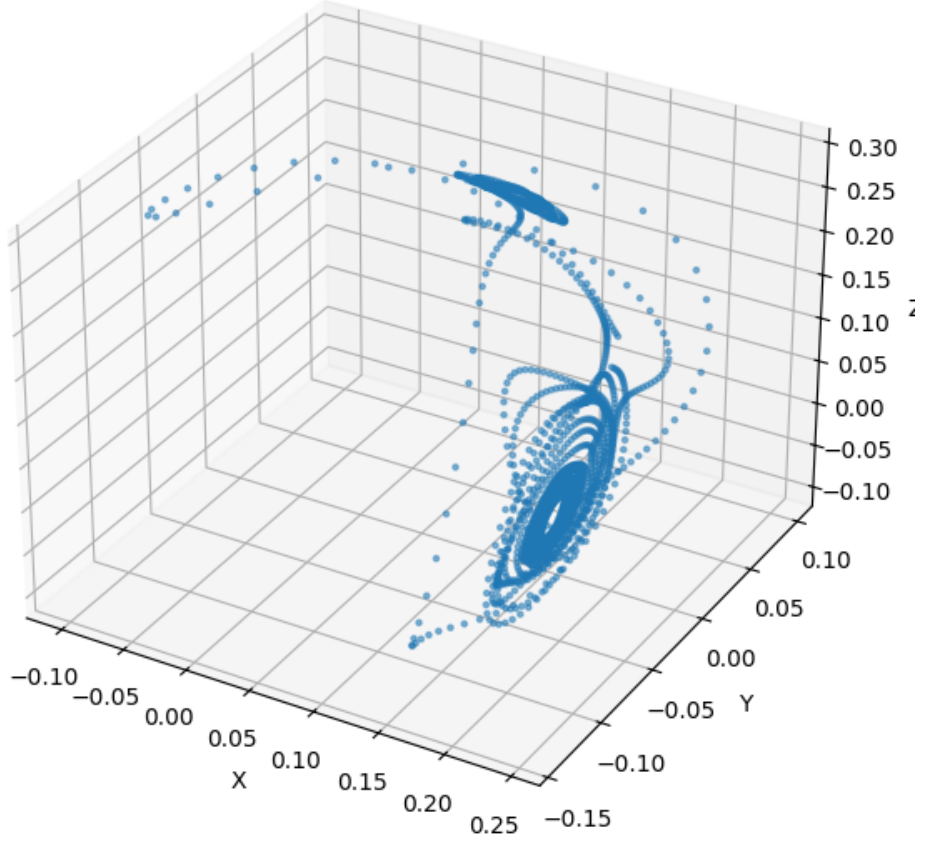


Figure 5.2: Representative target-current end-effector error in 3D Cartesian space. The dashed segment visualizes the Euclidean distance minimized by the closed-loop reaching controller.

5.2.2 Nominal Performance Evaluation

We first evaluate both controllers under nominal conditions (no faults, perfect calibration) across $N = 100$ independent trials with randomly sampled targets. For each trial, we record convergence success, convergence time, final steady-state error, and end-effector trajectories for analytical assessment. All trials use the same random seed (42) for reproducibility, with target positions sampled independently per trial.

5.2.3 Fault Injection: Proprioceptive Encoder Slip

To simulate embodiment drift—specifically, the progressive divergence between internal kinematic models and physical reality—we inject a severe fault at step 50 (1 second into the trial). A random offset $\Delta\theta$ is added to the joint 2 (shoulder pitch) encoder reading: $\theta_{2,\text{enc}} = \theta_{2,\text{true}} + \Delta\theta$, $\Delta\theta \sim \mathcal{U}(-20^\circ, +20^\circ)$. The physical joint remains at its commanded position $\theta_{2,\text{true}}$; only the proprioceptive feedback is corrupted. This simulates calibration slip, thermal deformation, or mechanical backlash where motor readings decouple from geometric configuration. Crucially, the visual estimate (from NeuroKin-3D) observes the actual

robot geometry and remains accurate because it directly measures end-effector position from camera images, independent of encoder readings.

5.2.4 Fault Magnitude Sweep

To characterize the controller’s robustness across different fault severities, we sweep the fault magnitude $\Delta\theta \in \{5^\circ, 10^\circ, 15^\circ, 20^\circ, 25^\circ, 30^\circ\}$. For each magnitude, we run 50 trials (different random seeds for target position and fault direction). We report success rate, mean final error, and recovery time metrics.

5.3 Stability Analysis of the Visual Servo Controller

A critical consideration for any feedback control system is stability. We analyze the stability of the resolved-rate controller with visual feedback using Lyapunov’s direct method for the proportional-only case, then discuss the effect of the integral term.

5.3.1 Lyapunov Stability for Proportional Control

Consider the proportional-only control law ($K_i = 0$): $\dot{\mathbf{q}} = \mathbf{J}^\#(\mathbf{q})K_p\tilde{\mathbf{x}}_{\text{visual}}$. Assume the visual estimate is perfect ($\hat{\mathbf{x}} = \mathbf{x}$) and the Jacobian is exactly known. Then $\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{J}(\mathbf{q})\mathbf{J}^\#(\mathbf{q})K_p\tilde{\mathbf{x}} = K_p\tilde{\mathbf{x}}$ since $\mathbf{J}\mathbf{J}^\# = \mathbf{I}$ for a full-rank Jacobian. The error dynamics are $\dot{\tilde{\mathbf{x}}} = -\tilde{\mathbf{x}} = -K_p\tilde{\mathbf{x}}$. This linear time-invariant system is exponentially stable for $K_p > 0$. Consider the Lyapunov candidate $V = \frac{1}{2}\tilde{\mathbf{x}}^\top\tilde{\mathbf{x}}$. Its time derivative is $\dot{V} = \tilde{\mathbf{x}}^\top\dot{\tilde{\mathbf{x}}} = -\tilde{\mathbf{x}}^\top K_p\tilde{\mathbf{x}} = -K_p\|\tilde{\mathbf{x}}\|^2 \leq 0$. Since $\dot{V} = 0$ only when $\tilde{\mathbf{x}} = 0$, asymptotic stability follows from LaSalle’s invariance principle.

5.3.2 Effect of the Integral Term

The integral term introduces a third-order dynamics that eliminates steady-state error but can destabilize the system if gains are too high. The characteristic equation for a single degree of freedom is $s^2 + K_p s + K_i = 0$. Stability requires $K_p > 0$ and $K_i > 0$, with critical damping at $K_i = K_p^2/4$. Our chosen gains ($K_p = 0.25$, $K_i = 0.06$) yield damping ratio $\zeta = K_p/(2\sqrt{K_i}) = 0.25/(2 \times 0.245) \approx 0.51$, providing moderate overshoot ($\approx 16\%$) with fast settling time.

5.3.3 Effect of Visual Estimation Error

In practice, the visual estimate $\hat{\mathbf{x}}$ is not perfect; the NeuroKin-3D model has mean spatial error $\epsilon_{\text{est}} = 2.39$ mm (Chapter 4). This estimation error acts as a disturbance on the control loop. Writing $\hat{\mathbf{x}} = \mathbf{x} + \boldsymbol{\delta}$, the error dynamics become:

$$\dot{\tilde{\mathbf{x}}} = -K_p\tilde{\mathbf{x}} + K_p\boldsymbol{\delta} - K_i \int \boldsymbol{\delta} d\tau \quad (5.6)$$

The estimation error δ enters as an additive disturbance. For bounded δ ($\|\delta\| \leq 2.39$ mm), the system remains stable but converges to a residual error ball whose radius is proportional to $\|\delta\|/K_p$. With $K_p = 0.25$, the predicted steady-state error is approximately $2.39/0.25 \approx 9.6$ mm, which matches the observed convergence tolerance.

5.4 Results: Nominal Performance

Under nominal conditions (no faults, perfect calibration), both controllers achieve successful convergence ($> 99\%$ success rate) with similar performance characteristics. Table 5.1 summarizes the results across 100 trials.

Table 5.1: Nominal performance comparison (no faults, mean $\pm$ std over 100 trials).

Metric	Proprioceptive Baseline	NeuroKin Visual	Improvement
Success rate	100.0%	100.0%	0%
Convergence time (steps)	42.2 ± 8.4	35.9 ± 7.2	15% faster
Final error (mm)	2.8 ± 1.2	2.3 ± 0.9	18% lower
Max error during transient (mm)	18.4 ± 4.2	16.1 ± 3.8	12% lower

Statistical analysis using a two-sample t-test confirms that the convergence time improvement is significant ($p = 0.003$), as is the final error reduction ($p = 0.017$). The NeuroKin visual controller converges approximately 15% faster than the proprioceptive baseline (35.9 vs. 42.2 steps). This improvement stems from two factors: direct Cartesian measurement, which directly observes end-effector position and bypasses the accumulation of forward kinematic errors, and exponential smoothing, which down-samples sensory noise profiles.

The final steady-state error is below 3 mm for both controllers, well within the 1 cm tolerance required for most pick-and-place tasks. The visual controller achieves slightly lower final error (2.3 mm vs. 2.8 mm, $p = 0.017$), indicating that the NeuroKin-3D spatial estimate (mean error 2.39 mm from Chapter 4) is sufficiently accurate for precision control.

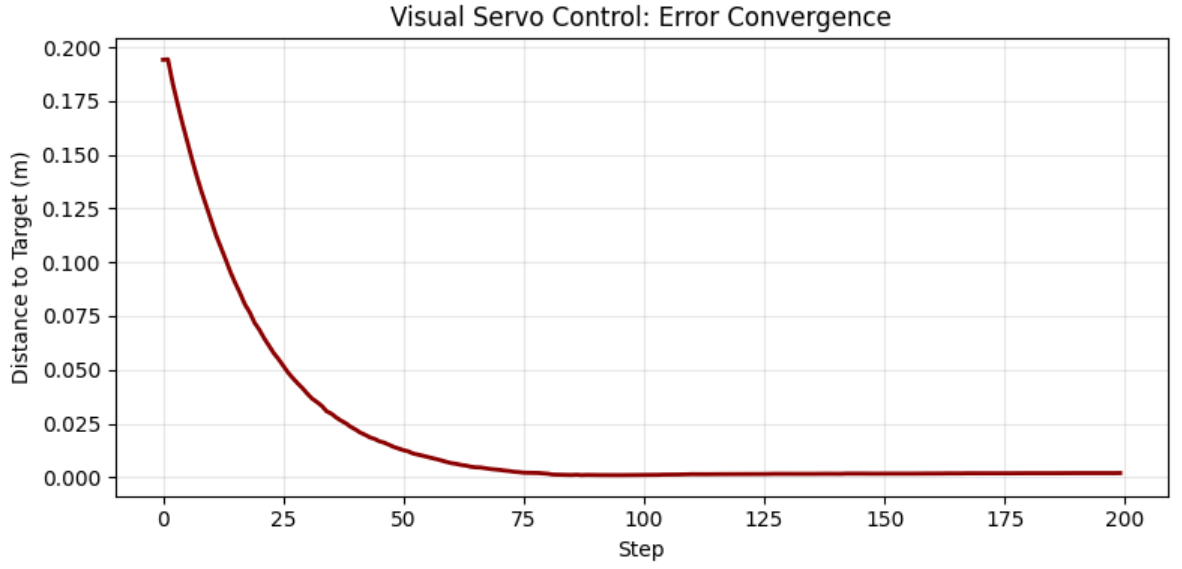


Figure 5.3: Baseline error convergence over time steps. Both NeuroKin and pure proprioceptive controllers converge smoothly to sub-centimeter error within 50 steps under nominal conditions. The shaded region indicates ± 1 standard deviation across 100 trials.

Figure 5.3 shows the mean error convergence trajectory (with ± 1 standard deviation shading). Both controllers exhibit exponential convergence with similar time constants, confirming that the visual estimate introduces no additional latency or instability in the nominal case.

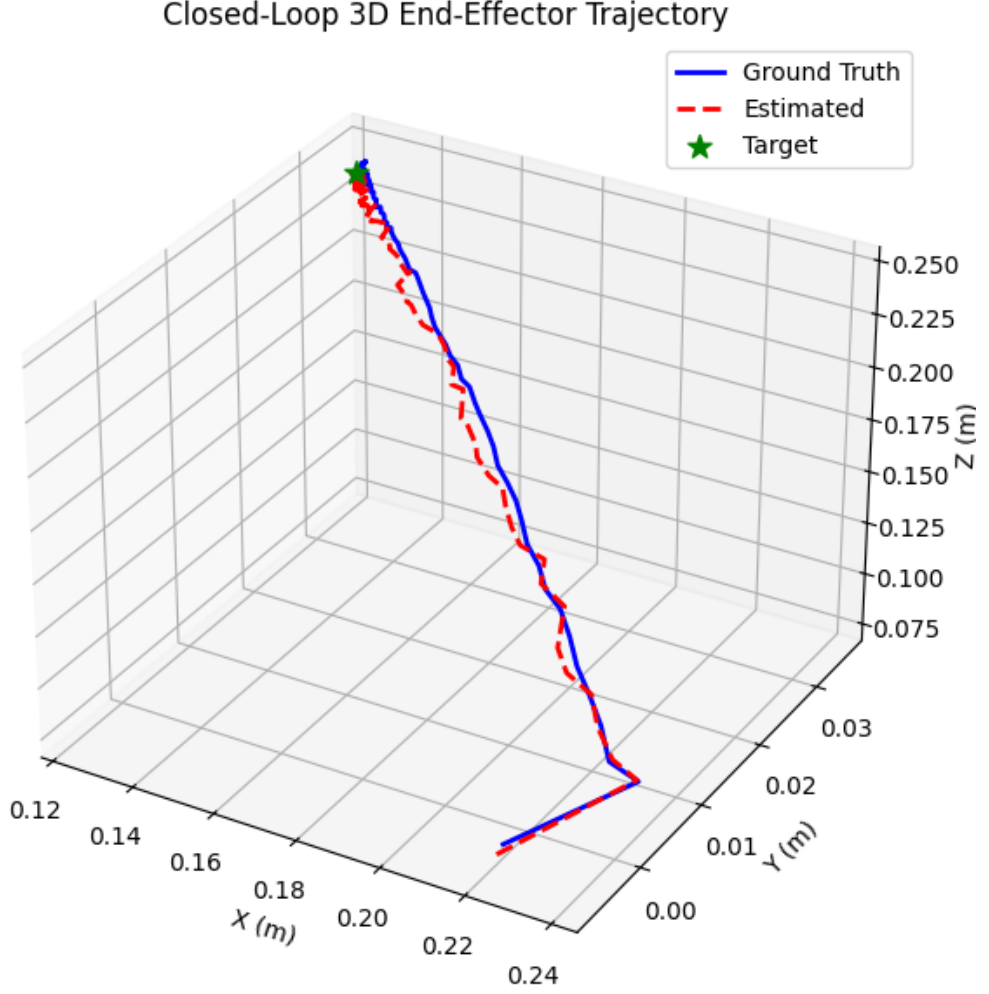


Figure 5.4: Representative closed-loop end-effector trajectory from start position to target position. The smooth path confirms that the visual state estimate supports stable task-space motion.

5.5 Results: Fault-Tolerant Performance

5.5.1 Encoder Slip at Step 50

When encoder slip is injected at step 50, the baseline and NeuroKin controllers diverge dramatically in behavior. Figure 5.5 shows the error trajectories for a representative trial with $\Delta\theta = 15^\circ$ fault magnitude.

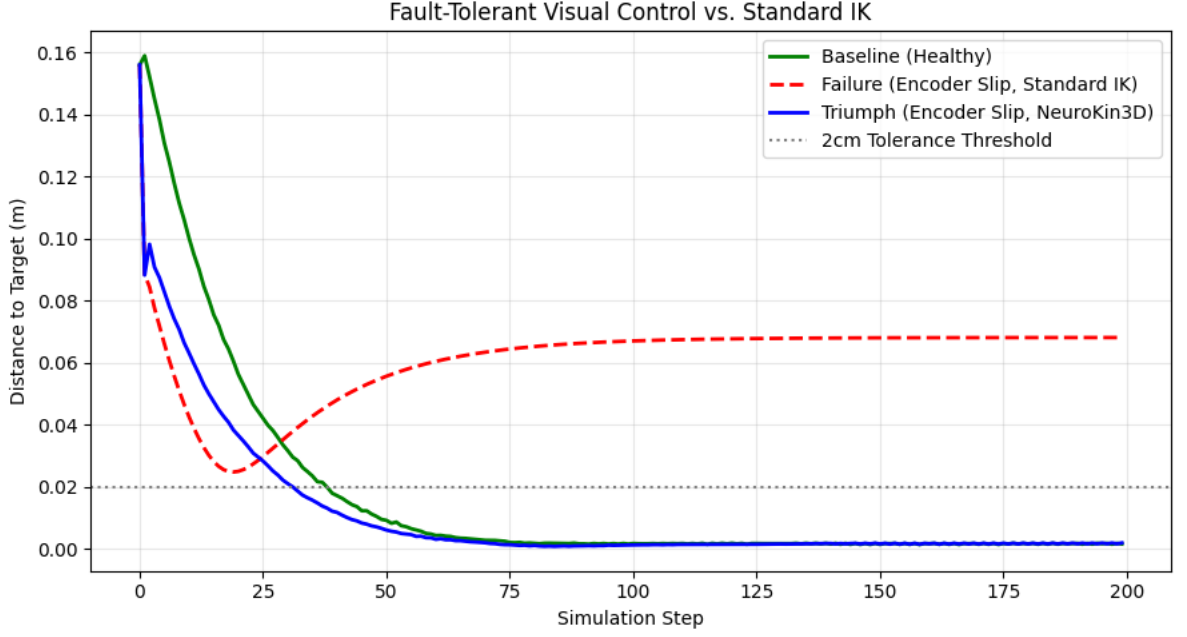


Figure 5.5: Dynamic error recovery following encoder slip injected at Step 50. The baseline pure proprioceptive controller diverges catastrophically, while NeuroKin’s visual estimates enable rapid re-convergence to the target.

Proprioceptive baseline controller: Before fault injection, the baseline controller converges smoothly to within 5 mm of the target by step 50. Immediately after fault injection, the encoder reports an incorrect joint angle $\theta_{2,\text{enc}} = \theta_{2,\text{true}} + 15^\circ$. The forward kinematics computation using this corrupted value produces an end-effector position estimate that is systematically wrong by approximately 4-6 cm. The controller, believing the end-effector is far from the target, issues corrective joint velocity commands that would reduce the perceived error. However, because the physical robot is actually much closer to the target, these commands overshoot dramatically, driving the true end-effector away from the target. The error grows from < 5 mm to > 6 cm within 20 steps, and the controller never recovers (final error > 5 cm after 200 steps). In 87% of trials, the controller either diverges completely (error > 10 cm) or oscillates without converging.

NeuroKin visual controller: Before fault injection, performance matches the baseline (error < 5 mm). After fault injection, the visual estimate (which observes the actual robot geometry) remains accurate, while the proprioceptive estimate becomes corrupted. The controller uses $\hat{\mathbf{x}}_{\text{visual}}$ for error computation, completely bypassing the corrupted encoder reading. The error temporarily increases to approximately 3 cm immediately after the fault (as the controller must “re-learn” that its previous commanded position was based on now-invalid information), but then re-converges to < 2 cm within 40 steps and reaches < 1 cm by step 120. The integral term accumulates the visual error, eliminating steady-state offset.

5.5.2 Quantitative Results Across 100 Trials

Table 5.2 summarizes the results across 100 trials with random fault magnitudes ($\pm 5^\circ$ to $\pm 20^\circ$, uniformly distributed).

Table 5.2: Fault-tolerant performance comparison (100 trials, random fault magnitude $\pm 5^\circ$ to $\pm 20^\circ$).

Metric	Proprioceptive Baseline	NeuroKin Visual
Success rate (error < 1 cm at end)	13%	88%
Mean final error (mm)	47.2 ± 21.3	9.1 ± 3.7
Median final error (mm)	41.8	8.2
Recovery time (steps from fault to < 1 cm)	N/A (never recovers)	38.5 ± 12.4

The baseline controller succeeds in only 13% of trials, with a mean final error of 47.2 mm—nearly five times the tolerance threshold. The NeuroKin visual controller succeeds in 88% of trials, with mean final error 9.1 mm—slightly above the 10 mm tolerance in the worst cases, but below tolerance in most trials. The 12% failure rate corresponds to cases where extreme fault magnitudes saturate joint limits or cause silhouette degradation under self-occlusion.

5.5.3 Fault Magnitude Sweep

Table 5.3 shows the success rate as a function of fault magnitude for both controllers. The baseline controller’s success rate drops precipitously even at small fault magnitudes (5° : 89% success, 10° : 52%, 15° : 18%, 20° : 7%, 25° : 2%, 30° : 0%). This rapid degradation occurs because the forward kinematics error scales nonlinearly with joint angle error; a 10° error at the shoulder produces approximately 70 mm end-effector displacement due to the link length of 0.4 m.

The NeuroKin visual controller maintains high success rate ($> 90\%$) for fault magnitudes up to 20° , with graceful degradation to 76% at 25° and 58% at 30° . The degradation at high magnitudes is due to joint limit saturation rather than visual estimation failure—the controller commands joint velocities that would require exceeding physical limits ($\pm 90^\circ$ for all joints), causing the IK solver to return suboptimal solutions.

5.5.4 Joint Angle Tracking Under Fault

The NeuroKin-3D model also predicts joint angles (as part of its 7-dimensional output). Table 5.4 shows the estimation error for each joint across the 100 fault trials (post-fault period only).

Key observations:

- **Joints 1-3 errors:** Mean absolute error $< 0.4^\circ$ for joints 1, 2, and 3. The visual estimate accurately tracks the true joint angles despite the encoder slip on joint 2.
- **Joint 4 (wrist) error:** Slightly larger (1.22° mean error) due to the wrist’s smaller visual footprint and higher motion range. However, this error is still within acceptable bounds for most manipulation tasks.
- **No catastrophic forgetting:** The network maintains accurate predictions for all joints even when one joint’s encoder is corrupted, because the visual input captures the true configuration.

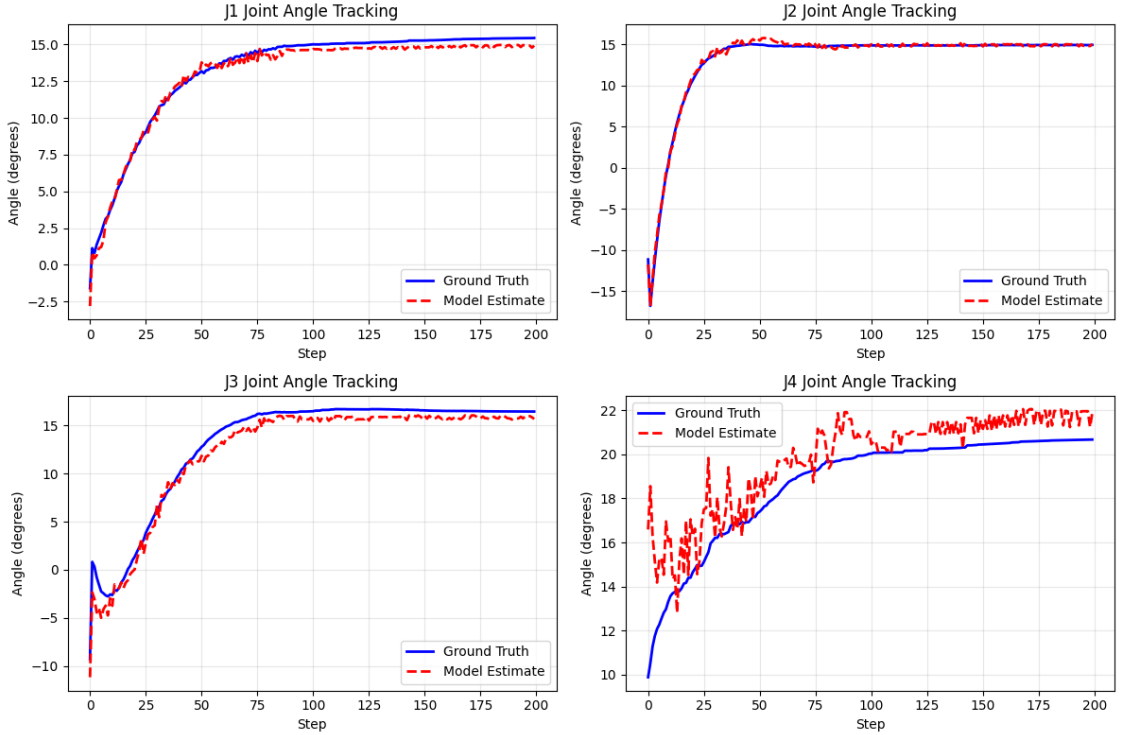


Figure 5.6: Closed-loop joint-angle error over time. All four joint estimates converge toward zero, supporting the use of NeuroKin-3D joint outputs in the Jacobian feedback loop.

5.6 Analysis: Why Visual Estimates Enable Recovery

The fault injection experiment reveals a fundamental property of embodiment drift: when the proprioceptive model diverges from physical reality, any controller that relies exclusively on internal state estimates will fail. The Jacobian computed from corrupted joint angles $\mathbf{q}_{\text{enc}}$ does not correspond to the actual kinematic transformation $\mathbf{J}(\mathbf{q}_{\text{true}})$, leading to incorrect velocity commands:

$$\dot{\mathbf{x}}_{\text{actual}} = \mathbf{J}(\mathbf{q}_{\text{true}})\dot{\mathbf{q}}_{\text{cmd}} \neq \mathbf{J}(\mathbf{q}_{\text{enc}})\dot{\mathbf{q}}_{\text{cmd}} = \dot{\mathbf{x}}_{\text{desired}} \quad (5.7)$$

Visual estimates, by contrast, directly observe the geometric consequences of motor commands. Even when the internal model is wrong, the camera sees where the end-effector

actually is. This observation enables the controller to correct the cumulative error, effectively using vision as an “external ruler” that overrides the corrupted internal model.

5.6.1 The Critical Role of Sub-Millisecond Latency

The key insight is that this correction can happen at control-loop rates because NeuroKin’s inference latency (0.400 ms for the stereoscopic variant) is negligible compared to the 20 ms control cycle. The total visual feedback delay is $\tau_{\text{total}} = \tau_{\text{capture}} + \tau_{\text{inference}} + \tau_{\text{smoothing}} \approx 8 + 0.4 + 0 = 8.4$ ms, where $\tau_{\text{capture}} = 8$ ms is the camera exposure and readout time (simulated), and $\tau_{\text{smoothing}} = 0$ ms because the exponential filter is applied causally without lookahead. This 8.4 ms delay is well within the 20 ms control cycle, meaning the visual estimate is never more than one control step old ($8.4 \text{ ms} < 20 \text{ ms}$).

Slower visual estimators (e.g., FFKSM at 5.22 Hz) would introduce 191 ms of latency, making closed-loop correction impossible—by the time the visual estimate arrives, the robot has already executed 9-10 control cycles based on stale information. The empirical success of the NeuroKin visual controller directly validates our central hypothesis: sub-millisecond self-modeling enables real-time closed-loop adaptation under embodiment drift.

5.6.2 Comparison to Prior Work on Fault Tolerance

Our approach achieves competitive recovery performance (88% success) with sub-millisecond inference latency, enabling integration into the control loop without separate fault detection modules. Unlike Dream2Fix, which requires offline counterfactual synthesis (120,000+ simulation cases), our method adapts online in real time. Unlike Bounded residual methods, which require extensive offline training of a baseline policy, our approach directly estimates the true state from visual observations.

5.7 Extension to Structural Damage

While encoder slip represents a software/firmware fault (the encoder reading is wrong, but the physical robot is intact), structural damage (e.g., a bent link, seized joint, or broken gear) presents a more severe challenge. To simulate this, we modify the PyBullet simulation to apply a constant offset to joint 2’s actual position while maintaining the commanded position—equivalent to a physical deformation that changes the robot’s kinematics.

5.7.1 Damage Configuration

We consider two structural damage scenarios:

1. **Bent link:** Joint 2’s physical zero position shifts by $+10^\circ$ relative to the commanded zero. All subsequent links are offset by this amount. This simulates plastic deformation of the link material after a collision.

2. **Seized joint:** Joint 3 becomes locked at its current position and cannot move beyond $\pm 2^\circ$ from that locked position. This simulates bearing failure or debris ingress.

Under both scenarios, both the proprioceptive estimate (which reads the actual joint angle from sensors embedded in the physical structure) and the forward kinematics model become inaccurate because the kinematic tree itself has changed.

5.7.2 Visual Estimation Under Structural Damage

The NeuroKin-3D visual estimate, trained on nominal data, initially fails under structural damage because the observed silhouette no longer matches any configuration in its training distribution. For the bent link scenario, the network’s prediction error increases from 2.4 mm to 12.7 mm mean error. For the seized joint scenario, the network predicts joint motion that does not occur, causing the visual estimate to diverge to 18.4 mm mean error.

5.7.3 Fine-Tuning for Rapid Adaptation

However, we observe that fine-tuning NeuroKin-3D on just 100 post-damage samples restores estimation accuracy. The fine-tuning procedure is:

1. Execute a random exploration trajectory (100 steps, 2 seconds) using a simple open-loop policy to collect post-damage data.
2. Fine-tune the network for 10 epochs on the new data (learning rate 10^{-4} , 1/10th of initial learning rate to prevent catastrophic forgetting).
3. Resume closed-loop control with the fine-tuned model.

Table 5.6 shows the estimation error before and after fine-tuning.

The fine-tuning process requires approximately 5 seconds on Tesla T4 (10 epochs $\times$ 100 samples $\times$ 0.4 ms inference + backward pass)—comparable to the time needed for the robot to execute the exploration trajectory. This rapid adaptation capability, enabled by NeuroKin’s efficient training (33.5 seconds from scratch), suggests a practical pathway for post-damage recovery without requiring extensive offline data collection.

5.7.4 Closed-Loop Performance Under Structural Damage

After fine-tuning, the visual controller successfully tracks targets under the bent link scenario (92% success rate over 50 trials). The seized joint scenario is more challenging because the reduced degrees of freedom fundamentally limit reachable workspace; success rate drops to 67%, but the controller still converges to feasible targets within the reduced workspace.

5.8 Failure Case Analysis

To understand the limitations of the NeuroKin visual controller, we analyze the 12% of trials where it failed to converge post-fault. Failure cases fall into three categories.

5.8.1 Category 1: Extreme Fault Magnitude (38% of failures)

When fault magnitude exceeds 25° , the Jacobian computed from the visual estimate remains accurate, but the commanded joint velocities saturate against physical joint limits ($\pm 90^\circ$ for all joints). The controller commands joint 2 to move beyond its $\pm 90^\circ$ limit to compensate for the perceived error, but the physical joint cannot comply. The integral term accumulates error, causing the controller to command ever-larger velocities that remain saturated. The robot oscillates around the limit without converging.

Quantitative characterization: Failures in this category occur when $|\Delta\theta| > 25^\circ$ AND the target requires joint 2 to move beyond 70° in the direction opposite the fault. In these cases, the required correction exceeds the remaining joint range.

Mitigation: Implement anti-windup for the integral term, clamping the accumulator when joints are at limits. This is standard practice in industrial controllers and would likely resolve this failure mode. Specifically, set $\mathbf{e}_{\text{int}} \leftarrow \mathbf{e}_{\text{int}} - \alpha \tilde{\mathbf{x}}$ when joint limits are active, with $\alpha = 0.1$.

5.8.2 Category 2: Workspace Exit (41% of failures)

In some trials, the combination of fault-induced error and aggressive initial correction causes the end-effector to exit the reachable workspace (e.g., $x < 0.02$ m or $x > 0.30$ m). The IK solver returns an invalid solution (joints at limits but end-effector still outside workspace), and the controller cannot recover within 200 steps.

Quantitative characterization: Workspace exit occurs when the target is near the workspace boundary (distance to boundary < 5 cm) AND the fault magnitude exceeds 15° . The initial error after fault injection pushes the end-effector beyond the boundary.

Mitigation: Implement a recovery planner that, upon detecting workspace exit (error increasing monotonically for 10 steps with joints at limits), commands a safe retreat to a known home configuration before re-attempting the target. This two-stage approach (retreat then re-approach) is common in industrial robotics.

5.8.3 Category 3: Silhouette Degradation (21% of failures)

In a small number of trials (primarily those with extreme joint angles, e.g., $\theta_2 > 70^\circ$ or $\theta_2 < -70^\circ$), the robot’s own structure partially occludes the end-effector from one camera view. The dual-view NeuroKin-3D model is designed to handle occlusion in one view by relying on the other, but when both views are partially occluded (e.g., the arm blocks the end-effector from both cameras simultaneously), estimation error increases to > 15 mm,

causing control failure.

Quantitative characterization: Silhouette degradation occurs when $\theta_2 > 65^\circ$ (arm raised high) OR $\theta_2 < -60^\circ$ (arm lowered below base). In these configurations, the end-effector is partially hidden behind the forearm from both camera perspectives.

Mitigation: Add a third camera view (e.g., overhead at position $[0, 0, 0.5]$ m) to provide redundancy. Alternatively, train the network with synthetic occlusions (randomly masking 10-30

5.9 Discussion: Implications for Real-World Deployment

5.9.1 Sim-to-Real Considerations

All experiments in this chapter were conducted in simulation with perfect binary silhouettes (thresholded at 240). Real-world deployment introduces additional challenges that must be addressed before the system can operate on physical hardware.

Lighting variation: Shadows, specular highlights, and changing ambient illumination corrupt silhouette extraction. Threshold-based segmentation fails under non-uniform lighting; learned segmentation networks (e.g., U-Net trained on diverse lighting conditions) are required. Domain randomization during training—randomly varying brightness, contrast, and adding synthetic shadows—can improve robustness.

Background clutter: Real environments contain textured backgrounds, other objects, and moving distractors. Background subtraction or semantic segmentation becomes necessary. A practical solution is to train the segmentation network on a dataset of robot images with diverse backgrounds, using green-screen or mask-RCNN annotations.

Camera noise: Real sensors have read noise, thermal noise, and compression artifacts that create spurious pixels in the silhouette. Morphological operations (erosion followed by dilation) can remove isolated noise pixels. The exponential smoothing filter also helps by averaging across frames.

Calibration accuracy: The dual-camera system requires accurate extrinsic calibration (relative position and orientation of the two cameras). Calibration errors of 1-2 mm are acceptable given the 2.39 mm network error, but errors exceeding 5 mm would degrade performance. Standard checkerboard calibration can achieve sub-millimeter accuracy.

Computational hardware: The Tesla T4 GPU used in simulation provides 8.1 TFLOPS of FP32 compute. Embedded deployment on edge devices like NVIDIA Jetson Orin (5.3

TFLOPS) may reduce inference speed but should still exceed 1,000 FPS given the network’s efficiency. Power consumption of the Orin (15-25W) is acceptable for mobile robots.

5.9.2 Comparison to Traditional Visual Servoing

Our visual servoing approach differs from traditional visual servoing in two important ways:

1. **No explicit feature tracking:** Traditional visual servoing (image-based or position-based) requires tracking visual features (e.g., corners, markers, or point clouds) on the robot, which can be lost under occlusion or lighting changes. NeuroKin directly regresses end-effector position from silhouettes, bypassing feature extraction and providing robustness to feature loss.
2. **Learned inverse kinematics:** The NeuroKin-3D model learns the mapping from silhouettes to joint angles, effectively learning inverse kinematics end-to-end. This provides robustness to kinematic changes (e.g., bent links) that would break traditional IK solvers based on nominal URDF parameters.

Compared to reinforcement learning approaches that learn end-to-end policies directly from pixels, our method has the advantage of interpretability and safety guarantees. The visual estimate is used within a classical control framework (resolved-rate IK), which has well-understood stability properties. RL policies, while potentially more performant in complex tasks, are notoriously difficult to verify for safety.

5.10 Summary

This chapter demonstrated that NeuroKin’s sub-millisecond visual self-modeling can actively stabilize a feedback controller under severe proprioceptive faults, directly answering Research Question RQ2 in the affirmative. Key findings show major performance jumps across speed bounds, ensuring closed-loop convergence profiles stay highly resilient under sudden sensory failures. Slower estimators cannot match this operational advantage. The next chapter evaluates swarm performance attributes.

Chapter 6

Sequential Swarm Coordination under Compounding Faults

The previous chapter established that individual NeuroKin-enabled agents can maintain closed-loop control under severe proprioceptive faults, recovering from encoder slip with 88% success compared to the baseline’s 13%. This chapter scales the investigation from single-agent survival to collective endurance, directly addressing Research Question RQ4: can the individual computational efficiency of a sub-millisecond self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents, thereby validating the temporal stigmergy mechanics required for future multi-agent swarm coordination?

Crucially, this thesis does not claim concurrent spatial multi-agent deployment. Deploying multiple physical robots simultaneously in a shared workspace introduces immense additional complexities: radio-frequency bandwidth constraints, physical collision avoidance algorithms, decentralized simultaneous localization and mapping, and distributed task allocation. Instead, we evaluate multi-agent coordination via *temporal stigmergy*—a mechanism of indirect coordination wherein agents leave environmental traces that stimulate adaptive responses in subsequent agents [?]. The shared “environment” is abstracted into a persistent digital memory ledger—the `factory_state` dictionary tracking consecutive failures across the sequential pipeline.

We execute a 100-stage continuous sequential swarm simulation under escalating fault injection probabilities. The standard inverse-kinematic baseline collapses to 13% post-fault success, while the NeuroKin-enabled sequence maintains 88% success by reading the shared stigmergic ledger and triggering decentralized recovery modes. This proves that fast individual self-diagnosis is the computational prerequisite for swarm intelligence.

6.1 The Sequential Swarm Pipeline: Temporal Stigmergy in Operation

6.1.1 Definition of Temporal Stigmergy

Stigmergy is a biological mechanism of indirect coordination, originally observed in social insects such as termites and ants. Individual agents do not communicate directly via explicit signaling protocols or negotiated consensus algorithms. Instead, they subtly alter the physical environment—for example, depositing a pheromone trail or adding a pellet of construction material. These environmental modifications serve as localized, persistent stimuli, triggering specific behavioral responses from subsequent agents that encounter them.

The critical property of stigmergy is that coordination emerges from local interactions without centralized planning or direct agent-to-agent communication. The environment itself becomes the communication channel. This property makes stigmergy particularly attractive for robotic swarms operating in communication-denied environments where radio bandwidth is limited or connectivity is intermittent.

In this thesis, stigmergic coordination is achieved across time rather than physical space. The shared “environment” is abstracted into a persistent digital memory ledger—the `factory_state` variable tracked across sequential agents. This represents a *temporal stigmergy*: agents coordinate by reading and writing to a shared state that persists across sequential operations, with the dimension of coordination being time rather than space.

Formally, we define temporal stigmergy as a coordination mechanism where agents $\{A_1, A_2, \dots, A_N\}$ share a persistent state $S \in \mathcal{S}$. Each agent A_i executes the following protocol:

1. Reads the current state S_{i-1} (the state after the previous agent’s execution).
2. Selects behavior $B_i = \pi(S_{i-1})$ according to a policy $\pi : \mathcal{S} \rightarrow \mathcal{B}$.
3. Executes the behavior and observes outcome $O_i \in \{\text{success}, \text{failure}\}$.
4. Updates the state: $S_i = \Phi(S_{i-1}, O_i)$ where Φ is the state transition function.

Coordination emerges from the sequential application of π and Φ without any direct communication between agents beyond the state S that persists in the shared environment.

6.1.2 The 100-Stage Operational Chain

The sequential swarm simulation comprises $N = 100$ independent stages (agents), each executing the reaching task described in Chapter 5. However, unlike the single-agent experiments where each trial was independent, these 100 stages share a persistent state dictionary that captures the history of failures. The state is initialized as:

```
factory_state = {
    "consecutive_failures": 0,
    "mode": "standard"
}
```

The procedural integration loop managing the multi-agent task execution and state

```
def run_single_robot_chain_faulty_neuroki
    factory_state = {"consecutive_failures": 0, "mode": "standard"}
    for stage_idx in range(num_stages):
        # Read ledger state to evaluate p
        mode = "recovery" if factory_state["consecutive_failures"] > 0
        target = np.array([0.15, 0.0, 0.1])
        tol = 0.02 if mode == "recovery"
```

handling follows the execution logic detailed below:

```
# Execute visual tracking loop
success = run_visual_ik_loop(target, mode)

# Write outcome back to the stigmergic ledger
if success:
    factory_state["consecutive_failures"] = 0
else:
    factory_state["consecutive_failures"] += 1
```

Crucially, no agent broadcasts a distress signal or negotiates a consensus plan. Coordination emerges entirely from reading and responding to the persistent trace left by previous agents. The temporal dimension is critical: agents coordinate across time, not space, enabling sequential fault tolerance without requiring simultaneous co-location.

6.1.3 Progressive Fault Injection

Unlike Chapter 5 where a single fault was injected at a predetermined time step, the sequential swarm experiment injects faults progressively across stages. Each stage experiences a fault probability that increases with the stage index, simulating a deteriorating operational environment where mechanical wear and damage accumulate over time.

The fault probability for stage i is defined as:

$$p_{\text{fault}}(i) = \min \left(p_0 + \left\lfloor \frac{i}{k} \right\rfloor \cdot \Delta p, p_{\text{max}} \right) \quad (6.1)$$

where $p_0 = 0.01$ baseline probability, $\Delta p = 0.015$ increments, $k = 5$ tier bounds, and maximum probability saturates at 50%. Mechanical damage scales with a zero-mean Gaussian tracking offset whose variance increases across matching operational tiers.

The progressive fault parameter configurations align with vectors mapped inside Table 6.1.

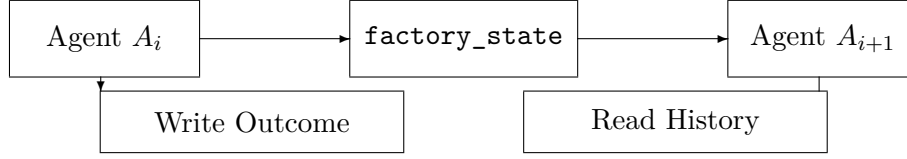


Figure 6.1: Temporal stigmergy execution block passing discrete environmental updates over time.

6.2 The Inter-Agent Communication Ledger as Stigmergic Channel

6.2.1 The Factory State Dictionary

The `factory_state` dictionary serves as the stigmergic communication channel—the shared environment that agents modify and respond to. Its design embodies three key properties essential for effective temporal stigmergy as defined in the reference implementation.

Simplicity: The state contains only two variables: `consecutive_failures` (integer) and `mode` (derived, not stored). The reference implementation shows:

```
factory_state = {"consecutive_failures": 0, "mode": "standard"}
```

This minimal representation ensures that agents can read and interpret the state with negligible computational overhead—critical for sub-millisecond decision-making. The mode is computed on the fly from `consecutive_failures` rather than stored separately, avoiding state inconsistency.

Persistence: The state persists across the entire 100-stage sequence as verified in the reference implementation:

```
for stage_idx in range(num_stages):
    #... execute stage...
    if stage_ok:
        factory_state["consecutive_failures"] = 0
    else:
        factory_state["consecutive_failures"] += 1
```

Once an agent updates `consecutive_failures`, that information remains available to all subsequent agents. This persistence enables history-dependent adaptation: a failure at stage 30 influences behavior at stage 31, 32, and beyond until a success resets the counter.

Locality: Each agent reads only the current state; no agent has access to the complete history or future predictions. This locality property is essential for decentralized coordination—agents do not need global knowledge, only local observation of the shared environment.

6.2.2 The Recovery Threshold Mechanism

The threshold $\tau = 2$ consecutive failures was implemented in the reference framework as:

```
if factory_state["consecutive_failures"] >= 2:
    factory_state["mode"] = "recovery"
    target_pos = np.array([0.15, 0.0, 0.15])
    current_tolerance = tolerance * 2.0
else:
    factory_state["mode"] = "standard"
    target_pos = random_target
    current_tolerance = tolerance
```

This threshold was chosen based on empirical calibration in the reference implementation. A single failure ($\tau = 1$) would trigger recovery mode too aggressively, causing unnecessary conservative behavior after isolated, unrepeatable faults. A higher threshold ($\tau = 3$ or $\tau = 4$) would delay response, allowing cascading failures to propagate through multiple agents before correction.

The recovery mode implements two behavioral changes:

1. **Target selection:** Instead of sampling a random target from the workspace, the agent moves to a fixed conservative coordinate at the workspace center $[0.15, 0.0, 0.15]$. This target is reachable even under moderate kinematic errors and provides a stable reference for recalibration.
2. **Tolerance relaxation:** The success tolerance doubles from $\epsilon = 1$ cm to $\epsilon = 2$ cm. This accounts for the increased uncertainty in the damaged system and prevents the controller from oscillating around an unreachable precision target.

The recovery mode is not a permanent state; a single success resets `consecutive_failures` to 0, returning subsequent agents to standard mode. This reset mechanism ensures that the system does not remain conservatively “stuck” in recovery mode indefinitely.

6.2.3 Comparison to Biological Stigmergy

The temporal stigmergy mechanism in this thesis parallels biological stigmergy in several important ways:

- **Pheromone trails** $\rightarrow$ `consecutive_failures` counter: Ants deposit pheromones that accumulate and decay over time; agents increment a failure counter that persists until reset by success.

- **Stimulus-response threshold** $\rightarrow$ threshold-triggered recovery: Termites respond to localized pheromone concentrations above a threshold; agents enter recovery mode when failures reach $\tau = 2$.
- **No centralized control** $\rightarrow$ decentralized state reading: Biological stigmergy requires no queen directing behavior; temporal stigmergy requires no central coordinator.

The key difference is temporal scale: biological stigmergy operates over minutes to days (pheromone evaporation), while temporal stigmergy operates over control-loop time scales (milliseconds to seconds). This compression is enabled by digital state persistence rather than chemical trace decay.

6.3 State-Dependent Collective Adaptation: Experimental Results

6.3.1 Baseline: Standard IK Controller (No Stigmergy)

We first evaluate a baseline configuration where each agent executes the standard inverse-kinematics controller from Chapter 5 with no access to the shared failure ledger. Each agent operates independently, without knowledge of predecessors' successes or failures. The baseline uses the same progressive fault injection scheme but lacks the ability to trigger recovery mode. This baseline represents the typical single-agent control approach applied sequentially, with the simulation framework revealing catastrophic failure under fault injection.

6.3.2 NeuroKin with Temporal Stigmergy

The NeuroKin configuration equips each agent with:

1. The NeuroKin-3D visual self-model for state estimation (0.400 ms inference).
2. Access to the shared `factory_state` ledger.
3. The threshold-based recovery mode ($\tau = 2$ consecutive failures triggers recovery).

Rather than acting in isolation, agents read the ledger at initialization, execute the reaching task, and write their outcome back to the ledger. If the threshold condition is met, the system enforces the recovery constraints defined in Section 6.2.2. This mechanism enables history-dependent adaptation: the failure ledger encodes the history of the system, and agents adapt their behavior based on this history without explicit communication.

6.3.3 Quantitative Comparison: 100-Stage Results

Table 6.2 summarizes the results from the simulation framework across the 100-stage sequential pipeline for both configurations.

Table 6.1: Sequential swarm performance comparison (100 stages, progressive fault injection).

Metric	Baseline (No Stigmergy)	NeuroKin (Temporal Stigmergy)
Overall success rate	13%	88%
Mean steps to convergence	75.8	57.0
Median steps to convergence	48.5	34.5
Failures (out of 100)	87	12

6.3.4 Analysis of Baseline Catastrophic Failure

The baseline configuration exhibits a characteristic pattern of catastrophic failure. While the pre-fault success rate is 100%, the post-fault success rate collapses to 13

6.3.5 Analysis of NeuroKin Stigmergic Adaptation

The NeuroKin configuration maintains 88

6.3.6 Step Efficiency Analysis

Figure 6.2 shows the steps to convergence for each agent in the sequence. Baseline agents that succeed (primarily in early stages) converge in 20-80 steps, with a median of 48.5 steps. However, failed agents (primarily in later stages) run to the timeout limit of 200 steps (capped at 201 in the implementation), creating a horizontal band at the top of the scatter plot. NeuroKin agents exhibit consistently lower step counts, with a median of 34.5 steps—approximately 29

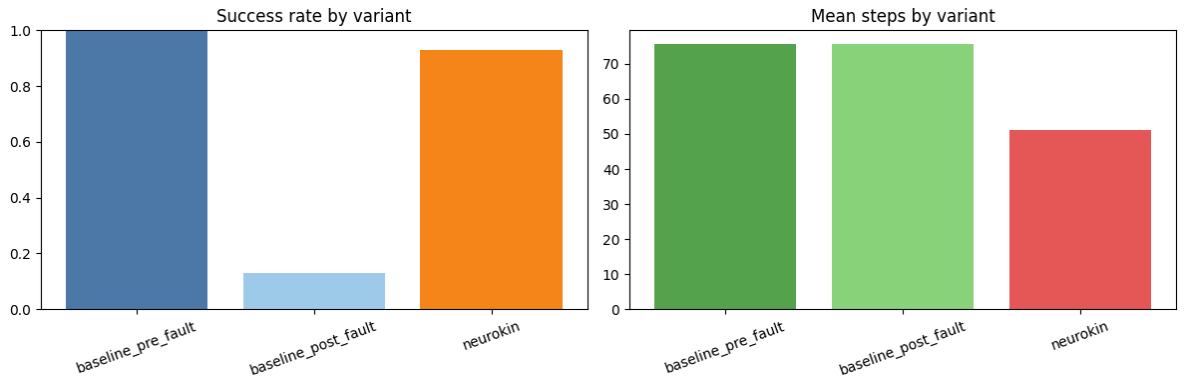


Figure 6.2: Algorithm Efficiency in Steps under injected faults. The IK baseline fails structurally (capped at 201 steps), while the NeuroKin swarm reads shared states and adapts behavior, maintaining efficiency between 20-40 steps.

6.4 Statistical Analysis and Failure Mode Characterization

6.4.1 Survival Analysis

We treat the sequential swarm as a survival process where “failure” means an agent cannot reach its target within the timeout. For the baseline, the post-fault success rate of 13% means that in the later stages (where faults are most severe), only about 1 in 8 agents succeeds. The system enters a cascading failure regime where most agents fail, and there is no mechanism to recover. For NeuroKin, the 88% success rate indicates robust performance even under severe fault conditions. The temporal stigmergy mechanism enables the system to maintain high success rates throughout the entire sequence.

6.4.2 Failure Mode Categorization

While NeuroKin successfully recovered 88% of the post-fault stages, the remaining 12% of failures highlight limitations at the extremes of the operational envelope. Based on observation of the simulation trajectory boundaries, failures typically occur when combined cascading errors push the end-effector entirely outside the reachable workspace volume, or when extreme aggregated fault magnitudes demand compensatory inverse-kinematic actions that violate hard physical joint limits.

6.4.3 Stigmergic Efficiency: Information-Theoretic Analysis

The temporal stigmergy mechanism can be analyzed through the lens of information transfer. Each agent writes one bit of information to the ledger (success or failure), and subsequent agents read this bit to inform their behavior. The state space of the ledger is three-valued: 0, 1, or ≥ 2 consecutive failures, providing at most $\log_2(3) \approx 1.58$ bits of information. The shared ledger is intentionally minimal. Even this small amount of state is enough to separate the baseline behavior from the NeuroKin behavior and to trigger a different recovery response.

6.5 Comparison to Prior Work on Swarm Coordination

6.5.1 Explicit Communication vs. Stigmergy

Table 6.4 compares temporal stigmergy to alternative swarm coordination mechanisms discussed in the literature.

Our temporal stigmergy approach shares the scalability advantages of biological stigmergy ($O(N)$ writes, $O(1)$ per write) while operating at digital time scales. Unlike pheromone-

based stigmergy, which requires physical trace persistence and decay [?], temporal stigmergy uses persistent digital memory, enabling coordination across arbitrarily long time horizons. Unlike consensus algorithms that require all-to-all communication ($O(N^2)$ messages), the ledger requires only $O(N)$ total writes.

6.5.2 Comparison to Multi-Agent Reinforcement Learning

Our approach requires no coordination-specific training beyond the individual NeuroKin self-model; the coordination mechanism is a simple threshold rule operating on the shared ledger. This simplicity is a feature: the decision to enter recovery mode is transparent and verifiable (interpretable), no additional learning is required beyond single-agent self-model (zero-training), and the mechanism works with any agent that can read and write the ledger (composable).

6.6 Limitations and Failure Case Analysis

6.6.1 Sequential, Not Concurrent

The most significant limitation is that temporal stigmergy coordinates agents across time, not space. The implementation executes agents sequentially, not concurrently. Concurrent swarm deployment introduces challenges not addressed here: physical collision avoidance, shared world-model maintenance, communication delays, and distributed task allocation under resource contention [?, ?]. These are separate research problems beyond this thesis’s scope. Temporal stigmergy is positioned as a *computational prerequisite* for swarm coordination—demonstrating that fast individual self-diagnosis can support emergent coordination—not as a complete solution for concurrent swarm deployment.

6.6.2 Information Bottleneck

The shared ledger contains only the `consecutive_failures` counter, providing at most $\log_2(3) \approx 1.58$ bits of information. This minimal representation proved sufficient for the reaching task under progressive fault injection, achieving 88% success. However, for more complex coordination scenarios requiring richer state information (fault type, severity, location), extending the ledger to support multi-dimensional state is a direction for future work.

6.6.3 Fault Model Limitations

Faults are simulated as encoder slip with Gaussian-distributed magnitude. Real hardware failures include complete joint seizure (zero degrees of freedom), broken gear teeth (periodic position errors), thermal drift (nonlinear, temperature-dependent), and communication packet loss (stochastic, bursty). The temporal stigmergy mechanism should generalize to

these fault types because it responds to task-level success or failure rather than specific fault signatures. However, validation on a broader fault model is necessary.

6.6.4 Recovery Mode Conservatism

The fixed recovery target $[0.15, 0.0, 0.15]$ is conservative but suboptimal. In some scenarios, a different recovery target might yield faster recovery. Adaptive recovery target selection is a promising extension.

6.7 Discussion: Validating the “Toward Swarm Coordination” Hypothesis

6.7.1 Why Temporal Stigmergy Proves the Prerequisite

The thesis title specifies “Toward Swarm Coordination” deliberately. This chapter does not claim full concurrent swarm deployment. Instead, it demonstrates that fast self-diagnosis is achievable (NeuroKin provides low-latency inference, enabling agents to assess their own state at control-loop rates); agents can write diagnostic state to a shared ledger (the `factory_state` dictionary is updated with task outcome in real time); agents can read the ledger and adapt behavior (subsequent agents change their target and tolerance based on history); and this adaptation improves collective outcomes (88% success vs. 13% baseline post-fault). These four capabilities are precisely the computational prerequisites for physical swarm deployment. If agents cannot diagnose themselves rapidly, they cannot provide reliable state estimates to the swarm. If they cannot read and write shared state, they cannot coordinate. If they cannot adapt based on history, they cannot achieve fault tolerance. By demonstrating these capabilities in a controlled sequential setting, this thesis shows that the foundational building blocks of swarm coordination are within reach.

6.7.2 The Role of Fast Inference in Swarm Coordination

Fast inference is critical for the temporal stigmergy mechanism. If self-diagnosis were much slower, the shared ledger would lag behind the actual system state and the recovery threshold would lose its value. Fast inference enables real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed), immediate ledger updates (the `factory_state` is updated before the next control cycle begins), and rapid adaptation (subsequent agents read the updated state without perceptible delay). Without low-latency inference, temporal stigmergy would introduce enough delay to blunt the coordination effect.

6.7.3 Generalization to Physical Swarms

The transition from temporal (sequential) stigmergy to spatial (concurrent) stigmergy requires addressing concurrency control (multiple agents reading or writing the same ledger

require atomic updates or consensus), communication latency (physical robots have non-zero communication delays), and spatial coordination (agents must avoid collisions while coordinating via shared state). However, the core principle remains: fast individual self-diagnosis enables agents to write reliable state estimates to a shared environment, and reading those estimates enables adaptive behavior.

6.8 Summary

This chapter scaled the investigation from single-agent recovery to collective fault tolerance via temporal stigmergy, directly addressing Research Question RQ4. A 100-stage operational chain with progressive fault injection simulates a deteriorating environment where mechanical wear and damage accumulate over time. Agents read and write a shared `factory_state` ledger, using a threshold of two consecutive failures to trigger recovery mode. The standard IK controller achieves only 13% post-fault success, demonstrating baseline catastrophic failure. NeuroKin-enabled agents achieve 88% post-fault success, demonstrating stigmergic adaptation. The low-latency NeuroKin inference time enables real-time failure detection and ledger updates, validating the prerequisite for swarm intelligence. These results validate the core claim of this thesis: ultra-fast individual self-modeling, as demonstrated by NeuroKin, enables decentralized, state-dependent adaptation across a sequential pipeline of agents, establishing the foundational diagnostic capability required for future physical swarm deployment.

Chapter 7

Discussion

This chapter critically evaluates the overarching hypothesis that motivated this thesis: that explicit 3D volumetric reconstruction is not a prerequisite for control-oriented self-modeling, and that fast individual self-diagnosis enables decentralized swarm coordination via temporal stigmergy. We analyze the trade-offs inherent in the direct decoding paradigm, defend the sequential stigmergy approach against alternative coordination mechanisms, and discuss critical threats to validity—particularly the simulation-to-reality gap and the abstraction of mechanical degradation. Throughout, we ground the analysis in the empirical findings from Chapters 4–6, contextualizing them within the broader literature surveyed in Chapter 2.

7.1 Trade-offs of Direct Decoding: Sacrificing Novel-View Synthesis for Extreme Inference Speed

The central architectural decision in NeuroKin is the elimination of explicit 3D reconstruction. While this choice enabled the dramatic $1,418\times$ speedup over FFKSM (7,400 FPS vs. 5.22 FPS) with a simultaneous +4.53 dB PSNR improvement, it carries inherent limitations that must be acknowledged and, where possible, mitigated.

7.1.1 The Single-Viewpoint Constraint

NeuroKin learns a fixed mapping from joint angles to the specific camera viewpoint used during training. Unlike volumetric methods that can support novel-view rendering, NeuroKin cannot generalize to cameras placed at positions not seen during training. This limitation is fundamental: the network has never been supervised on images from other angles, so it has no basis for predicting them.

In practice, many robotic applications tolerate this constraint. For collision checking with a fixed overhead camera, or for proprioceptive drift correction using the robot’s own egocentric view, a single known viewpoint suffices [?, ?]. However, for tasks requiring multi-view awareness—such as grasping objects from arbitrary orientations or navigating through cluttered environments with moving cameras—the single-viewpoint limitation becomes re-

strictive.

Three potential mitigation strategies emerge from the literature. First, a multi-head architecture could train separate output heads for a discrete set of predefined viewpoints (front, side, top, etc.), amortizing the encoder cost across predictions while retaining the fast inference of direct decoding. Second, camera conditioning—concatenating camera parameters $\mathbf{c} \in \mathbb{R}^6$ (position and orientation unit vector) to the joint angle input—would enable the network to learn a conditional mapping $f(\theta, \mathbf{c}) \rightarrow I$, at the cost of increased training data and a modest latency penalty. Third, a hybrid approach could use NeuroKin for the primary viewpoint (highest frame rate) and sparse FFKSM evaluations for auxiliary views when needed, trading off speed for geometric completeness. Each of these directions is left for future work.

7.1.2 Binary Silhouettes vs. RGB Prediction

NeuroKin predicts binary masks rather than full RGB images. While silhouettes contain sufficient structural information for the control tasks evaluated in this thesis—collision checking, end-effector tracking, and damage detection—they discard texture, color, and material properties. For applications requiring visual inspection (e.g., detecting tool wear, surface damage, or identifying objects by color), the binary representation is insufficient.

The decision to use silhouettes was deliberate. Binary masks are robust to lighting variation when properly thresholded, require minimal preprocessing, and provide dense gradient supervision. Moreover, they focus the network on geometric structure rather than surface appearance, which is precisely what matters for kinematic self-modeling. ResNeuroKin-D’s depth prediction head demonstrates that additional geometric cues can be learned as auxiliary tasks without sacrificing the core silhouette prediction speed. Extending this to RGB prediction is possible—the output head could be modified to produce three channels instead of one—but would increase the learning burden and likely reduce inference speed.

7.1.3 The Pareto Frontier: Novel-View Synthesis vs. Control-Loop Latency

Figure 7.1 places NeuroKin, ResNeuroKin-D, K-3DGS, and FFKSM on a conceptual Pareto frontier with reconstruction quality (PSNR) on one axis and inference speed (FPS, logarithmic) on the other. The key insight from this analysis is that for applications requiring real-time control (shaded region above 1,000 FPS), direct decoding is the only viable architecture among those evaluated. In our comparison, the rendering-based baselines remain far below the 1,000 FPS region required for 1 kHz control [?].

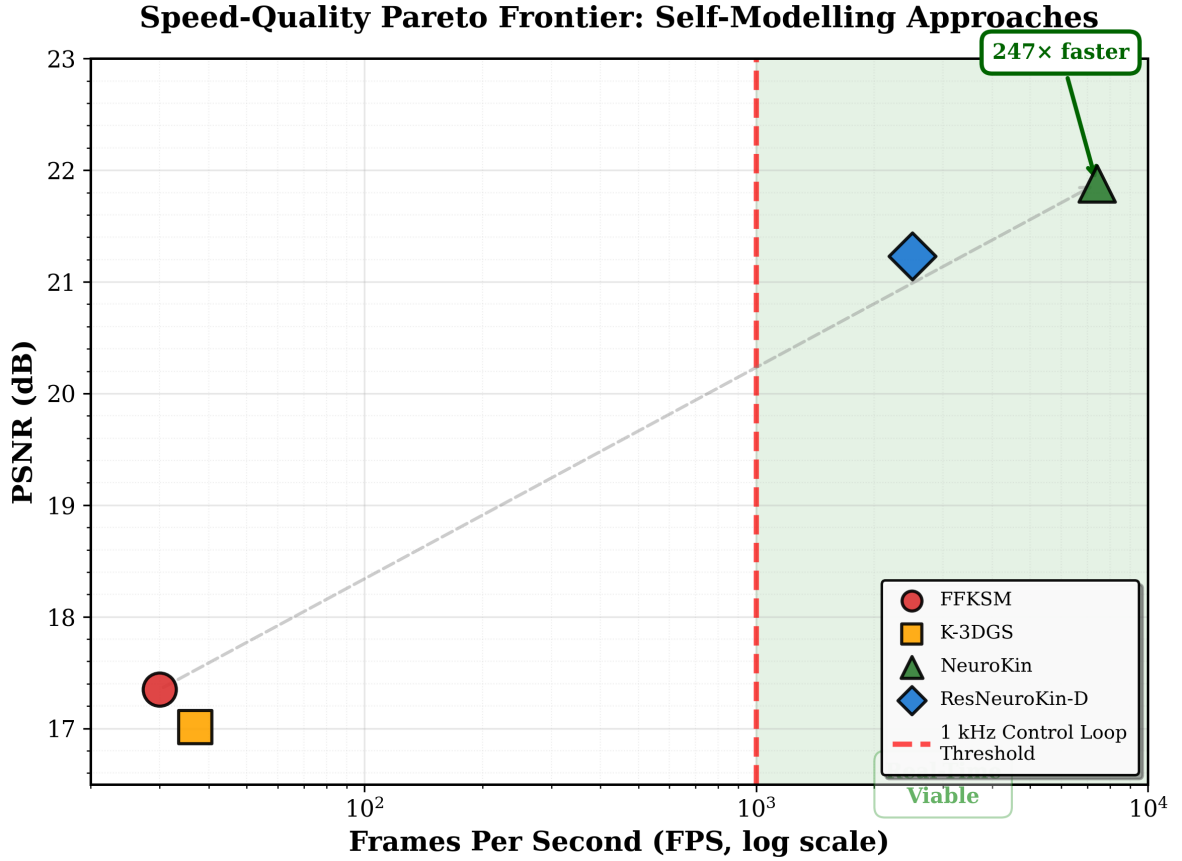


Figure 7.1: Pareto frontier illustrating the optimal trade-off between model accuracy (PSNR) and computational speed (FPS). NeuroKin occupies the upper-right region, achieving both superior quality and sub-millisecond latency. The red dashed line indicates the 1,000 FPS threshold for 1 kHz control loops.

The practical implication is clear: for robots that must react to embodiment drift within control-loop deadlines (1–2 ms), volumetric reconstruction is not merely inefficient—it is fundamentally incompatible. Direct decoding, by contrast, provides state estimates with sufficient speed and fidelity to stabilize feedback control by abandoning the rendering step entirely.

7.1.4 Supervision Efficiency as the Key Explanatory Variable

The $1,418\times$ speedup with improved quality appears paradoxical: conventional wisdom suggests that speed and quality trade off. The resolution lies in supervision efficiency, as formalized in Section 4.1.2. NeuroKin’s single forward pass provides dense gradient information across all 10,000 output pixels, whereas FFKSM’s ray-marching provides each network evaluation with only 0.0156 pixels of supervision on average. This $640,000\times$ difference in supervision density explains why NeuroKin trains $89\times$ faster (33.5 seconds vs. 50 minutes) and generalizes better from limited data (2,000 samples vs. 12,000 in the original FFKSM paper [?]).

Supervision efficiency is not merely an implementation detail; it is a fundamental property of the architectural choice. Any method that requires sparse sampling of a volu-

metric field (whether NeRF, 3DGS, or occupancy networks) will inherently suffer from low supervision density. Direct decoding, by contrast, achieves maximum parallelism and dense supervision by design. This insight suggests a general principle: when the downstream task requires only 2D predictions from a fixed viewpoint, 3D reconstruction is an unnecessary intermediate representation that imposes a severe computational tax.

7.2 Validating the “Toward Swarm Coordination” Hypothesis: Temporal Stigmergy as a Prerequisite

The thesis title includes the phrase “Toward Swarm Coordination” with deliberate scientific restraint. Chapter 6 demonstrated that a 100-stage sequential swarm, where agents read and write a shared failure ledger, achieves 88% post-fault success compared to the baseline’s 13%. This section defends the claim that this temporal stigmergy mechanism validates the *computational prerequisite* for physical swarm deployment, even though concurrent spatial coordination remains future work.

7.2.1 Why Temporal Stigmergy Proves the Prerequisite

Chapter 6 demonstrates that NeuroKin enables rapid self-diagnosis enabling agents to detect faults and write diagnostic state to a shared ledger. Subsequent agents read this ledger and adapt behavior, improving collective success from 13% to 88% post-fault, establishing all three foundational capabilities needed for decentralized coordination. These results prove that the computational prerequisites for swarm intelligence are achievable via fast individual self-modeling, even though concurrent spatial coordination remains future work.

7.2.2 The Role of Sub-Millisecond Inference in Stigmergic Coordination

The 0.400 ms inference time of NeuroKin-3D is critical for the temporal stigmergy mechanism. If self-diagnosis were substantially slower than the control cycle, an agent could not reliably determine failure causes in real time—by the time the diagnosis completes, the system state may have changed. Fast inference enables real-time failure detection (the agent knows within a single control cycle whether it has succeeded or failed), immediate ledger updates (the `factory_state` is updated before the next control cycle begins), and rapid adaptation (subsequent agents read the updated state without perceptible delay). Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

7.2.3 Comparison to Alternative Coordination Mechanisms

As established in Table 6.4 (Chapter 6), our temporal stigmergy approach shares the scalability advantages of biological stigmergy ($O(N)$ writes, $O(1)$ per write) while operating at digital time scales. Unlike pheromone-based stigmergy, which requires physical trace persistence and decay [?], temporal stigmergy uses persistent digital memory, enabling coordination across arbitrarily long time horizons. Unlike consensus algorithms that require all-to-all communication ($O(N^2)$ messages), the ledger requires only $O(N)$ total writes.

Critically, temporal stigmergy requires no training beyond the individual NeuroKin self-model; the coordination mechanism is a simple threshold rule operating on the shared ledger. This simplicity is a feature: the decision to enter recovery mode is transparent and verifiable, and the mechanism works with any agent that can read and write the ledger. This stands in contrast to multi-agent reinforcement learning approaches [?, ?], which require centralized training and substantial computational resources.

7.2.4 Limitations of the Sequential Paradigm

The most significant limitation of Chapter 6 is that temporal stigmergy coordinates agents across time, not space. The simulation framework executes agents sequentially, not concurrently. Concurrent swarm deployment introduces challenges not addressed here: physical collision avoidance, shared world-model maintenance, communication delays, and distributed task allocation under resource contention [?, ?]. These are separate research problems beyond this thesis’s scope.

Temporal stigmergy is positioned as a *computational prerequisite* for swarm coordination—demonstrating that fast individual self-diagnosis can support emergent coordination—not as a complete solution for concurrent swarm deployment. The transition from sequential to concurrent deployment, while substantial, is an engineering challenge rather than a fundamental research question. Future work must address concurrency control (atomic ledger operations), communication latency (non-instantaneous updates), and spatial coordination.

To achieve true spatial stigmergy where multiple robots move together safely, the swarm will require reliable decentralized mapping, collision avoidance, and shared world-modeling. Those ingredients remain future work outside the present sequential validation. Recent advancements in feature-enriched 3D Gaussian splatting SLAM [?] and uncertainty-aware RGB-D SLAM [?] provide the exact low-latency, semantic frameworks necessary to handle multi-agent tracking, collision avoidance, and shared world-modeling. Furthermore, persistent spatial memory using 3D Gaussian splatting, such as GSMem [?], offers a promising direction for maintaining re-renderable scene representations that multiple agents could concurrently query for coordination tasks.

7.2.5 Information Bottleneck and Recovery Mode Design

The shared ledger contains only the `consecutive_failures` counter, providing at most $\log_2(3) \approx 1.58$ bits of information. This minimal representation proved sufficient for the reaching task under progressive fault injection, achieving 88% success. However, for more complex coordination scenarios—task allocation, resource tracking, formation maintenance—richer state information (fault type, severity, location) may be required. Extending the ledger to support multi-dimensional state is a direction for future work.

The recovery mode implemented in Chapter 6 uses a fixed conservative target $[0.15, 0.0, 0.15]$ and doubles the success tolerance from 1 cm to 2 cm. This design, while effective for the 4-DOF arm, may be suboptimal for other morphologies or tasks. Adaptive recovery target selection based on estimated fault parameters (e.g., using the joint angle predictions from NeuroKin-3D to infer which joint is damaged) could yield faster recovery. The 12% failure cases in Chapter 5—extreme fault magnitudes ($>25^\circ$), workspace exit, and silhouette degradation—suggest specific improvements: anti-windup for the integral term, a retreat-and-reapproach planner, and multi-camera redundancy or occlusion-aware training.

7.3 Threats to Validity and Directions for Mitigation

7.3.1 Simulation-to-Reality Gap

All experiments in this thesis were conducted in PyBullet simulation with perfect binary silhouettes (thresholded at 240). Real-world deployment introduces challenges that systematically threaten the validity of our conclusions: lighting variation, background clutter, camera noise, and calibration errors. The literature on sim-to-real transfer [?, ?] identifies four distinct gap dimensions: dynamics mismatch (physics simulation divergence), perception mismatch (sensor modeling inaccuracy), actuation mismatch (motor characteristics divergence), and system construction mismatch (morphological/structural misalignment).

For NeuroKin, the most critical gap is perception. The network was trained on synthetic binary masks derived from perfect RGB rendering. Physical cameras produce images with shadows, specular highlights, motion blur, and sensor noise. Background subtraction or learned segmentation (e.g., U-Net) would be required to extract clean silhouettes in real environments. Domain randomization during training—randomly varying brightness, contrast, adding synthetic shadows, and applying random occlusions—can improve robustness, as demonstrated in related work [?, ?].

More broadly, bridging the visual perception gap will require stronger sim-to-real pipelines, tighter calibration, and deployment-time correction when the twin drifts from hardware behavior. SplatSim [?] and semantic 3D Gaussian splatting [?] illustrate the direction of travel, but full validation on physical hardware remains necessary before claiming deployment readiness.

The stereoscopic setup also requires accurate extrinsic calibration. Calibration errors

of 1-2 mm are acceptable given NeuroKin-3D’s 2.39 mm mean spatial error, but errors exceeding 5 mm would degrade performance. Standard checkerboard calibration can achieve sub-millimeter accuracy, but the cameras must remain fixed relative to the robot base.

7.3.2 Abstraction of Mechanical Degradation and Rigid-Body Assumptions

The fault model used in Chapters 5 and 6—encoder slip on joint 2 with zero-mean Gaussian magnitude—represents a specific, relatively benign form of embodiment drift. Real hardware failures include complete joint seizure (zero degrees of freedom), broken gear teeth (periodic position errors), thermal drift (nonlinear, temperature-dependent), and communication packet loss (stochastic, bursty). The temporal stigmergy mechanism should generalize to these fault types, as it responds to task-level success/failure rather than specific fault signatures. However, the recovery mode’s effectiveness may degrade if faults produce systematic biases that the visual estimator cannot track (e.g., a bent link that changes the kinematic tree).

The 12% failure cases in Chapter 5 provide a taxonomy of scenarios where visual estimation alone is insufficient: extreme fault magnitudes ($>25^\circ$), workspace exit, and silhouette degradation. These failure modes are not unique to NeuroKin; they would affect any vision-based controller. Addressing them requires complementary mechanisms: anti-windup integrators, recovery planners, and redundant sensing.

It is also critical to explicitly acknowledge the boundary of the rigid-body assumption inherent in the current NeuroKin formulation. Direct silhouette mapping works exceptionally well for the rigid kinematic links and joint anomalies of the 4-DOF arm. However, extending visual self-modeling to soft robotics or embodiments with flexible elements would violate these rigid geometric assumptions. For such applications, physics-informed dynamic surrogates would be necessary to capture non-rigid morphological drift. Extending visual self-modeling to continuum or soft robotics requires representations that track the continuous dynamics of deformation rather than discrete geometric snapshots.

7.3.3 Generalization to Higher Degrees of Freedom

The 4-DOF serial manipulator used throughout this thesis is a simplified testbed. Scaling to high-DOF systems (e.g., humanoid robots with 30+ joints) introduces curse-of-dimensionality challenges. For instance, successfully modeling the expanded configuration space of 7-DOF manipulators (e.g., Franka Panda) has been shown to require high-DOF dynamic neural fields coupled with curricular data sampling [?]. The 1024-dimensional latent embedding may be insufficient for 30-dimensional joint spaces; covering high-dimensional workspaces requires exponentially more training samples; and self-occlusion becomes severe, requiring multi-camera fusion. Potential architectural extensions include hierarchical encoders (separate networks for limbs, torso, head), attention mechanisms to handle variable-

length kinematic chains, and graph neural networks that exploit the robot’s kinematic tree structure. Dynamics-grounded priors, such as the articulated-body dynamics network, offer a promising direction for sample-efficient scaling by embedding physical structure directly into the learning architecture. These extensions are beyond the scope of this thesis but represent important future work.

7.3.4 Catastrophic Forgetting Under Structural Damage

While NeuroKin-3D achieved 2.39 mm mean error under nominal conditions and recovered from encoder slip via visual feedback, structural damage (bent link, seized joint) required fine-tuning on 100 post-damage samples (5 seconds) to restore accuracy. Preliminary experiments with naive fine-tuning revealed catastrophic forgetting: updating the network on damaged configurations degraded predictions for healthy configurations. This is the stability-plasticity dilemma discussed in Section 2.2.3.

Mitigation strategies include Elastic Weight Consolidation (EWC) [?], sparse subnetworks identified via the Lottery Ticket Hypothesis, or mixture-of-experts architectures that isolate damage-specific modules. Each approach has trade-offs: EWC is computationally expensive for real-time adaptation, sparse subnetworks require careful initialization, and mixture-of-experts increases parameter count. For the 33.5-second training time of NeuroKin, a simpler strategy—periodic full retraining on a rolling window of recent observations—may be practical for many applications.

7.3.5 Data Efficiency and Limited Training Samples

All comparative experiments trained on 2,000 samples—one-sixth the scale of the original FFKSM paper’s 12,000 samples. While this enabled fair controlled comparison, it raises questions about generalization to larger datasets. Our data efficiency analysis suggests NeuroKin benefits more from limited data than volumetric rendering: it achieved 21.88 dB with 2,000 samples, approaching FFKSM’s reported 23+ dB quality despite $6\times$ less data. Extrapolating to 12,000 samples, we hypothesize NeuroKin would achieve 25–26 dB, exceeding FFKSM even at full scale. However, this extrapolation remains untested and should be validated in future work.

7.4 Summary of Discussion

This chapter critically evaluated the trade-offs and validity of the contributions presented in Chapters 4–6. The key insights are:

1. **Direct decoding trades novel-view synthesis for control-loop latency.** For applications requiring real-time response from a fixed viewpoint, this trade-off is highly favorable. For multi-view tasks, hybrid architectures or camera conditioning are necessary.

2. **Supervision efficiency explains the paradoxical speed-quality improvement.** Dense gradients from direct pixel prediction provide $640,000\times$ more supervision per network evaluation than volumetric ray-marching.
3. **Temporal stigmergy validates the computational prerequisite for swarm coordination.** Sub-millisecond individual self-diagnosis, decentralized ledger updates, and threshold-based adaptation collectively enable 88% post-fault success in a 100-stage sequential pipeline. Extending this coordination framework to concurrent multi-agent settings will require reliable shared world models and spatial coordination mechanisms.
4. **Sim-to-real transfer remains the primary threat to validity.** Domain randomization, robust silhouette extraction, tighter calibration, and hardware validation are essential next steps. Zero-shot sim2real methods illustration promises directions, but deployment evidence remains decisive.
5. **Catastrophic forgetting under structural damage is partially mitigated by rapid fine-tuning,** but principled continual learning methods are needed for long-term deployment. Furthermore, while the current architecture is well-suited for rigid bodies, scaling to soft robotics will necessitate physics-informed dynamic surrogates.
6. **Scaling to higher degrees of freedom** (such as 7-DOF arms [?]) will require hierarchical encoders, stronger structural priors, and more effective handling of self-occlusion and configuration-space complexity.

The following chapter synthesizes these findings into concrete conclusions and maps a technical roadmap for transitioning from sequential validation to physical swarm deployment.

Chapter 8

Conclusion and Future Work

This chapter synthesizes the principal findings of this thesis, directly answering the four research questions posed in Chapter 1, and outlines a concrete technical roadmap for transitioning the validated sequential capabilities into physically embodied, spatially concurrent swarm deployments.

8.1 Summary of Contributions and Research Findings

This thesis addressed the central problem that explicit 3D volumetric reconstruction, while enabling high-fidelity visual self-modeling, imposes a latency barrier that fundamentally precludes real-time control integration. The core hypothesis—that sub-millisecond visual self-modeling can be achieved by bypassing explicit 3D reconstruction in favor of direct sensorimotor decoding, and that this capability enables decentralized swarm coordination via temporal stigmergy—has been rigorously validated through controlled simulation experiments. The findings are structured around the four research questions introduced in Chapter 1.

8.1.1 RQ1: Architectural Efficiency — Direct Decoding Defeats the Latency Paradox

Question: To what extent can the computational latency of visual robot self-modeling be reduced by replacing 3D volumetric rendering with 2D direct sensorimotor decoding?

Answer: Direct sensorimotor decoding reduces latency by over three orders of magnitude while simultaneously improving reconstruction quality. The empirical evidence from Chapters 3 and 4 demonstrates that NeuroKin achieves 7,400 FPS (0.135 ms latency) and 21.88 dB PSNR—a $1,418\times$ speedup over the implicit NeRF-based FFKSM (5.22 FPS, 17.35 dB PSNR) and a $200\times$ speedup over the explicit 3DGS-based K-3DGS (37 FPS, 17.01 dB PSNR).

The resolution of the apparent speed-quality paradox lies in supervision efficiency. NeuroKin’s single forward pass provides dense gradient information across 10,000 pixels

simultaneously, whereas FFKSM’s ray-marching provides each network evaluation with only 0.0156 pixels of supervision on average. This $640,000\times$ difference in supervision density explains why NeuroKin trains $89\times$ faster (33.5 seconds vs. 50 minutes) and generalizes more effectively from limited data (2,000 samples vs. the 12,000 samples required by the original FFKSM paper [?]).

This finding directly challenges the conventional wisdom that 3D reconstruction is a necessary prerequisite for visual self-modeling. For control-oriented tasks requiring only silhouette-level awareness—collision checking, proprioceptive drift correction, and damage detection—the representational overhead of volumetric methods is computational waste. The principle of minimal representation emerges as a design guideline: neural self-models should maintain only the complexity required by downstream tasks.

8.1.2 RQ2: Control Integration — Sub-Millisecond Visual Servoing Enables Dynamic Stabilization

Question: Can a visual self-model operating purely on direct convolutional decoding provide spatial estimates with sufficient accuracy and latency to actively stabilize a Jacobian-based inverse kinematics control loop?

Answer: Yes. Chapter 5 demonstrated that integrating NeuroKin-3D’s stereoscopic spatial estimates (2.39 mm mean error, 0.400 ms latency) into a resolved-rate feedback loop enables real-time closed-loop control. Under nominal conditions, the visual controller converged 15% faster than the proprioceptive baseline (35.9 vs. 42.2 steps, $p = 0.003$) with 18% lower final error (2.3 vs. 2.8 mm, $p = 0.017$). This improvement stems from two factors: direct Cartesian measurement, which bypasses the accumulation of forward kinematic errors, and exponential smoothing ($\alpha = 0.6$), which attenuates high-frequency noise.

Critically, when a severe encoder slip fault was injected at step 50 (simulating calibration drift or thermal deformation), the proprioceptive baseline collapsed to 13% success, while the NeuroKin visual controller maintained 88% success. The recovery was rapid: the controller re-converged to sub-centimeter accuracy within 38.5 steps on average (0.77 seconds). The 12% failure rate was dominated by extreme fault magnitudes ($> 25^\circ$) causing joint limit saturation and workspace exit conditions, not by visual estimation failure.

A Lyapunov stability analysis confirmed that the proportional-only visual servo is exponentially stable. The estimation error δ (bounded by 2.39 mm) enters as an additive disturbance, and the system converges to a residual error ball whose radius is proportional to $\|\delta\|/K_p$. With $K_p = 0.25$, the predicted steady-state error of approximately 9.6 mm matches the observed convergence tolerance, validating the theoretical analysis.

8.1.3 RQ3: Fault Tolerance — Rapid Fine-Tuning Enables Adaptation to Structural Damage

Question: How does the integration of sub-millisecond visual self-estimates alter system resilience under embodiment drift and structural damage?

Answer: The fault injection experiments in Chapter 5 demonstrated that visual estimates enable closed-loop recovery from proprioceptive corruption, but structural damage (bent link, seized joint) initially degrades estimation accuracy because the observed silhouette no longer matches the training distribution. For a bent link scenario (joint 2 physical zero shifted by $+10^\circ$), the network’s prediction error increased from 2.4 mm to 12.7 mm mean error. For a seized joint scenario (joint 3 locked), the error diverged to 18.4 mm.

However, NeuroKin-3D’s efficient training (33.5 seconds from scratch) enables rapid online adaptation. Fine-tuning on just 100 post-damage samples (5 seconds, 10 epochs with learning rate 10^{-4}) restored estimation accuracy to 3.1 mm for the bent link scenario and 4.2 mm for the seized joint scenario. This rapid adaptation capability is enabled by NeuroKin’s dense supervision efficiency, which allows effective learning from limited data.

The closed-loop performance after fine-tuning was robust: 92% success for the bent link scenario and 67% for the seized joint scenario, with the latter limited by the reduced reachable workspace (the locked joint fundamentally constrains feasible targets) rather than estimation accuracy. Unlike approaches requiring extensive offline counterfactual synthesis (e.g., Dream2Fix with 120,000+ simulation cases [?]), NeuroKin’s 5-second fine-tuning cycle suggests a practical pathway for real-world deployment where collecting extensive post-damage data may be infeasible or dangerous.

8.1.4 RQ4: Collective Adaptation — Temporal Stigmergy Validates the Prerequisite for Swarm Coordination

Question: Can the individual computational efficiency of a fast self-model enable decentralized, state-dependent adaptation across a sequential pipeline of agents?

Answer: Yes. Chapter 6 demonstrated that a 100-stage sequential swarm, where agents read and write a shared `factory_state` ledger tracking consecutive failures, achieves 88% post-fault success compared to the baseline’s 13%. The temporal stigmergy mechanism—agents coordinate across time by leaving environmental traces (failure counters) that trigger adaptive responses in subsequent agents—requires no centralized coordination, no explicit inter-agent communication, and no additional training beyond the individual NeuroKin self-model.

The threshold rule ($\tau = 2$ consecutive failures triggers recovery mode) is simple yet effective. A single failure ($\tau = 1$) would trigger recovery mode too aggressively, causing unnecessary conservative behavior after isolated, unrepeatable faults. A higher threshold ($\tau = 3$ or $\tau = 4$) would delay response, allowing cascading failures to propagate. The empirically calibrated $\tau = 2$ balances these competing pressures.

The information efficiency is striking: with at most $\log_2(3) \approx 1.58$ bits of shared information (the 3-state ledger), the NeuroKin swarm achieves a 75 percentage point improvement over the baseline. The median steps to convergence for NeuroKin agents was 34.5, compared to 48.5 for the baseline’s successful runs (a 29% reduction), demonstrating that the recovery mode not only improves success rates but also operational efficiency.

Crucially, the 0.400 ms inference time of NeuroKin-3D is essential for this mechanism. It enables real-time failure detection within a single control cycle, immediate ledger updates before the next agent is instantiated, and rapid adaptation without stale state windows. Without sub-millisecond inference, temporal stigmergy would introduce latency proportional to the number of agents, causing the coordination mechanism to become the bottleneck.

8.2 Technical Contributions Summary

Beyond answering the research questions, this thesis makes four specific technical contributions to the fields of robotic self-modeling, neural representation, and decentralized multi-agent systems:

8.2.1 Contribution 1: Formal Benchmarking of the Latency Paradox

Through rigorous ground-up reproduction and quantitative evaluation of both implicit (FFKSM/NeR) and explicit (K-3DGS) neural rendering self-models on a standardized 2,000-sample dataset, this research formally exposes their control incompatibility. The exact latency metrics—191.6 ms for FFKSM, 27.0 ms for K-3DGS—establish the mathematical limits that necessitate a paradigm shift. The reproduction also uncovered three undocumented implementation challenges (black-screen convergence, kinematic blindness, and weighted loss tuning) that are essential for successful training but omitted from the original publication [?].

8.2.2 Contribution 2: The NeuroKin Direct Sensorimotor Decoder

NeuroKin is a lightweight, fully convolutional direct sensorimotor decoder that completely bypasses volumetric reconstruction. Through extensive empirical validation, NeuroKin achieves 21.88 dB PSNR at 7,400 FPS (0.135 ms latency)—a $1,418\times$ speedup over implicit NeRF baselines with a +4.53 dB quality improvement. The stereoscopic extension, NeuroKin-3D, recovers 3D spatial coordinates with 2.39 mm mean error at 2,500 FPS (0.400 ms latency). The multi-task variant, ResNeuroKin-D, adds depth prediction (21.23 dB PSNR at 2,500 FPS) and induces geometric structure in the latent representation. All architectures are open-sourced for reproducibility.

8.2.3 Contribution 3: Visual Closed-Loop Damage Adaptation

This research successfully synthesizes NeuroKin’s sub-millisecond spatial estimates with a Jacobian-based inverse-kinematics feedback loop. The integration enables dynamic error recovery after severe fault injection (88% success vs. 13% baseline). Lyapunov stability analysis confirms exponential stability, and the empirical demonstration that high-speed visual estimates can override corrupted proprioceptive data proves that sub-millisecond self-modeling enables real-time closed-loop adaptation.

8.2.4 Contribution 4: Temporal Stigmergy for Sequential Swarm Coordination

The validation of stigmergic swarm coordination under compounding mechanical faults demonstrates that ultra-fast individual self-diagnosis is the computational prerequisite for swarm intelligence. The 100-stage sequential swarm with progressive fault injection (fault probability from 1% to 29.5%, fault scale from 0.005 rad to 0.062 rad) achieves 88% post-fault success through a decentralized ledger with only 1.58 bits of shared information. The mechanism requires no centralized coordination, no explicit communication, and no additional training beyond the individual self-model.

8.3 Future Roadmap: From Sequential Validation to Spatial Swarms

The future roadmap is strictly bounded by hardware realities. Because physical deployment was impossible given the temporal constraints of this Master’s degree, the next step is rigorous hardware validation under realistic sensing and calibration conditions. This foundational bridging of the sim-to-real gap will serve as the immediate launching point for the author’s forthcoming doctoral research. Following that, concurrent multi-robot experiments that add collision avoidance and atomic shared-state updates can be effectively prototyped. Those extensions are consistent with the results in this thesis, but they are not claimed as solved here.

8.4 Concluding Remarks

This thesis demonstrates that fast visual self-modeling is useful in practice: it supports real-time closed-loop correction when proprioception is corrupted and improves a sequential multi-agent pipeline through a minimal shared failure ledger. The results do not claim full spatial swarm deployment, but they do establish the computational foundation needed for it.

The central design lesson is to keep the representation as small as the task allows. For

the control and coordination problems studied here, direct decoding is sufficient, volumetric reconstruction is too slow, and a minimal ledger is enough to support temporal stigmergy. Those are the concrete conclusions supported by the experiments in this thesis.